

POSTGRESQL ADMINISTRATION

A Complete Guide for the Oracle-Experienced
Database Architect

Baseline: PostgreSQL 18 · with notes for 15 – 17 and a look ahead to 19

Architecture · Configuration · Security · Storage · MVCC and Vacuum
WAL, Backup and PITR · Replication and HA · Performance · Operations

Prepared for Mukesh

September 2026

Contents at a Glance

Part I — Foundations

1. How to Use This Guide
2. Architecture
3. The Oracle → PostgreSQL Mental Model Map

Part II — Building and Configuring the Environment

4. Installation and Cluster Creation
5. Configuration
6. Connection Management and Pooling
7. Roles, Privileges, and Object Ownership

Part III — Security

8. Authentication

Part IV — Physical Storage, Indexes, and Partitioning

9. How Data Is Stored
10. Indexes
11. Partitioning

Part V — Transactions, Concurrency, and Vacuum

12. MVCC and Isolation
13. Locking
14. VACUUM and Autovacuum

Part VI — WAL, Backup, and Recovery

15. Write-Ahead Logging
16. Backup
17. Point-in-Time Recovery
18. Corruption: Detection and Response

Part VII — Replication and High Availability

19. Physical Streaming Replication
20. Logical Replication and CDC
21. High Availability

Part VIII — Performance

22. The Query Planner
23. Finding the Queries That Matter
24. A Query Tuning Workflow

Part IX — Operations

25. Monitoring
26. Routine Maintenance
27. Upgrades
28. Extensions
29. Managed PostgreSQL in the Cloud
30. Troubleshooting Playbooks
31. Migrating from Oracle

Appendices

- Appendix A — Parameter Quick Reference
- Appendix B — psql
- Appendix C — Oracle View → PostgreSQL Catalog Cross-Reference
- Appendix D — Snippet Library
- Appendix E — Version Feature Matrix
- Appendix F — Further Reading

Detailed Contents

Word builds this table when the document opens. If it appears empty, select all (Ctrl+A) and press F9.

Part I — Foundations

1. How to Use This Guide

1.1 Scope and baseline version

This guide is a working administration reference for PostgreSQL, written for a database professional who already thinks in Oracle terms. The baseline is **PostgreSQL 18** (first released 25 September 2025; current minor at the time of writing is **18.6**). Where behaviour differs meaningfully in **PostgreSQL 15, 16, or 17**, the difference is called out inline. **PostgreSQL 19** was in Beta 3 as of the time of writing, with final release expected September/October 2026; its relevant changes appear in Appendix E rather than in the main body, because they are not yet production material.

Supported major versions and their end-of-life dates:

Major version	First release	Final EOL
18	25 Sep 2025	14 Nov 2030
17	26 Sep 2024	8 Nov 2029
16	14 Sep 2023	9 Nov 2028
15	13 Oct 2022	11 Nov 2027
14	30 Sep 2021	12 Nov 2026

PostgreSQL supports each major version for five years. There are no extended-support or paid-lifecycle options from the community; commercial vendors (EDB, Fujitsu, Percona, cloud providers) sell their own extended support on forks or repackaged builds. Plan major upgrades on a five-year cadence at worst, and preferably every two to three years — the version you are on determines which performance and operational features you can use, and there is no "backport the feature" path in the community project.

1.2 Conventions

- Shell commands are shown for a Red Hat family host (RHEL/Rocky/AlmaLinux 9) using the PGDG packages, with Debian differences noted where they matter.
- SQL is shown as it would be typed in `psql`. Meta-commands (`\d`, `\l`, `\du`) are `psql` client features, not SQL.
- `$PGDATA` refers to the data directory. On PGDG RPM builds this is `/var/lib/pgsql/18/data`; on Debian/Ubuntu it is `/var/lib/postgresql/18/main`.
- **Oracle parallel** boxes translate a PostgreSQL concept into its nearest Oracle equivalent, and — more usefully — name the places where the analogy breaks down. The broken analogies are where production incidents come from.

1.3 The five things that will surprise you

If you read nothing else, read this list. These are the concepts that cause experienced Oracle DBAs the most trouble in their first year with PostgreSQL.

1. **There is no undo tablespace.** Old row versions live in the table itself. This means `DELETE` and `UPDATE` grow tables rather than shrinking them, long-running transactions are far more damaging than in Oracle, and a background process called autovacuum is doing work that Oracle does inside SMON and the undo segments. Autovacuum tuning is the single highest-value operational skill in PostgreSQL.
2. **Transaction IDs are 32-bit and can wrap around.** If vacuum falls far enough behind, the database refuses new writes to protect itself. This has no Oracle equivalent and it has taken down large sites.

3. **A "database" is not a "schema" and connections cannot cross databases.** A PostgreSQL cluster contains multiple databases; a session connects to exactly one and cannot query another without a foreign data wrapper or dblink. The Oracle-style "one database, many schemas" model maps to **one PostgreSQL database with many schemas**, not many PostgreSQL databases.
4. **DDL is transactional.** `BEGIN; ALTER TABLE ...; ROLLBACK;` genuinely rolls back. This is a large operational advantage over Oracle and changes how you write migrations.
5. **The planner has no hints and no stored outlines in core.** You steer the planner with statistics, cost parameters, index design, and query rewriting. `pg_hint_plan` exists as an extension but is not available on most managed services.

2. Architecture

2.1 Process model

PostgreSQL is a **process-per-connection** system. There is no thread pool, no shared server, no MTS. When a client connects, the postmaster forks a new backend process that serves that connection for its lifetime.

```

postmaster (the "postgres" supervisor process)
├── backend process      (one per client connection)
├── backend process
├── background writer    (bgwriter)
├── checkpointer
├── walwriter
├── autovacuum launcher
│   └── autovacuum worker (transient, up to autovacuum_max_workers)
├── logical replication launcher
│   └── logical rep. worker / apply worker / parallel apply worker
├── archiver              (when archive_mode = on)
├── walsender             (one per streaming replica or logical subscriber)
├── walreceiver           (on a standby)
├── io worker             (PG18+, when io_method = worker; io_workers, default 3)
├── stats collector       (in-process since PG15; no separate process)
└── background workers    (extensions: pg_cron, parallel query workers, etc.)

```

Consequences of the process model:

- **Connection establishment is expensive** — process fork plus catalog cache population. Budget 1–5 ms per connection on a warm system, more if the catalog is large. This is why an external connection pooler is effectively mandatory above a few hundred clients.
- **Each backend has private memory** (`work_mem`, `temp_buffers`, catalog and plan caches). A backend that has run a large query holds its peak memory footprint until it exits or the memory context is reset. Long-lived pooled connections can therefore accumulate resident memory; recycling connections periodically is a legitimate tactic.
- **`max_connections` is a hard, restart-required limit** that also sizes several shared memory arrays (lock tables, predicate lock space, PGPROC array). Setting it to 5000 "just in case" wastes shared memory and increases the cost of scanning the proc array.

Oracle parallel. The postmaster is roughly the listener plus PMON. Backends are dedicated server processes — there is no shared-server equivalent, which is why PgBouncer exists. `checkpointer` is CKPT+DBWn's coordinator, `bgwriter` is DBWn, `walwriter` is LGWR, `archiver` is ARCn. Autovacuum has no clean Oracle analogue: it does the work of SMON's space reclamation, plus the automatic statistics gathering job, plus part of what undo retention does for you invisibly.

2.2 Memory architecture

PostgreSQL divides memory into **shared memory** (allocated once at startup) and **per-backend local memory** (allocated on demand).

Shared memory (sized at startup, requires restart to change):

Structure	Parameter	Notes
Shared buffer pool	<code>shared_buffers</code>	The page cache. Default 128 MB — always change this.
WAL buffers	<code>wal_buffers</code>	Default -1 = 1/32 of <code>shared_buffers</code> , capped at 16 MB (segment size).
Lock table	derived from <code>max_locks_per_transaction × (max_connections + max_prepared_transactions)</code>	Sizes the heavyweight lock hash table.
Predicate lock space	<code>max_pred_locks_per_transaction</code>	Only used by SERIALIZABLE.
Replication slots	<code>max_replication_slots</code>	
WAL senders	<code>max_wal_senders</code>	
Visibility/commit-timestamp SLRUs	<code>commit_timestamp_buffers</code> , <code>subtransaction_buffers</code> , <code>transaction_buffers</code> , etc. (individually tunable since PG17; fixed fractions of <code>shared_buffers</code> before)	A classic source of contention on high-subtransaction workloads.

Per-backend memory (allocated per operation, not per backend — this is the part that catches people out):

Structure	Parameter	Default	Notes
Sort/hash working memory	<code>work_mem</code>	4 MB	Allocated per sort, hash, or materialize node , per parallel worker. A single query can use many multiples of it.
Hash-specific multiplier	<code>hash_mem_multiplier</code>	2.0	Hash nodes may use <code>work_mem × hash_mem_multiplier</code> .
Maintenance operations	<code>maintenance_work_mem</code>	64 MB	VACUUM, CREATE INDEX, ALTER TABLE ADD FK, REINDEX.
Autovacuum operations	<code>autovacuum_work_mem</code>	-1 (= <code>maintenance_work_mem</code>)	Multiplied by the number of autovacuum workers.
Temp table buffers	<code>temp_buffers</code>	8 MB	Per session, only allocated if temp tables are used.
Logical decoding	<code>logical_decoding_work_mem</code>	64 MB	Per walsender; spills to disk beyond this.

The practical memory budget for a dedicated server:

```
Total RAM
- OS and filesystem cache reservation (leave 25-50 % for the page cache)
- shared_buffers (typically 25 % of RAM)
- max_connections × work_mem × ~2..4 (worst-case concurrent sort/hash nodes)
- autovacuum_max_workers × autovacuum_work_mem
- per-backend baseline (~5-10 MB × max_connections)
= headroom
```

If that arithmetic goes negative, the Linux OOM killer will eventually resolve it for you, and it usually chooses the postmaster. Set `vm.overcommit_memory = 2` with a sensible `vm.overcommit_ratio`, or at minimum protect the postmaster with `oom_score_adj`.

Oracle parallel. `shared_buffers` is the buffer cache portion of the SGA — but PostgreSQL relies on the OS page cache as a second tier, so unlike Oracle you deliberately do **not** give it most of the machine's RAM. There is no shared pool: plans, cursors, and dictionary caches are private per backend, so there is nothing resembling library cache latch contention, but also no cursor sharing across sessions. `work_mem` is PGA workarea size, except that it is a per-operator limit rather than a per-

process target, and there is no `pga_aggregate_target` ceiling. Nothing enforces a global limit on total working memory — that discipline is yours.

2.3 Storage layout

A PostgreSQL **cluster** is a single `$PGDATA` directory managed by one postmaster on one port.

```
$PGDATA/
├── PG_VERSION          major version number, in text
├── postgresql.conf     main config (editable)
├── postgresql.auto.conf written by ALTER SYSTEM — do not hand-edit
├── pg_hba.conf         host-based authentication rules
├── pg_ident.conf       OS-user → DB-user name maps
├── postmaster.pid      lock file; holds PID, port, socket dir, shmem key
├── base/              one subdirectory per database, named by OID
│   ├── 1/            template1
│   ├── 4/            template0
│   ├── 5/            postgres
│   └── 16384/         your database
│       ├── 16385      a relation's main fork (1 GB segments: 16385, 16385.1, ...)
│       ├── 16385_fsm  free space map
│       ├── 16385_vm   visibility map
│       └── 16385_init  init fork (unlogged relations)
├── global/            cluster-wide catalogs (pg_database, pg_authid, ...)
│   └── pg_control      control file — the single most critical 8 kB in the cluster
├── pg_wal/            write-ahead log segments (16 MB default)
│   └── archive_status/ .ready / .done markers for the archiver
├── pg_xact/           transaction commit status (CLOG)
├── pg_multixact/      multixact (shared row lock) state
├── pg_commit_ts/      commit timestamps, if track_commit_timestamp = on
├── pg_subtrans/       subtransaction parent links
├── pg_tblspc/         symlinks to tablespace locations
├── pg_stat/           permanent statistics files
├── pg_stat_tmp/       transient statistics
├── pg_logical/        logical decoding state, snapshots, mappings
├── pg_repslot/        replication slot state
├── pg_snapshots/      exported snapshots
├── pg_serial/         SERIALIZABLE predicate lock data
├── pg_twophase/       prepared transaction state (2PC)
├── pg_dynshmem/       dynamic shared memory segments
└── pg_notify/         LISTEN/NOTIFY queue
```

Key facts:

- Every relation (table, index, sequence, materialised view, TOAST table) is one or more files named by its `relfilenode`, split into **1 GB segments**.
- The `relfilenode` is *usually* equal to the relation's OID, but diverges after `VACUUM FULL`, `CLUSTER`, `REINDEX`, or `TRUNCATE`. Always resolve it with `pg_relation_filepath('schema.table')`, never by assuming.
- `pg_control` holds the control data: system identifier, checkpoint location, WAL level, block size, and the wraparound counters. Losing or corrupting it means the cluster will not start. It is the reason `pg_resetwal` is a last-resort, data-losing tool and not a repair utility.
- **Do not** put anything of your own in `$PGDATA`, and do not remove files from `pg_wal` by hand. Both are common causes of unrecoverable clusters.

```
-- Where does a relation actually live?
SELECT pg_relation_filepath('public.orders');
-- Size, including indexes and TOAST
SELECT pg_size_pretty(pg_total_relation_size('public.orders')) AS total,
       pg_size_pretty(pg_relation_size('public.orders'))       AS heap_only,
       pg_size_pretty(pg_indexes_size('public.orders'))        AS indexes;
```

Oracle parallel. `$PGDATA` is a combination of `$ORACLE_HOME/dbs` (config), the datafiles, the online redo logs, and the controlfile — all in one directory tree. `pg_control` is the controlfile. `pg_wal` is the online redo log directory, except that segments are files that get recycled rather than a fixed set of groups and members; there is no multiplexing of redo within PostgreSQL, so you get redundancy from the filesystem or storage layer. `base/<oid>/<relfilenode>` is a datafile, but there

is exactly one relation per file set — no multi-object datafiles, no segment/extent header blocks, and no `DBA_EXTENTS` to inspect.

2.4 Logical structure: cluster, database, schema

This is the single most important structural difference from Oracle, and getting it wrong at design time is expensive to correct later.

```
Cluster (one $PGDATA, one port, one postmaster)
├── Database: postgres      (administrative default; do not put application data here)
├── Database: template0    (pristine template, do not connect/modify)
├── Database: template1    (template for CREATE DATABASE; customise carefully)
├── Database: appdb
│   ├── Schema: public
│   ├── Schema: sales
│   │   ├── Table: orders
│   │   └── Index: orders_pkey
│   └── Schema: hr
```

Rules that follow from this:

- **A session belongs to one database.** `\c otherdb` in psql closes the connection and opens a new one. Cross-database queries require `postgres_fdw` or `dblink`.
- **Roles, tablespaces, and replication slots are cluster-wide;** tables, schemas, extensions, and most settings are per-database.
- **Global catalogs** (`pg_database`, `pg_authid`, `pg_tablespace`, `pg_shdescription`) live in `global/` and are shared. Everything else is per-database.
- **search_path** determines unqualified name resolution. Default is `"$user", public` — meaning a schema named after the connecting role is searched first if it exists, then `public`.
- Since **PostgreSQL 15**, the `public` schema no longer grants `CREATE` to `PUBLIC`. Only the database owner can create objects in `public` by default. This broke a great many migration scripts written against PG 14 and earlier. The change applies to newly initialised clusters and newly created databases; a database restored from an older dump keeps whatever grants the dump carried, so audit rather than assume.

Oracle parallel. An Oracle *database* maps to a PostgreSQL *database* (not a cluster). An Oracle *schema*, which is inseparable from a user, maps to a PostgreSQL *schema*, which is a namespace fully decoupled from roles. A PostgreSQL cluster is closest to an Oracle CDB, and its databases to PDBs — but with far weaker isolation: they share memory, background processes, WAL, roles, and the configuration file. There is no per-database resource manager, no per-database `startup/shutdown`, and no `unplug/plugin`. If you need real isolation, run separate clusters. Practical rule for an Oracle migration: **one Oracle database → one PostgreSQL database, one Oracle schema → one PostgreSQL schema.**

2.5 Tablespaces

PostgreSQL tablespaces are far simpler — and far less useful — than Oracle's.

```
CREATE TABLESPACE fastdata LOCATION '/mnt/nvme/pgdata';
ALTER TABLE public.orders SET TABLESPACE fastdata; -- rewrites and locks the table
CREATE INDEX CONCURRENTLY orders_dt_idx ON public.orders (order_date) TABLESPACE fastdata;
ALTER DATABASE appdb SET TABLESPACE fastdata;      -- requires no connections to appdb
```

What they are: a named symlink in `pg_tblspc/` pointing at a directory, plus a default per-object placement.

What they are **not**: a space management unit. There are no datafiles to add, no `autoextend`, no extent management policy, no quotas, no free-space views. A tablespace grows until the filesystem fills. Monitor the filesystem, not the tablespace.

Legitimate uses: placing hot indexes on faster storage, isolating a very large table onto its own volume, and directing temporary files (`temp_tablespaces`) away from the main volume. Illegitimate uses: anything involving capacity management, and any expectation that a tablespace is a recovery or backup unit — it is not, and `pg_basebackup` must capture all tablespaces together with `$PGDATA`.

Oracle parallel. No datafiles, no extents, no `ALTER DATABASE DATAFILE ... RESIZE`, no transportable tablespaces, no read-only tablespaces, no tablespace-level point-in-time recovery. Treat a PostgreSQL tablespace as "a directory I told PostgreSQL about" and you will not be disappointed.

3. The Oracle → PostgreSQL Mental Model Map

3.1 Concept mapping

Oracle	PostgreSQL	Where the analogy breaks
Instance (SGA + background processes)	Cluster + postmaster + shared memory	One instance per cluster; no RAC, no shared-disk multi-instance.
Database	Cluster (\$PGDATA)	A PG cluster holds many databases.
PDB	Database	No isolation, no per-DB start/stop, no plug/unplug.
Schema (== user)	Schema (namespace)	Schemas and roles are fully independent.
User	Role with LOGIN	Roles are cluster-wide and double as groups.
Tablespace + datafiles	Tablespace (a directory)	No space management whatsoever.
Segment / extent / block	Relation file / (none) / page	Fixed 8 kB pages; no extent concept exposed.
Undo tablespace	(none — old row versions in the heap)	The single biggest architectural difference.
Redo log	WAL (<code>pg_wal</code>)	File-based, recycled; no groups/members/multiplexing.
Archived redo	Archived WAL (<code>archive_command</code> / <code>archive_library</code>)	You write the archive command; nothing is built in.
Controlfile	<code>global/pg_control</code>	Single file, no multiplexing.
SPFILE	<code>postgresql.auto.conf</code> (via <code>ALTER SYSTEM</code>)	Plain text, applied after <code>postgresql.conf</code> .
<code>ALTER SYSTEM SET ... SCOPE=BOTH</code>	<code>ALTER SYSTEM SET + pg_reload_conf()</code>	Not all parameters are reloadable; many need restart.
Data Guard (physical)	Streaming replication	Similar model; no broker, no fast-start failover in core.
Data Guard (logical) / GoldenGate	Logical replication / logical decoding	Built-in logical replication is table-level, not full-fidelity.
RMAN	<code>pgBackRest</code> / Barman / <code>pg_basebackup</code>	No catalog in core; use a real backup tool.
Flashback Database / Query	(none) / (none)	PITR is the only rewind; no flashback query, no recycle bin.
AWR / ASH / Statspack	<code>pg_stat_statements</code> + sampling extensions	No built-in historical repository; <code>pg_stat_*</code> is cumulative-since-reset.
SQL Plan Baselines / Profiles	(none in core)	<code>pg_hint_plan</code> extension only.
Optimizer hints	(none in core)	Planner is steered by statistics and cost settings.
DBMS_SCHEDULER	<code>pg_cron</code> extension, or OS cron	Nothing scheduled in core.
Materialized view fast refresh	<code>REFRESH MATERIALIZED VIEW [CONCURRENTLY]</code>	Full refresh only; no incremental refresh in core.
Resource Manager	(none in core)	Only crude knobs: <code>statement_timeout</code> , <code>cgroups</code> , pooler limits.

Oracle	PostgreSQL	Where the analogy breaks
Partitioning option	Declarative partitioning (core, free)	No automatic interval partitioning; no reference partitioning.
Advanced Compression	TOAST compression (pglz/lz4), <code>pg_columnar</code> -style extensions	No OLTP row compression.
TDE	Filesystem/volume encryption, or forked builds (EDB, Fujitsu, Percona)	Not in community core as of PG 18.
Audit Vault / FGA	<code>pgaudit</code> extension	Log-based; no separate audit repository.
V\$ / GV\$ views	<code>pg_stat_*</code> views + catalogs	Cumulative counters, not point-in-time wait interface.
Wait event interface (V\$SESSION.EVENT)	<code>pg_stat_activity.wait_event_type/wait_event</code>	Sampled state, no time accounting per event in core.
SQL*Plus	<code>psql</code>	
Data Pump	<code>pg_dump</code> / <code>pg_restore</code>	Logical only; no transportable tablespaces.
SQL*Loader	<code>COPY</code> , <code>\copy</code> , <code>pg_bulkload</code>	<code>COPY</code> is very fast and should be the default.
DUAL	(omit the FROM clause)	<code>SELECT 1;</code> is legal.
ROWNUM	<code>LIMIT/OFFSET</code> , <code>row_number()</code>	
ROWID	<code>ctid</code>	Not stable across updates or vacuum; never store it.
Sequences + triggers	<code>GENERATED ... AS IDENTITY</code> , <code>serial</code> , sequences	Identity columns are the modern choice.
Packages	Schemas + functions, or extensions	No package state, no package-level privileges.
PL/SQL	PL/pgSQL (plus PL/Python, PL/Perl, PL/v8, ...)	Similar but not compatible; <code>orafce</code> closes some gaps.
Autonomous transaction	(none in core)	Emulate with <code>dblink</code> or a background worker.
Global temporary table	<code>CREATE TEMP TABLE</code> (session-local, created each time)	Temp tables are real catalog objects created per session — high-frequency creation causes catalog bloat.

3.2 SQL and data type mapping

Oracle type	PostgreSQL type	Notes
VARCHAR2(n)	<code>varchar(n)</code> or <code>text</code>	<code>text</code> is not slower; length limits are a constraint, not an optimisation.
CHAR(n)	<code>char(n)</code>	Blank-padded, same semantics; avoid.
NUMBER(p,s)	<code>numeric(p,s)</code>	Exact, arbitrary precision, software arithmetic — slow for heavy math.
NUMBER (no precision)	<code>numeric</code>	Prefer <code>bigint/int/double precision</code> where the range permits; big performance win.
NUMBER(10)	<code>integer</code> / <code>bigint</code>	Map integral columns to native ints wherever possible.
BINARY_FLOAT / BINARY_DOUBLE	<code>real</code> / <code>double precision</code>	
DATE	<code>timestamp(0)</code>	Oracle <code>DATE</code> carries a time component; PostgreSQL <code>date</code> does not. This is a classic migration bug.
TIMESTAMP	<code>timestamp</code>	

Oracle type	PostgreSQL type	Notes
TIMESTAMP WITH TIME ZONE	timestampz	PG stores UTC and converts on I/O; it does not store the original zone.
TIMESTAMP WITH LOCAL TIME ZONE	timestampz	Closest match.
INTERVAL YEAR TO MONTH / DAY TO SECOND	interval	One unified type.
CLOB	text	TOASTed automatically; no LOB locators, no DBMS_LOB.
BLOB	bytea	Or large objects (lo_*) for streaming > 1 GB.
RAW(n)	bytea	
XMLTYPE	xml	
JSON (21c+)	jsonb	jsonb is the right choice; json preserves text exactly but cannot be indexed usefully.
ROWID / UROWID	ctid	Physical, unstable.
BOOLEAN (23ai)	boolean	PG has had it forever.
Nested tables / VARRAY	Arrays (type[])	
Object types	Composite types / domains	
SDO_GEOMETRY	PostGIS geometry	Extension; excellent, and generally considered superior.

Empty string is not NULL. In Oracle, '' and NULL are the same. In PostgreSQL they are different values. Every migrated WHERE col IS NULL predicate and every NVL(col, 'x') needs review.

Case folding is opposite. Oracle folds unquoted identifiers to UPPER; PostgreSQL folds to lower. A table created in Oracle as Orders is ORDERS; in PostgreSQL it is orders. Migration tools that quote every identifier produce "ORDERS", which then requires quoting forever. Fold to lower case at migration time and never quote identifiers again.

3.3 Function mapping quick reference

Oracle	PostgreSQL
NVL(a,b)	coalesce(a,b)
NVL2(a,b,c)	CASE WHEN a IS NOT NULL THEN b ELSE c END
DECODE(...)	CASE ... END
SYSDATE	current_date, localtime, now()
SYSTIMESTAMP	current_timestamp / now()
TRUNC(d, 'MM')	date_trunc('month', d)
ADD_MONTHS(d,n)	d + n * interval '1 month'
MONTHS_BETWEEN(a,b)	manual: extract(year ...) * 12 + extract(month ...)
LAST_DAY(d)	(date_trunc('month', d) + interval '1 month - 1 day')::date
TO_CHAR, TO_DATE, TO_NUMBER	Same names, mostly compatible format masks
SUBSTR, INSTR	substr; position()/strpos() — note INSTR with 4 args has no direct equivalent
LENGTH	length
LISTAGG(x, ',') WITHIN GROUP (ORDER BY y)	string_agg(x, ',' ORDER BY y)
WM_CONCAT	string_agg
CONNECT BY	Recursive CTE (WITH RECURSIVE)

Oracle	PostgreSQL
ROWNUM <= n	LIMIT n
SEQ.NEXTVAL	nextval('seq')
DBMS_OUTPUT.PUT_LINE	RAISE NOTICE
EXECUTE IMMEDIATE	EXECUTE (PL/pgSQL)
BULK COLLECT	Array aggregation, or just process the result set
MERGE	MERGE (PG15+) or INSERT ... ON CONFLICT
DBMS_RANDOM.VALUE	random()
SYS_GUID()	gen_random_uuid(); uuidv7() in PG18+ for time-ordered UUIDs
USER	current_user / session_user
DUAL	omit FROM

The `orafce` extension provides drop-in implementations of many Oracle built-ins (`nv1`, `decode`, `add_months`, `last_day`, `dbms_output`, `dbms_pipe`, `utl_file`, and an Oracle-compatible `date` type). It is worth installing during a migration and worth planning to remove afterwards, because it makes the schema harder for PostgreSQL-native developers to reason about.

Part II — Building and Configuring the Environment

4. Installation and Cluster Creation

4.1 Choosing a distribution

Source	Use when
PGDG community repository (<code>yum.postgresql.org / apt.postgresql.org</code>)	Default choice. Latest minors, all majors, all common extensions.
OS vendor packages (<code>dnf install postgresql-server</code>)	Only if policy forbids third-party repos. Usually an old major, patched slowly.
Source build	You need a non-default block size, a patch, or a custom <code>--with-*</code> option.
EDB Postgres Advanced Server	You need Oracle compatibility (PL/SQL, packages, hierarchical queries) or TDE.
Container images	Development and CI. For production, only with a mature operator (CloudNativePG, Crunchy PGO, Zalando).

4.2 Host preparation (RHEL 9 family)

```
# Kernel and filesystem
# XFS or ext4. XFS is the common choice; avoid ZFS unless you understand the
# interaction between its COW and PostgreSQL's full_page_writes (you can then
# safely set full_page_writes = off on ZFS).
mkfs.xfs -f /dev/nvme1n1
mkdir -p /var/lib/pgsql
echo '/dev/nvme1n1 /var/lib/pgsql xfs noatime,nodiratime 0 0' >> /etc/fstab
mount -a

# Huge pages: size to shared_buffers plus ~10 %.
# For shared_buffers = 64GB with 2MB huge pages: 64*1024/2 = 32768, round up.
```

```

cat >> /etc/sysctl.d/30-postgresql.conf <<'EOF'
vm.nr_hugepages = 34000
vm.swappiness = 1
vm.overcommit_memory = 2
vm.overcommit_ratio = 90
vm.dirty_background_bytes = 67108864
vm.dirty_bytes = 536870912
kernel.shmmax = 68719476736
kernel.shmall = 16777216
net.core.somaxconn = 1024
EOF
sysctl --system

# Disable transparent huge pages (THP) – it causes latency spikes.
echo never > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag
# Persist via a systemd unit or kernel cmdline: transparent_hugepage=never

# I/O scheduler for NVMe
echo none > /sys/block/nvme1n1/queue/scheduler

# Limits
cat > /etc/security/limits.d/postgres.conf <<'EOF'
postgres soft nfile 65536
postgres hard nfile 131072
postgres soft nproc 16384
postgres hard nproc 32768
EOF

```

`vm.dirty_background_bytes` and `vm.dirty_bytes` matter more than most people expect. The Linux defaults are ratio-based and on a large-memory host allow gigabytes of dirty pages to accumulate, which turns a checkpoint into a multi-second stall. Byte-based limits give predictable behaviour.

4.3 Install and initialise

```

dnf -y install https://download.postgresql.org/pub/repos/yum/reposrms/EL-9-x86_64/pgdg-redhat-repo-
latest.noarch.rpm
dnf -qy module disable postgresql
dnf -y install postgresql18-server postgresql18-contrib postgresql18-devel

# Initialise. Note the explicit choices – the defaults are rarely what you want.
export PGSETUP_INITDB_OPTIONS="--data-checksums --encoding=UTF8 --locale=C --lc-collate=C --lc-ctype=C --wal-
segsize=64"
/usr/pgsql-18/bin/postgresql-18-setup initdb

systemctl enable --now postgresql-18

```

Decisions made at `initdb` time that **cannot be changed later without re-initialising**:

Choice	Flag	Guidance
Data checksums	<code>--data-checksums</code>	On. Default in PG18; explicitly request it on 17 and earlier. Since PG18 you can toggle checksums on an offline cluster with <code>pg_checksums</code> , but enabling at <code>initdb</code> is still correct.
Encoding	<code>--encoding=UTF8</code>	UTF8, always.
Locale / collation	<code>--locale</code> , <code>--lc-collate</code>	See below.
WAL segment size	<code>--wal-segsize</code>	16 MB default; 64 MB reduces file churn on high-write systems.
Block size	compile-time <code>--with-blocksize</code>	Leave at 8 kB. Changing it makes you incompatible with every binary tool and extension build.

Collation is a trap. A libc collation (`en_US.UTF-8`) can change its sort order when the operating system's glibc is upgraded, silently corrupting every text index. The mitigations, in order of preference:

1. Use `--locale=C` at the cluster level and apply linguistic collation per column or per query where it is actually needed (`ORDER BY name COLLATE "en_US"`). C collation is also measurably faster and enables the `text_pattern_ops` prefix-match optimisation for `LIKE 'abc%'` without a special operator class.
2. Use ICU collations (`--locale-provider=icu --icu-locale=en-US`), which version the collation and let PostgreSQL warn you on mismatch.
3. If you must use libc collations, record the glibc version and reindex all text indexes after any OS upgrade that changes it.

PostgreSQL 18 added the `PG_UNICODE_FAST` builtin collation, which gives full Unicode semantics with better performance than libc — a good middle ground when C is too blunt.

```
-- Detect a collation version mismatch after an OS upgrade
SELECT datname, datcollate, datcollversion, pg_database_collation_actual_version(oid)
FROM pg_database;
-- If they differ: REINDEX DATABASE appdb; then ALTER DATABASE appdb REFRESH COLLATION VERSION;
```

Oracle parallel. `initdb` is `CREATE DATABASE` at the instance level, and its irreversible choices are analogous to choosing the database character set — get it wrong and the fix is export/import. There is no `ALTER DATABASE CHARACTER SET`.

4.4 Running multiple clusters on one host

Each cluster needs its own `$PGDATA`, port, and service unit.

```
# RHEL: copy the unit and override PGDATA
cp /usr/lib/systemd/system/postgresql-18.service /etc/systemd/system/postgresql-18-reporting.service
mkdir -p /etc/systemd/system/postgresql-18-reporting.service.d
cat > /etc/systemd/system/postgresql-18-reporting.service.d/override.conf <<'EOF'
[Service]
Environment=PGDATA=/var/lib/pgsql/18/reporting
EOF
sudo -u postgres /usr/pgsql-18/bin/initdb -D /var/lib/pgsql/18/reporting --data-checksums --locale=C
# set port = 5433 in that cluster's postgresql.conf
systemctl daemon-reload && systemctl enable --now postgresql-18-reporting
```

Debian/Ubuntu has first-class multi-cluster tooling and you should use it: `pg_createcluster 18 reporting`, `pg_ctlcluster 18 reporting start`, `pg_lsclusters`, `pg_upgradeclass`.

4.5 The postgres OS account and pg_ctl

```
sudo -iu postgres
export PGDATA=/var/lib/pgsql/18/data PATH=/usr/pgsql-18/bin:$PATH

pg_ctl status
pg_ctl start -l /var/lib/pgsql/18/logfile
pg_ctl stop -m fast          # fast = disconnect clients, roll back, checkpoint. Use this.
pg_ctl stop -m smart         # wait for all clients to disconnect. Usually hangs forever.
pg_ctl stop -m immediate     # crash-equivalent; recovery on next start. Emergency only.
pg_ctl reload                # SIGHUP: re-read config
pg_ctl promote               # standby → primary
pg_ctl restart -m fast -t 300
```

Prefer `systemctl` in production so the service manager's view of state stays accurate. Use `pg_ctl` directly only for maintenance clusters and recovery work.

Oracle parallel. `-m fast` is `SHUTDOWN IMMEDIATE`, `-m smart` is `SHUTDOWN NORMAL`, `-m immediate` is `SHUTDOWN ABORT`. Note the inversion of the word "immediate" — this has caused real outages.

5. Configuration

5.1 How configuration is resolved

Precedence, lowest to highest:

1. Compiled-in defaults
2. `postgresql.conf` (and anything it includes)
3. `postgresql.auto.conf` (written by `ALTER SYSTEM`, always read last among files)
4. Command-line options to `postgres`
5. `ALTER DATABASE ... SET`
6. `ALTER ROLE ... SET`
7. `ALTER ROLE ... IN DATABASE ... SET`
8. `SET` in the session / `SET LOCAL` in the transaction / per-function `SET`

```
-- What is the current value and where did it come from?
SELECT name, setting, unit, source, sourcefile, sourceline, pending_restart
FROM pg_settings WHERE name = 'work_mem';

-- All non-default settings – the first thing to look at on an unfamiliar cluster
SELECT name, setting, unit, source
FROM pg_settings
WHERE source NOT IN ('default', 'override')
ORDER BY name;

-- Settings that were changed but need a restart
SELECT name, setting FROM pg_settings WHERE pending_restart;

-- Show the resolved config file contents (PG15+)
SELECT * FROM pg_file_settings WHERE NOT applied;
```

5.2 Parameter contexts

`pg_settings.context` tells you what it takes to change a parameter:

Context	Change requires	Examples
internal	Recompile / initdb	<code>block_size</code> , <code>data_checksums</code> (pre-18)
postmaster	Server restart	<code>shared_buffers</code> , <code>max_connections</code> , <code>wal_level</code> , <code>port</code> , <code>shared_preload_libraries</code>
sighup	<code>pg_reload_conf()</code>	<code>work_mem</code> defaults, <code>log_*</code> , <code>autovacuum_*</code> , <code>checkpoint_*</code> , <code>archive_command</code>
superuser-backend	New connection by superuser	<code>log_connections</code>
backend	New connection	<code>post_auth_delay</code>
superuser	<code>SET</code> by superuser or a role granted the parameter	<code>session_replication_role</code> , <code>log_min_duration_statement</code>
user	<code>SET</code> by anyone	<code>work_mem</code> , <code>statement_timeout</code> , <code>search_path</code>

```
ALTER SYSTEM SET work_mem = '32MB';
SELECT pg_reload_conf();
ALTER SYSTEM RESET work_mem;           -- remove the override
ALTER SYSTEM RESET ALL;                -- empty postgresql.auto.conf
```

Since PostgreSQL 15 you can delegate a single parameter to a non-superuser:

```
GRANT SET ON PARAMETER log_min_duration_statement TO app_support;
GRANT ALTER SYSTEM ON PARAMETER work_mem TO platform_team;
```

Oracle parallel. `postgresql.auto.conf` is the SPFILE and `ALTER SYSTEM SET` is `ALTER SYSTEM SET ... SCOPE=SPFILE` — it never takes effect until reload or restart, which is the opposite of Oracle's `SCOPE=BOTH` default. There is nothing equivalent to `SCOPE=MEMORY`; the closest is `SET` in a session. Do not hand-edit `postgresql.auto.conf`; a syntax error there prevents startup and there is no `CREATE PFILE FROM SPFILE` to fall back on (though a plain-text copy is trivial to keep).

5.3 A production baseline

The following is a starting point for a dedicated OLTP server with **64 GB RAM, 16 vCPU, NVMe storage**. Every value should be revisited against measurements, but these are defensible defaults rather than the shipped ones, which are sized for a laptop.

```
# ---- Connections ----
listen_addresses = '*'
port = 5432
max_connections = 300                      # keep low; use PgBouncer for scale
superuser_reserved_connections = 5
reserved_connections = 5                  # PG16+: for pg_use_reserved_connections role

# ---- Memory ----
shared_buffers = 16GB                     # ~25 % of RAM
effective_cache_size = 48GB               # planner hint: RAM available for caching (~75 %)
work_mem = 32MB                           # per sort/hash node; raise per-session for ETL
hash_mem_multiplier = 2.0
maintenance_work_mem = 2GB               # index builds, vacuum
autovacuum_work_mem = 1GB
temp_buffers = 32MB
huge_pages = try                          # 'on' once you have confirmed nr_hugepages

# ---- WAL and checkpoints ----
wal_level = replica                       # 'logical' if you need logical replication/CDC
wal_compression = zstd                    # PG15+: lz4 | zstd | pglz | on | off
wal_buffers = 64MB
min_wal_size = 2GB
max_wal_size = 16GB                       # bigger = fewer checkpoints = less write amplification
checkpoint_timeout = 15min
checkpoint_completion_target = 0.9
checkpoint_flush_after = 256kB
synchronous_commit = on                  # 'off' trades durability for throughput – see Chapter 15
full_page_writes = on
archive_mode = on
archive_timeout = 300                     # bound RPO on a quiet system
archive_command = 'pgbackrest --stanza=main archive-push %p'

# ---- Replication ----
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 0                         # rely on slots + archive, not on wal_keep_size
hot_standby = on
hot_standby_feedback = off               # 'on' protects standby queries but bloats the primary
max_slot_wal_keep_size = 64GB            # PG13+: cap slot-retained WAL so a dead slot can't fill the disk

# ---- Planner ----
random_page_cost = 1.1                    # NVMe/SSD; 4.0 is the spinning-disk default
seq_page_cost = 1.0
cpu_tuple_cost = 0.01
effective_io_concurrency = 200            # PG18 default is 16; raise for NVMe
maintenance_io_concurrency = 200
default_statistics_target = 100          # raise to 250-1000 on specific skewed columns
jit = on
jit_above_cost = 100000

# ---- Parallelism ----
max_worker_processes = 16
max_parallel_workers = 16
max_parallel_workers_per_gather = 4
max_parallel_maintenance_workers = 4

# ---- Async I/O (PG18+) ----
io_method = worker                        # worker | io_uring | sync
```



```

io_workers = 3                                # raise on high-concurrency sequential-scan workloads
io_combine_limit = 128kB

# ---- Autovacuum ----
autovacuum = on                                # never turn this off
autovacuum_max_workers = 6
autovacuum_naptime = 15s
autovacuum_vacuum_scale_factor = 0.05
autovacuum_vacuum_insert_scale_factor = 0.1
autovacuum_analyze_scale_factor = 0.02
autovacuum_vacuum_cost_limit = 3000           # default 200 is far too low on modern storage
autovacuum_vacuum_cost_delay = 2ms
vacuum_cost_limit = 3000
log_autovacuum_min_duration = 0               # log every autovacuum; invaluable and cheap

# ---- Background writer ----
bgwriter_delay = 100ms
bgwriter_lru_maxpages = 1000
bgwriter_lru_multiplier = 4.0
bgwriter_flush_after = 512kB

# ---- Statement behaviour ----
statement_timeout = 0                         # set per-role, not globally
lock_timeout = 5s
idle_in_transaction_session_timeout = 10min
idle_session_timeout = 0                     # PG14+; careful with poolers
deadlock_timeout = 1s

# ---- Logging ----
logging_collector = on
log_destination = 'stderr,jsonlog'           # jsonlog added in PG15
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_rotation_age = 1d
log_rotation_size = 1GB
log_truncate_on_rotation = on
log_min_duration_statement = 1000            # log statements over 1 s
log_checkpoints = on                         # default on since PG15
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0                           # log every temp file; tells you work_mem is too small
log_line_prefix = '%m [%p] %q%u@%d app=%a %h '
log_statement = 'ddl'
log_error_verbosity = default
log_timezone = 'UTC'
compute_query_id = on                        # links pg_stat_activity/logs to pg_stat_statements

# ---- Extensions ----
shared_preload_libraries = 'pg_stat_statements,auto_explain,pgaudit,pg_cron'
pg_stat_statements.max = 10000
pg_stat_statements.track = top
auto_explain.log_min_duration = 3000
auto_explain.log_analyze = on
auto_explain.log_buffers = on
auto_explain.log_timing = off                 # timing has real overhead; off is usually right
auto_explain.log_nested_statements = on
auto_explain.sample_rate = 0.1

```

5.4 The parameters that matter most, and why

shared_buffers. 25 % of RAM is the standard starting point. Unlike Oracle, more is not automatically better: PostgreSQL uses the OS page cache as a second tier, and a very large `shared_buffers` increases checkpoint work and double-buffers data. Above roughly 25–40 % you usually stop gaining. Measure with `pg_buffercache` and the hit ratio in `pg_stat_database` before growing it.

effective_cache_size. Pure planner input — it allocates nothing. It tells the planner how much data is likely to be cached across `shared_buffers` plus the OS page cache, which drives index-scan versus sequential-scan costing. Set it to about 75 % of RAM. Setting it too low is a common cause of unwanted sequential scans.

work_mem. The most dangerous parameter in PostgreSQL, because the limit is per node per worker, not per query or per session. A query with three hash joins running with four parallel workers can use `work_mem × 3 × 5`. Set a conservative global value and raise it per session or per role for reporting workloads:

```
ALTER ROLE etl_user SET work_mem = '512MB';
ALTER ROLE etl_user SET maintenance_work_mem = '4GB';
-- or per transaction
BEGIN; SET LOCAL work_mem = '1GB'; ...; COMMIT;
```

Use `log_temp_files = 0` to find queries that spill; each spill is a candidate for more `work_mem` or a better index.

max_wal_size. Controls how much WAL accumulates before a checkpoint is forced. Too small and you get frequent checkpoints, each of which re-enables full-page writes and multiplies WAL volume. Watch `pg_stat_checkpoint` (PG17+; `pg_stat_bgwriter` on earlier versions) — if `num_requested` is a meaningful fraction of `num_timed`, `max_wal_size` is too small.

```
-- PG17+
SELECT num_timed, num_requested, write_time, sync_time, buffers_written
FROM pg_stat_checkpoint;
-- PG16 and earlier
SELECT checkpoints_timed, checkpoints_req, checkpoint_write_time, checkpoint_sync_time
FROM pg_stat_bgwriter;
```

random_page_cost. The default of 4.0 encodes a 1990s spinning disk. On NVMe, 1.0–1.2 is realistic. This single change often eliminates a whole class of "why is it doing a seq scan" problems.

default_statistics_target. 100 buckets in the histogram and 100 most-common values. Raise it selectively on columns with skew or high cardinality, not globally — a global raise makes `ANALYZE` and planning slower everywhere:

```
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 1000;
ANALYZE orders;
```

5.5 include directives and configuration management

```
# postgresql.conf
include_dir = 'conf.d'
include_if_exists = 'local.conf'
```

Keep the shipped `postgresql.conf` untouched and manage everything through `conf.d/*.conf` under version control (Ansible, Puppet, Chef). Files in `include_dir` are read in C-locale filename order, so number them: `10-memory.conf`, `20-wal.conf`, `30-logging.conf`. Reserve `ALTER SYSTEM` for emergency changes and reconcile it back into configuration management afterwards — `postgresql.auto.conf` silently overrides everything in your version-controlled files, and drift there is a recurring source of confusion.

6. Connection Management and Pooling

6.1 Why pooling is not optional

Because every connection is an OS process, PostgreSQL's throughput curve peaks at roughly **2–4 × the number of CPU cores** for active connections and then degrades. A thousand mostly-idle application connections cost memory, proc-array scan time, and snapshot computation on every statement.

The rule of thumb for the pool size that actually executes work:

```
connections ≈ (2 × cores) + effective_spindle_count
```

For a 16-core NVMe server that is roughly 32–40 active connections. Everything above that should be queuing in the pooler, not in the database.

6.2 PgBouncer

```
; /etc/pgbouncer/pgbouncer.ini
[databases]
appdb = host=10.0.0.10 port=5432 dbname=appdb pool_size=40
appdb_ro = host=10.0.0.11 port=5432 dbname=appdb pool_size=60

[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 6432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
auth_query = SELECT username, passwd FROM pg_shadow WHERE username = $1

pool_mode = transaction
max_client_conn = 5000
default_pool_size = 40
min_pool_size = 10
reserve_pool_size = 10
reserve_pool_timeout = 3
max_db_connections = 100
server_idle_timeout = 600
server_lifetime = 3600
query_wait_timeout = 30
ignore_startup_parameters = extra_float_digits,options

server_tls_sslmode = verify-full
server_tls_ca_file = /etc/pki/tls/certs/ca.crt
client_tls_sslmode = require
client_tls_cert_file = /etc/pgbouncer/server.crt
client_tls_key_file = /etc/pgbouncer/server.key
```

Pool modes, and what each one breaks:

Mode	Connection returned to pool	Breaks
session	On client disconnect	Nothing. Gives you almost no multiplexing.
transaction	On COMMIT/ROLLBACK	Session state: SET, LISTEN/NOTIFY, advisory locks held across transactions, WITH HOLD cursors, prepared statements (unless PgBouncer 1.21+ with <code>max_prepared_statements</code> set), temp tables.
statement	After every statement	Multi-statement transactions entirely. Only for autocommit-only workloads.

`transaction` mode is the right answer for almost all applications, provided you audit for the broken features above.

PgBouncer 1.21 introduced prepared-statement support in transaction mode (`max_prepared_statements = 200`), which removed the biggest objection for JDBC and asyncpg users.

Monitoring: connect to the `pgbouncer` pseudo-database and run `SHOW POOLS;`, `SHOW STATS;`, `SHOW CLIENTS;`, `SHOW SERVERS;`. The number to alert on is `cl_waiting` in `SHOW POOLS` — clients queued for a server connection.

Alternatives. `pgcat` (Rust, adds load balancing and sharding), `Odyssey` (Yandex, multi-threaded), and `Supavisor` (Elixir). Application-side pools (HikariCP, `pgxpool`) are complementary, not substitutes: they bound one application instance, whereas PgBouncer bounds the database.

Oracle parallel. PgBouncer in transaction mode is Oracle's shared server / connection pooling (DRCP), moved outside the database. The session-state limitations are the same ones that make DRCP unsuitable for applications relying on session context.

6.3 Server-side connection limits

```
ALTER ROLE reporting CONNECTION LIMIT 20;
ALTER DATABASE appdb CONNECTION LIMIT 200;

-- Who is connected, and what are they doing?
SELECT pid, username, application_name, client_addr, backend_start,
       state, wait_event_type, wait_event,
       now() - state_change AS in_state,
       left(query, 120) AS query
FROM pg_stat_activity
WHERE backend_type = 'client backend' AND pid <> pg_backend_pid()
ORDER BY backend_start;

-- Terminate politely, then forcefully
SELECT pg_cancel_backend(pid) FROM pg_stat_activity WHERE pid = 12345; -- cancels the query
SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE pid = 12345; -- kills the session
```

`idle in transaction` sessions are the most dangerous state in PostgreSQL: they hold a snapshot, which blocks vacuum from removing dead rows cluster-wide and drives table bloat. Always set `idle_in_transaction_session_timeout`.

```
-- Find and kill long idle-in-transaction sessions
SELECT pid, username, application_name, now() - xact_start AS xact_age, state
FROM pg_stat_activity
WHERE state = 'idle in transaction' AND now() - xact_start > interval '5 minutes';
```

7. Roles, Privileges, and Object Ownership

7.1 Roles

There are no "users" and "groups" — only roles, which may or may not have `LOGIN`, and which may be granted to each other.

```
-- A login role for an application
CREATE ROLE app_user LOGIN PASSWORD 'xxx' CONNECTION LIMIT 50 VALID UNTIL '2027-01-01';

-- A group role holding privileges
CREATE ROLE app_readwrite NOLOGIN;
CREATE ROLE app_readonly NOLOGIN;

GRANT app_readwrite TO app_user;

-- Attributes
ALTER ROLE app_user WITH NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT NOREPLICATION NOBYPASSRLS;
ALTER ROLE app_user SET search_path = app, public;
ALTER ROLE app_user SET statement_timeout = '30s';
```

Attribute	Meaning
LOGIN	May be used as a connection name
SUPERUSER	Bypasses all permission checks. Grant to nobody in production except break-glass.
CREATEDB / CREATEROLE	Self-explanatory; CREATEROLE is powerful — since PG16 it can only manage roles it created
INHERIT / NOINHERIT	Whether privileges of granted roles apply automatically or require <code>SET ROLE</code>
REPLICATION	May start streaming replication and use replication slots
BYPASSRLS	Ignores row-level security policies
CONNECTION LIMIT <i>n</i>	Concurrent sessions
VALID UNTIL <i>ts</i>	Password expiry (does not disable the role, only the password)

Predefined roles (grant these instead of superuser):

Role	Grants
pg_read_all_data	SELECT on all tables/views/sequences (PG14+)
pg_write_all_data	INSERT/UPDATE/DELETE on all tables (PG14+)
pg_read_all_settings	Read all GUCs including superuser-only
pg_read_all_stats	Read all pg_stat_* including other users' queries
pg_stat_scan_tables	Take ACCESS SHARE locks for monitoring functions
pg_monitor	Union of the three above — the correct role for a monitoring user
pg_signal_backend	Cancel/terminate other users' backends (not superusers')
pg_checkpoint	Run CHECKPOINT (PG15+)
pg_maintain	VACUUM, ANALYZE, CLUSTER, REINDEX, REFRESH MATVIEW on all relations (PG17+)
pg_use_reserved_connections	Use the reserved_connections pool (PG16+)
pg_create_subscription	Create logical replication subscriptions (PG16+)
pg_execute_server_program, pg_read_server_files, pg_write_server_files	Effectively superuser-equivalent — treat as such

7.2 A privilege model that works

The pattern below survives contact with real applications. The key insight is that **object privileges do not apply retroactively to new objects**, so `ALTER DEFAULT PRIVILEGES` is mandatory, and it applies only to objects created by the role you name in `FOR ROLE`.

```
-- 1. Lock down the database and public schema
REVOKE ALL ON DATABASE appdb FROM PUBLIC;
REVOKE ALL ON SCHEMA public FROM PUBLIC;      -- default since PG15
GRANT CONNECT ON DATABASE appdb TO app_readonly, app_readwrite;

-- 2. Owner role that owns all objects; nobody logs in as it
CREATE ROLE app_owner NOLOGIN;
CREATE SCHEMA app AUTHORIZATION app_owner;

-- 3. Usage and object privileges for the two access tiers
GRANT USAGE ON SCHEMA app TO app_readonly, app_readwrite;

GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_readonly;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO app_readwrite;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA app TO app_readwrite;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA app TO app_readwrite;

-- 4. Make it apply to FUTURE objects created by app_owner
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT SELECT ON TABLES TO app_readonly;
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_readwrite;
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT USAGE, SELECT ON SEQUENCES TO app_readwrite;
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT EXECUTE ON FUNCTIONS TO app_readwrite;

-- 5. Login roles inherit from the tier roles
CREATE ROLE app_svc LOGIN PASSWORD 'xxx';
GRANT app_readwrite TO app_svc;
CREATE ROLE bi_svc LOGIN PASSWORD 'xxx';
GRANT app_readonly TO bi_svc;

-- 6. DDL is applied as app_owner
-- psql -U migration_user -c "SET ROLE app_owner; \i migration.sql"
```

Inspecting privileges:

```
\dp app.* -- psql: table ACLs
\ddp -- psql: default privileges
SELECT * FROM information_schema.role_table_grants WHERE table_schema = 'app';
SELECT has_table_privilege('bi_svc', 'app.orders', 'SELECT');
SELECT * FROM pg_roles WHERE rolname NOT LIKE 'pg\_%';
-- Role membership graph
SELECT r.rolname AS member, g.rolname AS granted_role, m.admin_option
FROM pg_auth_members m
JOIN pg_roles r ON r.oid = m.member
JOIN pg_roles g ON g.oid = m.roleid
ORDER BY 1,2;
```

Oracle parallel. Roles behave like Oracle roles crossed with users. `INHERIT` is the difference between a default role and one that needs `SET ROLE`. There is no `GRANT ... WITH ADMIN OPTION` distinction at the object level (that is `WITH GRANT OPTION`), no system privileges beyond the role attributes and predefined roles, no profiles, and no password complexity policy in core (`passwordcheck` is a contrib module; managed services often add their own). Object privileges are not retroactive, which has no Oracle analogue — `ALTER DEFAULT PRIVILEGES` is the fix and it is easy to forget.

7.3 Ownership and its consequences

The owner of an object can drop it, alter it, and grant on it. Dropping a role requires first reassigning or dropping everything it owns:

```
REASSIGN OWNED BY departing_user TO app_owner; -- transfers ownership
DROP OWNED BY departing_user; -- drops remaining privileges and objects it owns
DROP ROLE departing_user;
```

Run `REASSIGN OWNED` / `DROP OWNED` in **every database** — they are per-database operations, while the role is cluster-wide.

Part III — Security

8. Authentication

8.1 pg_hba.conf

Host-based authentication is evaluated **top to bottom, first match wins**. A non-matching rule does not fall through to a later one — if the first matching line's method fails, the connection is rejected. This is the opposite of what most people assume and is the cause of most "why can't I log in" tickets.

#	TYPE	DATABASE	USER	ADDRESS	METHOD	OPTIONS
	local	all	postgres		peer	
	local	all	all		scram-sha-256	
	host	all	all	127.0.0.1/32	scram-sha-256	
	host	all	all	:::1/128	scram-sha-256	
# Application subnet, TLS required						
	hostssl	appdb	app_svc	10.20.0.0/16	scram-sha-256	
	hostssl	appdb	+app_readonly	10.20.0.0/16	scram-sha-256	
# Replication						
	hostssl	replication	replicator	10.20.1.0/24	scram-sha-256	
# Enterprise SSO						
	hostssl	all	+ldap_users	10.30.0.0/16	ldap	ldapserver=ldap.corp.example
			ldappedn="ou=users,dc=corp,dc=example"	ldapbinddn="cn=pgbind,dc=corp,dc=example"	ldapbindpasswd="..."	
			ldapsearchattribute="uid"	ldaptls=1		
# Kerberos / Active Directory						
	hostgssenc	all	all	10.40.0.0/16	gss	include_realm=0 krb_realm=CORP.EXAMPLE
	map	adusers				

```
# Certificate authentication for a service account
hostssl appdb metrics 10.50.0.0/24 cert clientcert=verify-full map=certmap

# Explicit deny for everything else
host all all 0.0.0.0/0 reject
```

Connection types:

Type	Meaning
local	Unix domain socket
host	TCP, encrypted or not
hostssl	TCP with TLS only
hostnossl	TCP without TLS only
hostgssenc	TCP with GSSAPI encryption
hostnogssenc	TCP without GSSAPI encryption

Methods, ranked by what you should actually use:

Method	Use
scram-sha-256	The default choice. Challenge–response, no password on the wire, salted server-side.
cert	Mutual TLS. Excellent for service accounts and replication.
gss	Kerberos / Active Directory. The right answer in a Windows enterprise.
ldap	Works, but sends the password to the LDAP server in a bind — requires <code>ldaptls=1</code> and a TLS connection to PostgreSQL, or you have just moved the plaintext password problem.
peer	Local socket only; matches OS user to DB user. Correct for <code>postgres</code> admin access.
oauth	PG18+; delegated OAuth 2.0 via a validator extension.
trust	No authentication at all. Only for single-user recovery mode on a socket.
md5	Deprecated. Removal announced; migrate to SCRAM.
password	Cleartext. Never.

Special tokens: `all`, `sameuser`, `samerole`, `replication` (matches only physical replication connections; logical replication connects to a real database), `+rolename` (role and its members), `@filename` (include a list).

`pg_ident.conf` maps external identities to database roles:

```
# MAPNAME  SYSTEM-USERNAME  PG-USERNAME
adusers    /^(.*)@CORP\.EXAMPLE$    \1
certmap    /^CN=([a-z]+),OU=svc,.*$  \1
```

Reload and verify:

```
SELECT pg_reload_conf();
-- PG10+: the parsed rules, with any errors
SELECT line_number, type, database, user_name, address, auth_method, error
FROM pg_hba_file_rules ORDER BY line_number;
SELECT * FROM pg_ident_file_mappings; -- PG16+
```

Always check `pg_hba_file_rules` after editing. A syntax error there does **not** prevent a reload from succeeding — it leaves the old rules in place and logs a warning, which is easy to miss until the next restart fails.

8.2 Migrating off MD5

```
-- Check what is in use
SELECT rolname,
       CASE WHEN rolpassword LIKE 'SCRAM-SHA-256$%' THEN 'scram'
            WHEN rolpassword LIKE 'md5%'           THEN 'md5'
            WHEN rolpassword IS NULL                THEN 'none'
            ELSE 'other' END AS method
FROM pg_authid WHERE rolcanlogin;
```

Procedure: set `password_encryption = 'scram-sha-256'`, reload, have every role reset its password (`\password rolename` in psql, which never sends the cleartext to the server log), then change `pg_hba.conf` from `md5` to `scram-sha-256`. Client drivers must support SCRAM — libpq 10+, JDBC 42.2+, npgsql 4+, psycopg2 2.7+.

8.3 TLS

```
ssl = on
ssl_cert_file = '/etc/pki/tls/certs/server.crt'
ssl_key_file  = '/etc/pki/tls/private/server.key'
ssl_ca_file   = '/etc/pki/tls/certs/ca.crt'
ssl_crl_file  = '/etc/pki/tls/certs/crl.pem'
ssl_min_protocol_version = 'TLSv1.3'
ssl_ciphers   = 'HIGH:!aNULL:!MD5'
ssl_tls13_ciphers = 'TLS_AES_256_GCM_SHA384' # PG18+
ssl_prefer_server_ciphers = on
ssl_ecdh_curve = 'prime256v1'
```

Client `sslmode` values, and what they actually protect against:

sslmode	Encrypted	Server authenticated	Protects against
disable	No	No	Nothing
allow / prefer	Maybe	No	Nothing (silently downgrades)
require	Yes	No	Passive eavesdropping only
verify-ca	Yes	CA chain	Eavesdropping; not a CA-signed impostor
verify-full	Yes	CA chain + hostname	Use this.

`prefer` is the libpq default and it is not a security setting — an attacker who can strip the TLS handshake gets a plaintext session. Always set `sslmode=verify-full` with `sslrootcert` in production connection strings.

```
-- Who is connected without TLS?
SELECT a.pid, a.username, a.client_addr, s.ssl, s.version, s.cipher, s.bits
FROM pg_stat_activity a
LEFT JOIN pg_stat_ssl s USING (pid)
WHERE a.backend_type = 'client backend';

SELECT * FROM pg_stat_gssapi; -- GSSAPI/Kerberos session details
```

Rotating certificates requires only `SELECT pg_reload_conf();` — no restart.

8.4 Encryption at rest

PostgreSQL community core has no transparent data encryption as of version 18. Your options:

Approach	Protects against	Cost
LUKS / dm-crypt full-volume encryption	Stolen disks, decommissioned hardware	~2–5 % CPU; transparent
Cloud provider volume encryption (EBS, Azure Disk, OCI Block Volume)	Same	None; usually free and default-on
Filesystem encryption (fscrypt, eCryptfs)	Same	Similar

Approach	Protects against	Cost
pgcrypto column encryption	A DBA reading the table	Large: breaks indexing, range queries, and the planner
Application-side encryption	Everything below the app	Same as above, plus key management
Forked builds with TDE (EDB Advanced Server, Fujitsu Enterprise Postgres, Percona pg_tde)	Same as LUKS, plus per-tablespace granularity and key rotation	Licensing / support contract

For most compliance regimes (PCI-DSS, HIPAA, SOC 2), volume-level encryption plus TLS in transit satisfies the requirement. Reach for `pgcrypto` only for a small number of genuinely sensitive columns, and understand that an encrypted column cannot be range-queried, cannot be indexed for equality unless you use a deterministic scheme (which leaks equality), and will appear in WAL, in backups, and in `pg_stat_statements` if it appears in a literal.

```
CREATE EXTENSION pgcrypto;
-- Symmetric, key supplied per statement (never store the key in the database)
INSERT INTO patients(name, ssn_enc)
VALUES ('A. Smith', pgp_sym_encrypt('123-45-6789', current_setting('app.enc_key')));
SELECT pgp_sym_decrypt(ssn_enc, current_setting('app.enc_key')) FROM patients;
```

Note that anything passed as a SQL literal can end up in `log_statement` output and in `pg_stat_statements`. Pass keys and plaintext as bind parameters, and set `pg_stat_statements.track = 'top'` with parameterised queries.

8.5 Row-level security

RLS gives you per-row access control enforced by the server, which is a genuine improvement over Oracle's VPD in terms of simplicity.

```
ALTER TABLE app.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.orders FORCE ROW LEVEL SECURITY;  -- also applies to the table owner

CREATE POLICY tenant_isolation ON app.orders
  USING (tenant_id = current_setting('app.tenant_id')::bigint)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::bigint);

CREATE POLICY support_read ON app.orders
  FOR SELECT TO support_role
  USING (true);

-- Application sets the context per transaction
BEGIN;
SET LOCAL app.tenant_id = '4711';
SELECT * FROM app.orders;
COMMIT;
```

Points to watch:

- `USING` filters rows that are read; `WITH CHECK` validates rows that are written. Omitting `WITH CHECK` on a policy with `USING` means the `USING` expression is reused for writes.
- Multiple `PERMISSIVE` policies are OR'd; `RESTRICTIVE` policies are AND'd on top.
- Table owners and `BYPASSRLS` roles skip policies unless you use `FORCE ROW LEVEL SECURITY`.
- RLS predicates are added to every query and **can defeat index usage or leak information through timing and error messages** if the policy calls user-defined functions. Keep policies to simple column comparisons against `current_setting()` or `current_user`.
- With a connection pooler in transaction mode, always use `SET LOCAL`, never `SET` — otherwise the tenant context leaks to the next client on that server connection. This is the single most common RLS security bug.

Oracle parallel. RLS is VPD/FGAC. `current_setting()` plays the role of an application context, and `SET LOCAL` is the safe equivalent of a session context in a pooled world. There is no equivalent of Oracle Label Security or Database Vault.

8.6 Auditing with pgaudit

```
shared_preload_libraries = 'pgaudit'
pgaudit.log = 'ddl, role, write'      # read, write, function, role, ddl, misc, misc_set, all
pgaudit.log_catalog = off
pgaudit.log_parameter = on
pgaudit.log_relation = on
pgaudit.log_statement_once = off
pgaudit.log_level = log
```

Object-level auditing lets you audit access to specific tables without logging everything:

```
CREATE ROLE auditor NOLOGIN;
ALTER SYSTEM SET pgaudit.role = 'auditor';
SELECT pg_reload_conf();
GRANT SELECT, UPDATE ON app.patients TO auditor;  -- now those operations are audited
```

`pgaudit` writes to the PostgreSQL log. Ship the logs to a system the DBA cannot alter (syslog to a remote collector, CloudWatch, Splunk) — otherwise the audit trail is not defensible. Set `log_destination = 'jsonlog'` (PG15+) to make parsing reliable.

A lighter-weight alternative for change tracking is a trigger-based history table, or the `temporal_tables/periods` extensions. For "who changed this row and when", triggers writing to an append-only journal table remain the most portable approach.

8.7 Hardening checklist

- `listen_addresses` bound to specific interfaces, never `*` on an internet-facing host.
- No `trust` and no `md5` in `pg_hba.conf`; end with an explicit `reject`.
- `REVOKE ALL ON SCHEMA public FROM PUBLIC` (default in PG15+) and `REVOKE ALL ON DATABASE x FROM PUBLIC`.
- No application role with `SUPERUSER`, `CREATEROLE`, `pg_execute_server_program`, `pg_read_server_files`, or `pg_write_server_files`.
- `password_encryption = scram-sha-256`; passwords rotated and set with `\password`, never with `ALTER ROLE ... PASSWORD 'literal'` (which lands in the log and in `~/.psql_history`).
- TLS on, `ssl_min_protocol_version = TLSv1.2` or higher, clients on `verify-full`.
- `search_path` explicitly set on `SECURITY DEFINER` functions — otherwise a caller can hijack them:

```
CREATE FUNCTION app.do_thing() RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path = app, pg_temp AS $$ ... $$;
```

- `log_connections`, `log_disconnections`, `log_statement = 'ddl'` at minimum.
- Backups encrypted (`pgbackrest --repo1-cipher-type=aes-256-cbc`) and restore-tested.
- OS-level: `$PGDATA` mode 0700, owned by `postgres`, no shell access to the `postgres` account except through audited `sudo`.

Part IV — Physical Storage, Indexes, and Partitioning

9. How Data Is Stored

9.1 The page

Every relation is a sequence of fixed **8 kB pages** (blocks). The page layout:

PageHeaderData (24 bytes) pd_lsn, pd_checksum, pd_flags, pd_lower, pd_upper, pd_special, pd_pagesize	offset 0
ItemIdData array (line pointers, 4 bytes each) [1] -> off=8000 len=120 flags=NORMAL [2] -> off=7850 len=150 flags=DEAD [3] -> redirect to [7] (HOT chain)	grows down
free space	
tuple 3 tuple 2 (dead) tuple 1	grows up
special space (index access methods only)	offset 8192

Each heap tuple carries a 23-byte header plus a null bitmap:

Field	Purpose
t_xmin	Transaction ID that inserted this version
t_xmax	Transaction ID that deleted/updated it (0 if live)
t_cid / t_xvac	Command ID within the transaction
t_ctid	Physical location (block, offset); points to the newer version in an update chain
t_infomask, t_infomask2	Hint bits: committed/aborted, HOT-updated, has nulls, has varwidth, frozen
t_hoff	Header length

`ctid` is the tuple's physical address and changes on every update. It is the closest thing to an Oracle `ROWID`, but it is not stable and must never be persisted.

```
CREATE EXTENSION pageinspect;
SELECT lp, lp_off, lp_flags, t_xmin, t_xmax, t_ctid, t_infomask::bit(16)
FROM heap_page_items(get_raw_page('app.orders', 0));

SELECT * FROM page_header(get_raw_page('app.orders', 0));
```

9.2 Row versions and MVCC on disk

An `UPDATE` in PostgreSQL is physically an insert of a new tuple plus a mark on the old one:

1. New tuple written (same page if there is room, otherwise another page).
2. Old tuple's `t_xmax` set to the updating transaction's XID.
3. Old tuple's `t_ctid` points at the new one.

The old version stays on disk until vacuum removes it. Consequences:

- **Tables grow on UPDATE and DELETE.** Space is reclaimed for reuse by vacuum, but the file does not shrink unless you run `VACUUM FULL`, `CLUSTER`, or `pg_repack`.
- **Every index on the table must be updated** on an update, even if the indexed column did not change — unless HOT applies (below).
- **A long-running transaction anywhere in the cluster prevents vacuum from removing rows** newer than its snapshot, in every table.

HOT (Heap-Only Tuple) updates are the optimisation that makes OLTP workloads viable. If the update does not change any indexed column *and* the new version fits on the same page, PostgreSQL chains the versions within the page via a redirect line pointer and leaves the indexes untouched. HOT updates are dramatically cheaper.

To encourage HOT:

```
ALTER TABLE app.sessions SET (fillfactor = 80); -- reserve 20 % per page for updates
-- then rewrite so the setting takes effect on existing pages
VACUUM FULL app.sessions; -- or pg_repack, online
```

```
-- HOT ratio per table: high n_tup_hot_upd relative to n_tup_upd is what you want
SELECT relname,
       n_tup_upd, n_tup_hot_upd,
       round(100.0 * n_tup_hot_upd / nullif(n_tup_upd,0), 1) AS hot_pct,
       n_tup_newpage_upd                                     -- PG16+
FROM pg_stat_user_tables
WHERE n_tup_upd > 10000
ORDER BY hot_pct NULLS LAST;
```

`fillfactor` defaults to 100 for tables (90 for B-tree indexes). Lower it to 70–85 for tables with heavy in-place updates on non-indexed columns; leave it at 100 for append-only tables, where lowering it just wastes space.

Oracle parallel. Oracle updates rows in place and writes the before-image to undo. PostgreSQL writes a new row and leaves the old one. This is why Oracle needs undo retention tuning and PostgreSQL needs vacuum tuning; it is the same problem — how long to keep old versions — solved in different places. `PCTFREE` maps to `100 - fillfactor`. Row migration/chaining in Oracle is roughly the non-HOT update case here.

9.3 TOAST

Values too large for a page are moved out of line. The threshold is `TOAST_TUPLE_THRESHOLD`, about 2 kB (a quarter of a page).

Storage strategies per column:

Strategy	Compress	Out-of-line	Default for
PLAIN	No	No	Fixed-width types (int, timestamp)
EXTENDED	Yes	Yes	text, jsonb, bytea, numeric
EXTERNAL	No	Yes	—
MAIN	Yes	Only if unavoidable	—

```
ALTER TABLE app.docs ALTER COLUMN payload SET STORAGE EXTERNAL; -- skip compression
ALTER TABLE app.docs ALTER COLUMN payload SET COMPRESSION lz4; -- PG14+: pglz | lz4
SET default_toast_compression = 'lz4'; -- cluster/session default
```

`lz4` is roughly 3–5× faster to compress and decompress than the historical `pglz` at a modest cost in ratio; it is the right default for most workloads on PG14+. Use `EXTERNAL` for data that is already compressed (JPEG, gzip, Parquet blobs) — compressing it again wastes CPU and can make it larger.

Practical notes:

- The TOAST table is a separate relation (`pg_toast.pg_toast_<oid>`) with its own index and its own vacuum needs. `pg_total_relation_size()` includes it; `pg_relation_size()` does not.
- Retrieving a TOASTed value costs extra I/O. `SELECT *` on a wide table with large `jsonb` columns is much more expensive than selecting the columns you need. Column order in the `SELECT` list does not matter; column *selection* does.
- Substring access on a TOASTed `text` value can avoid detoasting the whole value (`substr(col, 1, 100)` is optimised for `EXTERNAL` storage).
- Maximum field size is 1 GB regardless of TOAST. For genuinely large binaries, use large objects (`lo_import`, `lo_get`) or object storage with a pointer in the table — the latter is almost always the better architecture.

9.4 Free space map and visibility map

Each relation has up to three forks:

Fork	File	Purpose
main	<code><relfilenode></code>	The data
FSM	<code><relfilenode>_fsm</code>	Per-page free space, so inserts find a page without scanning
VM	<code><relfilenode>_vm</code>	Two bits per page: all-visible, all-frozen

The **visibility map** is what makes index-only scans possible: if every tuple on a page is visible to all transactions, an index scan can return the indexed columns without visiting the heap. It is maintained by vacuum, which is one more reason vacuum matters for read performance, not just for space.

```
CREATE EXTENSION pg_visibility;
SELECT * FROM pg_visibility_map_summary('app.orders');
-- all_visible / all_frozen page counts; low all_visible means index-only scans won't help
SELECT relname, relallvisible, relpages,
       round(100.0*relallvisible/nullif(relpages,0),1) AS pct_all_visible
FROM pg_class WHERE relkind = 'r' AND relpages > 1000 ORDER BY 4;
```

9.5 Bloat: measuring and fixing

Bloat is the space occupied by dead tuples and by pages that vacuum has emptied but not returned to the filesystem.

```
-- Quick signal from the statistics collector
SELECT schemaname, relname,
       n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/nullif(n_live_tup+n_dead_tup,0),1) AS dead_pct,
       last_vacuum, last_autovacuum, last_analyze, last_autoanalyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_pct DESC;

-- Accurate measurement (takes a full scan; run off-peak)
CREATE EXTENSION pgstattuple;
SELECT * FROM pgstattuple('app.orders');
SELECT * FROM pgstatindex('app.orders_pkey');
-- Sampled, much cheaper on large tables
SELECT * FROM pgstattuple_approx('app.orders');
```

Remediation, in increasing order of disruption:

Tool	Lock	Notes
VACUUM	SHARE UPDATE EXCLUSIVE	Reclaims space for reuse; only truncates trailing empty pages
REINDEX INDEX CONCURRENTLY	Brief SHARE UPDATE EXCLUSIVE	Rebuilds a bloated index online (PG12+)

Tool	Lock	Notes
<code>pg_repack</code>	Brief <code>ACCESS EXCLUSIVE</code> at start and end	Rebuilds table + indexes online; needs 2× disk; the standard answer
<code>pg_squeeze</code>	Brief exclusive	Logical-decoding-based alternative to <code>pg_repack</code>
<code>CLUSTER</code>	<code>ACCESS EXCLUSIVE</code>	Rewrites in index order; full outage on that table
<code>VACUUM FULL</code>	<code>ACCESS EXCLUSIVE</code>	Rewrites to a new file; full outage; needs 2× disk

```
pg_repack -h localhost -d appdb -t app.orders --jobs=4 --no-superuser-check
pg_repack -d appdb --only-indexes -t app.orders
```

Preventing bloat is cheaper than fixing it: tune autovacuum to keep up (Chapter 14), kill long transactions, avoid `UPDATE`-heavy designs on wide tables, and consider partitioning so that old data can be dropped rather than deleted.

10. Indexes

10.1 Index types and when to use each

Type	Good for	Notes
B-tree	Equality, ranges, sorting, uniqueness, <code>LIKE 'prefix%'</code> with <code>text_pattern_ops</code> or C collation	The default. ~95 % of indexes. Supports index-only scans, deduplication (PG13+), and skip scan on leading columns (PG18+).
Hash	Equality only	WAL-logged and crash-safe since PG10. Slightly smaller/faster than B-tree for equality on large values, but no ordering, no uniqueness, no multicolumn. Rarely worth it.
GIN	Multi-value columns: <code>jsonb</code> , arrays, full-text <code>tsvector</code> , trigram	Large, slow to update; use <code>fastupdate</code> and tune <code>gin_pending_list_limit</code> . Parallel builds in PG18+.
GIST	Geometric, range types, nearest-neighbour, exclusion constraints, PostGIS	Lossy and extensible; the framework behind PostGIS.
SP-GiST	Non-balanced structures: quadtrees, radix trees, IP prefixes, phone numbers	Niche but excellent when it fits.
BRIN	Very large tables with strong physical/logical correlation (time-series appended in order)	Tiny (kilobytes for a terabyte table). Only useful when correlation is high. <code>pages_per_range</code> is the key knob.
Bloom (extension)	Multi-column equality where any subset of columns may be queried	Probabilistic; returns false positives that are rechecked.

```
-- B-tree variants
CREATE INDEX CONCURRENTLY orders_cust_date_idx ON app.orders (customer_id, order_date DESC);
CREATE UNIQUE INDEX CONCURRENTLY orders_ext_key ON app.orders (external_id) WHERE external_id IS NOT NULL;
CREATE INDEX CONCURRENTLY orders_lower_email ON app.orders (lower(email)); -- expression
CREATE INDEX CONCURRENTLY orders_open_idx ON app.orders (order_date) WHERE status = 'OPEN'; -- partial
CREATE INDEX CONCURRENTLY orders_covering ON app.orders (customer_id) INCLUDE (total, status); -- covering

-- GIN on jsonb: jsonb_path_ops is smaller and faster for @> but supports fewer operators
CREATE INDEX CONCURRENTLY docs_data_gin ON app.docs USING gin (data jsonb_path_ops);
-- Trigram search
CREATE EXTENSION pg_trgm;
CREATE INDEX CONCURRENTLY cust_name_trgm ON app.customers USING gin (name gin_trgm_ops);
-- Full text
CREATE INDEX CONCURRENTLY docs_fts ON app.docs USING gin (to_tsvector('english', body));

-- BRIN on an append-ordered time column
CREATE INDEX CONCURRENTLY events_ts_brin ON app.events USING brin (created_at) WITH (pages_per_range = 32);
```

```
-- Exclusion constraint: no two reservations for the same room overlapping in time
CREATE EXTENSION btree_gist;
ALTER TABLE app.reservations
  ADD CONSTRAINT no_overlap EXCLUDE USING gist (room_id WITH =, during WITH &&);
```

10.2 Building indexes without an outage

```
CREATE INDEX CONCURRENTLY ...; -- two table scans, no ACCESS EXCLUSIVE lock
DROP INDEX CONCURRENTLY ...;
REINDEX INDEX CONCURRENTLY ...; -- PG12+
REINDEX TABLE CONCURRENTLY app.orders;
```

CONCURRENTLY cannot run inside a transaction block, takes roughly twice as long, and **can fail and leave an invalid index behind**:

```
SELECT c.relname, i.indisvalid, i.indisready
FROM pg_index i JOIN pg_class c ON c.oid = i.indexrelid
WHERE NOT i.indisvalid;
-- Clean up and retry
DROP INDEX CONCURRENTLY app.orders_bad_idx;
```

An invalid index is still maintained on writes but not used by the planner — the worst of both worlds. Always check after a concurrent build, especially in automated migrations.

CREATE INDEX CONCURRENTLY waits for all transactions that started before it, in the same database, to finish — twice. A single long-running transaction will stall it indefinitely.

10.3 Finding the indexes you need and the ones you don't

```
-- Unused indexes (reset stats first and let a full business cycle pass, including month-end)
SELECT s.schemaname, s.relname AS table, s.indexrelname AS index,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS size,
       s.idx_scan, s.last_idx_scan -- last_idx_scan: PG16+
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan = 0
      AND NOT i.indisunique
      AND NOT i.indisprimary
      AND pg_relation_size(s.indexrelid) > 10*1024*1024
ORDER BY pg_relation_size(s.indexrelid) DESC;

-- Duplicate / redundant indexes (same leading columns)
SELECT indrelid::regclass AS table,
       array_agg(indexrelid::regclass) AS indexes,
       indkey
FROM pg_index
GROUP BY indrelid, indkey
HAVING count(*) > 1;

-- Tables taking heavy sequential scans
SELECT schemaname, relname, seq_scan, seq_tup_read,
       seq_tup_read / nullif(seq_scan,0) AS avg_rows_per_seq_scan,
       idx_scan, n_live_tup
FROM pg_stat_user_tables
WHERE seq_scan > 1000 AND n_live_tup > 100000
ORDER BY seq_tup_read DESC;

-- Index bloat and health
SELECT indexrelname, pg_size_pretty(pg_relation_size(indexrelid)) AS size,
       idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes ORDER BY pg_relation_size(indexrelid) DESC LIMIT 20;
```

Every index costs write throughput, WAL volume, vacuum time, and disk. Dropping unused indexes is one of the highest-return maintenance activities and is routinely skipped.

Also worth knowing: **hypopg** lets you create hypothetical indexes and run **EXPLAIN** against them without building anything.

```
CREATE EXTENSION hypopg;
SELECT * FROM hypopg_create_index('CREATE INDEX ON app.orders (status, order_date)');
EXPLAIN SELECT * FROM app.orders WHERE status='OPEN' ORDER BY order_date LIMIT 50;
SELECT hypopg_reset();
```

10.4 Index-only scans

For an index-only scan the planner needs (a) every column referenced by the query to be in the index (key columns or `INCLUDE` columns) and (b) the visibility map to say the pages are all-visible. If the second condition fails, you get an index scan with heap fetches instead — which is why a table that is not being vacuumed loses index-only scans.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, total FROM app.orders WHERE customer_id = 42;
-- Index Only Scan using orders_covering ... Heap Fetches: 0      <- good
-- Index Only Scan using orders_covering ... Heap Fetches: 8134   <- vacuum this table
```

Oracle parallel. `CREATE INDEX CONCURRENTLY` is `CREATE INDEX ... ONLINE`. Covering indexes with `INCLUDE` match Oracle's index-organised access for a subset of columns; index-only scans are index-only access, but conditioned on the visibility map, which has no Oracle equivalent. There are no bitmap indexes (PostgreSQL builds bitmaps on the fly in a [Bitmap Heap Scan](#), which is a runtime structure and not stored), no reverse-key indexes, no function-based index limitations, and no index-organised tables — `CLUSTER` orders a heap once but does not maintain the order.

11. Partitioning

11.1 Declarative partitioning

```
-- RANGE: the common case for time-series
CREATE TABLE app.events (
    event_id    bigint GENERATED ALWAYS AS IDENTITY,
    tenant_id   bigint          NOT NULL,
    created_at  timestampz      NOT NULL,
    payload     jsonb,
    PRIMARY KEY (event_id, created_at)      -- partition key must be in every unique key
) PARTITION BY RANGE (created_at);

CREATE TABLE app.events_2026_09 PARTITION OF app.events
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE app.events_2026_10 PARTITION OF app.events
    FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
CREATE TABLE app.events_default PARTITION OF app.events DEFAULT;

-- LIST
CREATE TABLE app.customers (id bigint, region text NOT NULL, ...)
    PARTITION BY LIST (region);
CREATE TABLE app.customers_emea PARTITION OF app.customers FOR VALUES IN ('EMEA','UK');

-- HASH: even distribution when there is no natural range
CREATE TABLE app.sessions (id uuid, ...) PARTITION BY HASH (id);
CREATE TABLE app.sessions_0 PARTITION OF app.sessions FOR VALUES WITH (MODULUS 8, REMAINDER 0);

-- Sub-partitioning
CREATE TABLE app.events_2026_09 PARTITION OF app.events
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01')
    PARTITION BY HASH (tenant_id);
```

11.2 Operations

```
-- Detach without a long lock (PG14+), then drop or archive
ALTER TABLE app.events DETACH PARTITION app.events_2025_09 CONCURRENTLY;
DROP TABLE app.events_2025_09;

-- Attach an already-populated table; add a matching CHECK first to avoid a full validation scan
ALTER TABLE app.events_2026_11 ADD CONSTRAINT part_range
    CHECK (created_at >= '2026-11-01' AND created_at < '2026-12-01') NOT VALID;
ALTER TABLE app.events_2026_11 VALIDATE CONSTRAINT part_range;  -- scans, but no exclusive lock
```



```

ALTER TABLE app.events ATTACH PARTITION app.events_2026_11
FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');           -- now instant

-- Indexes: create on the parent and it cascades to all partitions
CREATE INDEX ON app.events (tenant_id, created_at);
-- Online variant: build CONCURRENTLY per partition, then attach to an invalid parent index
CREATE INDEX events_tenant_idx ON ONLY app.events (tenant_id);
CREATE INDEX CONCURRENTLY events_2026_09_tenant_idx ON app.events_2026_09 (tenant_id);
ALTER INDEX events_tenant_idx ATTACH PARTITION events_2026_09_tenant_idx;
-- repeat for every partition; the parent index becomes valid when all are attached

```

11.3 What partitioning does and does not give you

Gives you:

- **Partition pruning** at plan time and at execution time (`enable_partition_pruning`, on by default) — the query only touches relevant partitions.
- **Cheap bulk deletion**: `DROP TABLE partition` instead of `DELETE`, which avoids generating dead tuples entirely.
- **Smaller per-partition indexes** and per-partition vacuum, which keeps maintenance windows bounded.
- **Partition-wise joins and aggregates** (`enable_partitionwise_join`, `enable_partitionwise_aggregate` — both **off** by default; turn them on deliberately, they cost planning time and memory).

Does not give you:

- Automatic partition creation. Use `pg_partman`, or a `pg_cron` job, or your own scheduler. Nothing in core creates next month's partition.
- Global indexes. Every index is local. A unique constraint must include the partition key.
- Foreign keys *referencing* a partitioned table were only added in PG12; referencing *from* a partitioned table works, but be aware of the per-partition trigger cost.
- Automatic row movement across partitions on update — that works (PG11+) but is implemented as delete+insert and is expensive.

Practical guidance: partition when a table exceeds roughly 100 GB, or when you need to expire data by time, or when maintenance on the whole table no longer fits a window. Do not partition a 5 GB table because it "might grow" — you pay planning cost on every query for partitions you do not need. Keep the partition count in the hundreds, not the tens of thousands; planning time grows with partition count even with pruning.

`pg_partman` automates the whole lifecycle:

```

CREATE EXTENSION pg_partman SCHEMA partman;
SELECT partman.create_parent(
    p_parent_table => 'app.events',
    p_control      => 'created_at',
    p_interval     => '1 month',
    p_premake      => 4);
UPDATE partman.part_config
SET retention = '13 months', retention_keep_table = false
WHERE parent_table = 'app.events';
-- Then schedule partman.run_maintenance_proc() via pg_cron
SELECT cron.schedule('partman', '@hourly', $$CALL partman.run_maintenance_proc()$$);

```

```

-- Inspect the partition tree
SELECT relid::regclass, parentrelid::regclass, level, isleaf
FROM pg_partition_tree('app.events');
SELECT pg_size_pretty(sum(pg_total_relation_size(relid))) FROM pg_partition_tree('app.events');
-- Partition bounds
SELECT c.relname, pg_get_expr(c.relpartbound, c.oid)
FROM pg_class c JOIN pg_inherits i ON i.inhrelid = c.oid
WHERE i.inhparent = 'app.events'::regclass ORDER BY 1;

```

Oracle parallel. Range, list, hash, and composite partitioning all map directly. What is missing: interval partitioning (use `pg_partman`), reference partitioning, system partitioning, global indexes, partition exchange with index maintenance (`ATTACH/DETACH` is the equivalent and is very good), and the `PARTITION BY` clauses on indexes. Partition pruning corresponds

to partition elimination and works well, including at execution time for parameterised plans. Unlike Oracle, partitioning is core functionality with no separate licence.

Part V — Transactions, Concurrency, and Vacuum

12. MVCC and Isolation

12.1 How visibility is decided

Every transaction gets a **transaction ID (XID)** when it first writes. Read-only transactions get a *virtual* XID and consume no real one. A tuple is visible to a snapshot if its `xmin` committed before the snapshot and its `xmax` is either zero, aborted, or later than the snapshot.

A snapshot is (`xmin`, `xmax`, `xip_list`): the oldest still-running XID, the next XID to be assigned, and the list of in-progress XIDs.

```
SELECT txid_current();           -- assigns an XID (use sparingly)
SELECT pg_current_xact_id_if_assigned(); -- PG13+, does not assign one
SELECT pg_current_snapshot();
SELECT pg_snapshot_xmin(pg_current_snapshot());
```

Commit status lives in `pg_xact` (CLOG). Because checking CLOG for every tuple would be expensive, PostgreSQL caches the result in **hint bits** in the tuple header. The first reader of a page after a commit therefore performs writes (setting hint bits dirties the page) — which is why a `SELECT` immediately after a bulk load can generate surprising amounts of I/O and WAL (with `wal_log_hints` or checksums on).

12.2 Isolation levels

Level	Dirty read	Non-repeatable read	Phantom read	Serialization anomaly
Read Uncommitted	not possible in PG	possible	possible	possible
Read Committed (default)	no	possible	possible	possible
Repeatable Read	no	no	no	possible
Serializable	no	no	no	no

PostgreSQL never allows dirty reads; `READ UNCOMMITTED` behaves as `READ COMMITTED`.

- **Read Committed** takes a new snapshot at the start of **each statement**. Within a single `UPDATE`, if a row was concurrently modified, PostgreSQL re-evaluates the `WHERE` clause against the new version rather than failing — this is a real difference from Oracle, which raises `ORA-08177`-style errors far less often but also re-reads differently.
- **Repeatable Read** takes one snapshot at the first statement and keeps it. PostgreSQL's implementation also prevents phantoms (it is really snapshot isolation), which is stronger than the SQL standard requires. Concurrent updates to the same row raise `ERROR: could not serialize access due to concurrent update (SQLSTATE 40001)` — **the application must retry**.
- **Serializable** adds Serializable Snapshot Isolation (SSI): predicate locks track read/write dependencies and abort transactions that would produce a non-serializable outcome, with SQLSTATE 40001. It is genuinely serializable, and unlike two-phase locking it does not block readers. Cost: predicate lock memory (`max_pred_locks_per_transaction`) and a retry loop in the application.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
-- or
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN; SET TRANSACTION READ ONLY DEFERRABLE; -- serializable read-only that never aborts
```

Any application using `REPEATABLE READ` or `SERIALIZABLE` **must** implement a retry loop on SQLSTATE 40001 and 40P01 (deadlock). This is not optional and is the most common reason serializable deployments fail.

Oracle parallel. Oracle's `READ COMMITTED` and PostgreSQL's are close cousins. Oracle's `SERIALIZABLE` is snapshot isolation (equivalent to PostgreSQL's `REPEATABLE READ`), while PostgreSQL's `SERIALIZABLE` is genuinely serializable via SSI — strictly stronger than Oracle's. There is no `ORA-01555 snapshot too old` in PostgreSQL, because old versions are kept in the table rather than in a recycled undo segment: instead of a query failing, the table bloats. Choose your poison.

12.3 Explicit row locking

```
SELECT * FROM app.orders WHERE id = 1 FOR UPDATE;           -- exclusive
SELECT * FROM app.orders WHERE id = 1 FOR NO KEY UPDATE;    -- allows FK checks to proceed
SELECT * FROM app.orders WHERE id = 1 FOR SHARE;            -- what FK checks take
SELECT * FROM app.orders WHERE id = 1 FOR KEY SHARE;

SELECT * FROM q WHERE ready ORDER BY id
FOR UPDATE SKIP LOCKED LIMIT 10;    -- the correct pattern for a work queue
SELECT * FROM app.orders WHERE id = 1 FOR UPDATE NOWAIT;    -- fail immediately if locked
```

`SKIP LOCKED` makes PostgreSQL an entirely reasonable job queue for moderate volumes, and is the single most useful locking feature for application developers.

12.4 Transaction ID wraparound

XIDs are 32-bit. The visible range is 2^{31} (about 2.1 billion) transactions ahead and behind. To keep old rows visible forever, vacuum **freezes** tuples — marking them as unconditionally visible — before their age approaches the limit.

If freezing falls too far behind:

- At `vacuum_failsafe_age` (default 1.6 billion) autovacuum enters failsafe mode: it ignores cost delays and skips index cleanup to freeze as fast as possible.
- At **2 billion** (`autovacuum_freeze_max_age` cannot exceed this) the cluster starts refusing new write transactions with `ERROR: database is not accepting commands to avoid wraparound data loss`.
- Recovery requires a single-user-mode `VACUUM`, which can take hours or days on a large database.

Monitor this. Always.

```
-- Percentage of the way to wraparound, per database
SELECT datname,
       age(datfrozenxid) AS xid_age,
       round(100.0 * age(datfrozenxid) / 2147483648, 2) AS pct_to_wraparound,
       2147483648 - age(datfrozenxid) AS xids_remaining
FROM pg_database ORDER BY 2 DESC;

-- Worst tables
SELECT c.oid::regclass AS relation,
       age(c.relfrozenxid) AS xid_age,
       mxid_age(c.relminmxid) AS mxid_age,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS size
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r', 'm', 't') AND n.nspname NOT IN ('pg_catalog', 'information_schema')
ORDER BY age(c.relfrozenxid) DESC LIMIT 20;

-- Multixact age matters too, and is often overlooked
SELECT datname, mxid_age(datminmxid) FROM pg_database ORDER BY 2 DESC;
```

Alert at 500 million (`age(datfrozenxid) > 500000000`), page at 1 billion. The usual root causes of runaway XID age are: a long-running transaction, an abandoned replication slot, an abandoned prepared transaction (2PC), `hot_standby_feedback` on a standby with a long query, or autovacuum being unable to keep up (or having been switched off "temporarily").

```
-- The four things that hold back the oldest visible XID
SELECT 'long transaction' AS reason, pid::text AS id, now()-xact_start AS age
FROM pg_stat_activity WHERE xact_start IS NOT NULL
UNION ALL
SELECT 'replication slot', slot_name, NULL FROM pg_replication_slots WHERE NOT active
UNION ALL
SELECT 'prepared xact', gid, now()-prepared FROM pg_prepared_xacts
UNION ALL
SELECT 'standby feedback', client_addr::text, NULL FROM pg_stat_replication
ORDER BY 3 DESC NULLS LAST;
```

Oracle parallel. There is no Oracle equivalent. Oracle's SCN is 48-bit and effectively never wraps in practice. This is a genuinely new failure mode to manage, and it is the one that most often takes an unattended PostgreSQL cluster down hard.

13. Locking

13.1 Table-level lock modes

Eight modes, from weakest to strongest. Two modes conflict if the matrix says so; a `SELECT` (ACCESS SHARE) conflicts only with `ACCESS EXCLUSIVE`.

Mode	Taken by	Conflicts with
ACCESS SHARE	SELECT	ACCESS EXCLUSIVE
ROW SHARE	SELECT FOR UPDATE/SHARE	EXCLUSIVE, ACCESS EXCLUSIVE
ROW EXCLUSIVE	INSERT, UPDATE, DELETE, MERGE	SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
SHARE UPDATE EXCLUSIVE	VACUUM, ANALYZE, CREATE INDEX CONCURRENTLY, most ALTER TABLE variants	itself and everything stronger
SHARE	CREATE INDEX (non-concurrent)	ROW EXCLUSIVE and stronger
SHARE ROW EXCLUSIVE	CREATE TRIGGER, some ALTER TABLE	itself and stronger
EXCLUSIVE	REFRESH MATERIALIZED VIEW CONCURRENTLY	everything except ACCESS SHARE
ACCESS EXCLUSIVE	DROP, TRUNCATE, REINDEX, CLUSTER, VACUUM FULL, most ALTER TABLE	everything

13.2 The lock queue problem

PostgreSQL's lock queue is **FIFO and blocking**. If a transaction holds `ACCESS SHARE` on a table for an hour and you issue an `ALTER TABLE` (needing `ACCESS EXCLUSIVE`), your DDL waits — and every subsequent `SELECT` queues behind your DDL. One long query plus one DDL statement equals a full outage on that table.

The defence is mandatory in production:

```
BEGIN;
SET LOCAL lock_timeout = '3s';
ALTER TABLE app.orders ADD COLUMN notes text;
COMMIT;
-- Retry in a loop if it times out. Never run DDL without lock_timeout.
```

13.3 DDL that is safe (and that is not)

Operations that are metadata-only (fast, brief `ACCESS EXCLUSIVE`):

- `ADD COLUMN` with no default, or with a **constant** default (PG11+ stores it in the catalog).
- `DROP COLUMN` (marks the column dropped; space is reclaimed on rewrite).
- `ALTER COLUMN ... DROP NOT NULL`.

- `ALTER COLUMN ... SET NOT NULL` if a validated matching `CHECK (col IS NOT NULL)` constraint already exists (PG12+).
- `ADD CONSTRAINT ... NOT VALID`, then `VALIDATE CONSTRAINT` (which takes only `SHARE UPDATE EXCLUSIVE`).
- `ALTER TABLE ... SET (fillfactor = ...), SET STATISTICS, SET STORAGE`.
- Widening `varchar(50)` → `varchar(100)` or → `text`; `int` → `bigint` is **not** free and rewrites.

Operations that rewrite the whole table (long `ACCESS EXCLUSIVE` — plan an outage or use `pg_repack`/logical replication):

- `ALTER COLUMN ... TYPE` in most cases.
- `ADD COLUMN` with a volatile default (e.g. `DEFAULT now()` — but not `DEFAULT clock_timestamp()`'s constant cousins).
- `SET TABLESPACE, CLUSTER, VACUUM FULL`.

Safe patterns:

```
-- Add a NOT NULL column with a default, online
ALTER TABLE app.orders ADD COLUMN priority int;
ALTER TABLE app.orders ALTER COLUMN priority SET DEFAULT 5;
UPDATE app.orders SET priority = 5 WHERE priority IS NULL;
ALTER TABLE app.orders ADD CONSTRAINT priority_nn CHECK (priority IS NOT NULL) NOT VALID;
ALTER TABLE app.orders VALIDATE CONSTRAINT priority_nn;
ALTER TABLE app.orders ALTER COLUMN priority SET NOT NULL;
ALTER TABLE app.orders DROP CONSTRAINT priority_nn;

-- Add a foreign key without a long lock
ALTER TABLE app.orders ADD CONSTRAINT fk_cust
  FOREIGN KEY (customer_id) REFERENCES app.customers(id) NOT VALID;
ALTER TABLE app.orders VALIDATE CONSTRAINT fk_cust;
```

13.4 Diagnosing lock waits

```
-- PG9.6+: the definitive "who is blocking whom" query
SELECT blocked.pid AS blocked_pid,
       blocked.username AS blocked_user,
       left(blocked.query,80) AS blocked_query,
       blocked.wait_event_type, blocked.wait_event,
       now() - blocked.query_start AS blocked_for,
       blocking.pid AS blocking_pid,
       blocking.username AS blocking_user,
       blocking.state AS blocking_state,
       left(blocking.query,80) AS blocking_query,
       now() - blocking.xact_start AS blocking_xact_age
FROM pg_stat_activity blocked
JOIN LATERAL unnest(pg_blocking_pids(blocked.pid)) AS bp(pid) ON true
JOIN pg_stat_activity blocking ON blocking.pid = bp.pid
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0
ORDER BY blocked_for DESC;

-- Raw lock view
SELECT l.pid, l.locktype, l.mode, l.granted,
       coalesce(c.relname, l.locktype) AS object,
       a.state, left(a.query, 60) AS query
FROM pg_locks l
LEFT JOIN pg_class c ON c.oid = l.relation
LEFT JOIN pg_stat_activity a ON a.pid = l.pid
WHERE NOT l.granted OR l.mode LIKE '%Exclusive%'
ORDER BY l.granted, l.pid;
```

Set `log_lock_waits = on` (with `deadlock_timeout = 1s`) so that any wait longer than one second is logged with the blocking statement. This is close to free and repeatedly pays for itself.

13.5 Deadlocks

PostgreSQL detects deadlocks after `deadlock_timeout` and kills one transaction with `SQLSTATE 40P01`. The log entry names both statements and the locks involved:

```

ERROR:  deadlock detected
DETAIL:  Process 1234 waits for ShareLock on transaction 5678; blocked by process 9876.
        Process 9876 waits for ShareLock on transaction 5679; blocked by process 1234.
HINT:   See server log for query details.

```

Cures, in order: order your updates consistently (always touch parent before child, always ascending by primary key); shorten transactions; use `SELECT ... FOR UPDATE` on the parent row to serialise access explicitly; and retry on 40P01. Foreign keys are a frequent hidden cause — a child insert takes `KEY SHARE` on the parent row, which conflicts with a concurrent parent update.

13.6 Advisory locks

Application-level mutexes that PostgreSQL tracks but does not associate with any object.

```

SELECT pg_advisory_lock(42);           -- session-scoped, must be unlocked explicitly
SELECT pg_try_advisory_lock(42);       -- non-blocking
SELECT pg_advisory_xact_lock(42);      -- released at COMMIT/ROLLBACK – prefer this
SELECT pg_advisory_unlock(42);
SELECT * FROM pg_locks WHERE locktype = 'advisory';

```

Use `pg_advisory_xact_lock` for singleton jobs, leader election among workers, and serialising migrations. Do **not** use session-scoped advisory locks behind a transaction-mode pooler — the lock will outlive the client's use of that server connection.

14. VACUUM and Autovacuum

This chapter is the operational heart of PostgreSQL administration. If you master one thing, make it this.

14.1 What vacuum does

1. **Removes dead tuples** and frees their line pointers for reuse, updating the free space map.
2. **Freezes old tuples** so their XIDs can be recycled, advancing `relfrozenxid`.
3. **Updates the visibility map**, enabling index-only scans.
4. **Removes index entries** pointing at dead tuples (the expensive phase).
5. **Truncates trailing empty pages** back to the filesystem (needs a brief `ACCESS EXCLUSIVE` lock, which it gives up if anyone is waiting).
6. **Updates `pg_class.rel tuples/rel pages`** used by the planner.

VACUUM does **not** return space in the middle of a file to the OS. That is `VACUUM FULL`, `CLUSTER`, or `pg_repack`.

`ANALYZE` is separate: it samples rows and updates `pg_statistic`. Autovacuum runs both, on independent thresholds.

14.2 When autovacuum triggers

For each table, autovacuum runs `VACUUM` when:

```

n_dead_tup > autovacuum_vacuum_threshold
+ autovacuum_vacuum_scale_factor × reltuples

```

Defaults: `threshold = 50`, `scale_factor = 0.2`. That means **20 % of the table must be dead** before vacuum starts. On a 500-million-row table that is 100 million dead rows — far too much. Always lower the scale factor on large tables.

Since PG13 there is also an insert-driven trigger, which matters for append-only tables that would otherwise never be vacuumed (and so never freeze, and never get a visibility map):

```

n_ins_since_vacuum > autovacuum_vacuum_insert_threshold      (default 1000)
+ autovacuum_vacuum_insert_scale_factor × reltuples          (default 0.2)

```

`ANALYZE` triggers on:

```
n_mod_since_analyze > autovacuum_analyze_threshold (default 50)
+ autovacuum_analyze_scale_factor × reltuples (default 0.1)
```

And an anti-wraparound vacuum is forced when `age(relfrozenxid) > autovacuum_freeze_max_age` (default 200 million), regardless of dead tuples and regardless of `autovacuum = off`.

PostgreSQL 18 added `autovacuum_vacuum_max_threshold` (default 100 000 000), which caps the scale-factor calculation so that very large tables are vacuumed on an absolute row count rather than a percentage — a welcome fix for exactly the problem above.

14.3 Cost-based throttling

Autovacuum deliberately slows itself down. It accumulates a cost per page touched and sleeps when it exceeds a limit:

Parameter	Default	Meaning
<code>vacuum_cost_page_hit</code>	1	Page found in shared buffers
<code>vacuum_cost_page_miss</code>	2 (was 10 before PG14)	Page read from disk/OS cache
<code>vacuum_cost_page_dirty</code>	20	Page dirtied by vacuum
<code>autovacuum_vacuum_cost_limit</code>	-1 (→ <code>vacuum_cost_limit</code> , 200)	Cost budget before sleeping
<code>autovacuum_vacuum_cost_delay</code>	2ms	Sleep duration

With the defaults, throughput is roughly:

```
(cost_limit / cost_delay) × 8 kB ≈ (200 / 0.002 s) × 8 kB ≈ 780 MB/s of page *hits*
but only (200/20 dirty) × 500 sleeps/s × 8 kB ≈ 40 MB/s of *dirtied* pages
```

40 MB/s of dirty pages was reasonable for a 2005 disk array. On NVME it is absurd. Raise `autovacuum_vacuum_cost_limit` to 2000–5000 (or set it to -1 and raise `vacuum_cost_limit`) on any modern storage. This single change resolves a large fraction of "autovacuum can't keep up" situations.

Note the budget is shared across all autovacuum workers: adding workers without raising the cost limit makes each one slower, not the total faster.

14.4 Per-table tuning

```
-- A big, hot table: vacuum aggressively
ALTER TABLE app.orders SET (
  autovacuum_vacuum_scale_factor = 0.01,
  autovacuum_vacuum_threshold    = 5000,
  autovacuum_analyze_scale_factor = 0.005,
  autovacuum_vacuum_cost_delay    = 0,      -- no throttling for this table
  autovacuum_vacuum_cost_limit    = 10000,
  autovacuum_freeze_min_age       = 0,      -- freeze eagerly while we're here
  autovacuum_multixact_freeze_max_age = 150000000
);

-- An append-only audit table: freeze early, don't waste time looking for dead rows
ALTER TABLE app.audit_log SET (
  autovacuum_vacuum_insert_scale_factor = 0.0,
  autovacuum_vacuum_insert_threshold    = 100000,
  autovacuum_freeze_min_age              = 0,
  autovacuum_freeze_table_age            = 100000000
);

-- Inspect current per-table settings
SELECT relname, reloptions FROM pg_class WHERE reloptions IS NOT NULL;
```


14.5 Manual vacuum

```
VACUUM (VERBOSE, ANALYZE) app.orders;
VACUUM (FREEZE) app.orders;                -- freeze everything possible
VACUUM (DISABLE_PAGE_SKIPPING) app.orders;  -- ignore the visibility map
VACUUM (INDEX_CLEANUP OFF) app.orders;      -- PG12+: skip index vacuum; fast space reclaim
VACUUM (TRUNCATE OFF) app.orders;           -- don't take the exclusive lock to truncate
VACUUM (PARALLEL 4) app.orders;             -- PG13+: parallel index vacuuming
VACUUM (SKIP_LOCKED) ;                     -- PG12+: skip tables you'd have to wait for
VACUUM (BUFFER_USAGE_LIMIT '2GB') app.orders; -- PG16+: control ring buffer size
VACUUM (ONLY_DATABASE_STATS);              -- PG16+
ANALYZE (VERBOSE, SKIP_LOCKED) app.orders;
```

vacuumdb is the batch tool:

```
vacuumdb --analyze-in-stages -d appdb        # quick stats first, then better – post-restore
vacuumdb --jobs=8 --analyze -d appdb
vacuumdb --min-xid-age=500000000 --all --jobs=4 # PG9.6+: target only wraparound-risky tables
vacuumdb --min-mxid-age=200000000 --all
```

14.6 Monitoring vacuum

```
-- In-flight vacuums, with phase and progress
SELECT p.pid, p.datname, p.relid::regclass, p.phase,
       pg_size_pretty(p.heap_blks_total * 8192::bigint) AS total,
       round(100.0*p.heap_blks_scanned/nullif(p.heap_blks_total,0),1) AS pct_scanned,
       p.index_vacuum_count, p.max_dead_tuple_bytes, p.dead_tuple_bytes, -- PG17+ names
       a.wait_event_type, a.wait_event, now()-a.xact_start AS running_for
FROM pg_stat_progress_vacuum p JOIN pg_stat_activity a USING (pid);

SELECT * FROM pg_stat_progress_analyze;
SELECT * FROM pg_stat_progress_cluster;      -- CLUSTER / VACUUM FULL
SELECT * FROM pg_stat_progress_create_index;

-- Tables that vacuum is not keeping up with
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/nullif(n_live_tup+n_dead_tup,0),1) AS dead_pct,
       autovacuum_count, last_autovacuum,
       total_autovacuum_time, total_vacuum_time -- PG18+
FROM pg_stat_user_tables
WHERE n_dead_tup > 100000
ORDER BY n_dead_tup DESC LIMIT 25;
```

With `log_autovacuum_min_duration = 0`, every autovacuum logs a line like:

```
automatic vacuum of table "appdb.app.orders": index scans: 1
pages: 0 removed, 812345 remain, 12043 scanned (1.48% of total)
tuples: 1543210 removed, 48120394 remain, 12043 are dead but not yet removable
removable cutoff: 918273645, which was 12 XIDs old when operation ended
index scan needed: 8123 pages from table (1.00% of total) had 1543210 dead item identifiers removed
avg read rate: 42.113 MB/s, avg write rate: 18.204 MB/s
buffer usage: 41203 hits, 18234 misses, 7881 dirtied
WAL usage: 8123 records, 2011 full page images, 15903214 bytes
system usage: CPU: user: 2.14 s, system: 0.51 s, elapsed: 4.19 s
```

The line to watch is **"dead but not yet removable"**. A large number there means something is holding an old snapshot — go find the long transaction, the inactive replication slot, or the orphaned prepared transaction.

14.7 The autovacuum troubleshooting checklist

1. Is autovacuum on? `SHOW autovacuum;` — and check per-table `reloptions` for a local `autovacuum_enabled = false`.
2. Are workers saturated? Count rows in `pg_stat_activity` `WHERE backend_type = 'autovacuum worker'`. If it is pinned at `autovacuum_max_workers` all day, raise workers *and* the cost limit.
3. Is it being throttled? Raise `autovacuum_vacuum_cost_limit`.
4. Is something holding old snapshots? Long transactions, idle-in-transaction sessions, inactive replication slots, prepared transactions, `hot_standby_feedback`.

5. Are the thresholds too lax for large tables? Lower `autovacuum_vacuum_scale_factor` per table.
6. Is it being cancelled repeatedly? Autovacuum yields to conflicting lock requests; frequent DDL or `LOCK TABLE` on a table can starve it. Look for `canceling autovacuum task` in the log.
7. Is `maintenance_work_mem` too small? Below PG17, vacuum could only track ~178 million dead TIDs per index pass with 1 GB; multiple index scans per vacuum is the symptom (`index scans: 3` in the log). PG17's new TID store made this dramatically more memory-efficient.
8. Are there too many indexes? Index cleanup dominates vacuum time. Dropping unused indexes speeds up vacuum directly.

Oracle parallel. There is no equivalent process to tune. Oracle's undo retention plus SMON handles version cleanup; automatic statistics gathering runs in a maintenance window. In PostgreSQL, both jobs run continuously and are the DBA's responsibility to size. If you take one habit from Oracle into PostgreSQL, let it be "watch the maintenance job" — but here the job never stops.

Part VI — WAL, Backup, and Recovery

15. Write-Ahead Logging

15.1 Mechanics

Every change to a data page is first described in a WAL record, which is flushed to durable storage before the transaction is reported as committed. WAL lives in `$PGDATA/pg_wal` as fixed-size segment files (16 MB by default, set at `initdb` with `--wal-segsize`).

Positions are **LSNs** (log sequence numbers), printed as `hex/hex`:

```
SELECT pg_current_wal_lsn(), pg_current_wal_insert_lsn(), pg_current_wal_flush_lsn();
SELECT pg_walfile_name(pg_current_wal_lsn());
SELECT pg_size_pretty(pg_wal_lsn_diff('0/A5000000','0/A0000000')); -- distance between LSNs
SELECT * FROM pg_ls_waldir() ORDER BY modification DESC LIMIT 10;
```

`wal_level` controls how much is logged:

Level	Logs	Enables
<code>minimal</code>	Crash recovery only	Nothing else; some bulk operations skip WAL entirely
<code>replica</code> (default)	Enough for archiving and physical standbys	PITR, streaming replication
<code>logical</code>	Adds information for logical decoding	Logical replication, CDC (Debezium, pglogical)

Raising to `logical` costs roughly 5–15 % more WAL volume depending on the workload and requires a restart. If there is any prospect of CDC or a logical-replication-based upgrade, set it early.

15.2 Full page writes

Because a torn 8 kB write across a 4 kB device sector can leave a page half-updated, PostgreSQL writes a **full copy of a page** into WAL the first time it is modified after each checkpoint (`full_page_writes = on`). This is what makes recovery safe, and it is why WAL volume spikes immediately after a checkpoint.

Implications:

- Frequent checkpoints multiply WAL volume. Increasing `max_wal_size` and `checkpoint_timeout` reduces WAL, sometimes dramatically.

- `wal_compression = zstd` (or `lz4`) compresses full-page images specifically and often cuts total WAL by 30–60 % on write-heavy systems for a few percent of CPU. Turn it on.
- `full_page_writes = off` is only safe on filesystems that guarantee atomic 8 kB writes (ZFS with `recordsize ≥ 8k`, some enterprise arrays with battery-backed atomic writes). Getting this wrong means silent corruption discovered months later. Leave it on unless you can prove otherwise.

15.3 Checkpoints

A checkpoint flushes all dirty shared buffers to disk and records a restart point in `pg_control`.

```
checkpoint_timeout = 15min      # time-based trigger
max_wal_size = 16GB            # volume-based trigger
checkpoint_completion_target = 0.9 # spread writes over 90 % of the interval
checkpoint_flush_after = 256kB  # ask the OS to write back incrementally
```

Checkpoints triggered by `max_wal_size` ("requested") are the ones that hurt, because they are not spread out. Watch the ratio:

```
-- PG17+
SELECT num_timed, num_requested,
       round(100.0*num_requested/nullif(num_timed+num_requested,0),1) AS pct_requested,
       write_time, sync_time, buffers_written, stats_reset
FROM pg_stat_checkpointing;
-- PG16 and earlier: pg_stat_bgwriter with checkpoints_timed / checkpoints_req
```

Aim for more than 90 % timed checkpoints. If not, raise `max_wal_size`. With `log_checkpoints = on` (default since PG15) each checkpoint logs its duration and buffer counts.

`CHECKPOINT;` forces one immediately — useful before a shutdown or a filesystem snapshot, disruptive during peak load.

Oracle parallel. WAL is redo; LSN is roughly SCN plus a byte offset. Full page writes have no Oracle equivalent because Oracle handles fractured blocks through the redo stream and block recovery from the datafile. `checkpoint_completion_target` corresponds to incremental checkpointing driven by `FAST_START_MTTARGET`. There is no equivalent of redo log groups or members; WAL redundancy comes from archiving and replication, not multiplexing.

15.4 Durability trade-offs

Setting	Value	Meaning
<code>synchronous_commit</code>	<code>on</code>	WAL flushed locally (and to sync standbys) before commit returns. Default.
	<code>remote_apply</code>	Wait until the standby has replayed — read-your-writes on the standby
	<code>remote_write</code>	Wait until the standby has written to OS (not fsynced)
	<code>local</code>	Local flush only; ignore synchronous standbys
	<code>off</code>	Commit returns before flush. Risks losing the last <code>wal_writer_delay × 3</code> of commits on a crash — but never corrupts the database.
<code>fsync</code>	<code>off</code>	Risks total corruption. Never in production.
<code>full_page_writes</code>	<code>off</code>	Risks corruption on torn pages. See above.

`synchronous_commit = off` is a legitimate, safe-from-corruption tuning option for workloads that can tolerate losing a fraction of a second of committed transactions (event ingestion, telemetry, caches). It can be set per transaction:

```
BEGIN; SET LOCAL synchronous_commit = off; INSERT INTO telemetry ...; COMMIT;
```

`fsync = off` is not a tuning option. It is a way to make a test database fast and a production database unrecoverable.

15.5 Archiving

```
archive_mode = on
archive_command = 'pgbackrest --stanza=main archive-push %p'
archive_timeout = 300
```

Rules for `archive_command`:

- It must return **0 only on success**. If it returns 0 without archiving, PostgreSQL deletes the segment and your PITR chain is broken.
- It must **not overwrite** an existing archived file with different content. The classic naive command `cp %p /archive/%f` fails this test; use `test ! -f /archive/%f && cp %p /archive/%f` at minimum, or better, use a real tool.
- It runs serially, one segment at a time. On a high-write system a slow command becomes a bottleneck and `pg_wal` fills. `pgBackRest`'s asynchronous archiving (`archive-async=y`) exists precisely for this.
- If it fails, PostgreSQL retries forever and `pg_wal` grows without bound until the filesystem fills and the cluster shuts down. **Monitor `pg_stat_archiver` and alert on `failed_count`.**

```
SELECT archived_count, last_archived_wal, last_archived_time,
       failed_count, last_failed_wal, last_failed_time, stats_reset
FROM pg_stat_archiver;

-- How many segments are waiting to be archived?
SELECT count(*) FROM pg_ls_dir('pg_wal/archive_status') f WHERE f LIKE '%.ready';
```

PostgreSQL 15 added `archive_library`, which lets a shared library archive WAL in-process — faster and without a shell fork per segment. `pgBackRest` and others are moving to it.

16. Backup

16.1 Choosing a strategy

Method	Type	Granularity	PITR	Cross-version	Notes
<code>pg_dump</code>	Logical	Database / schema / table	No	Yes	Consistent snapshot; slow to restore; no PITR
<code>pg_dumpall</code>	Logical	Cluster (incl. roles)	No	Yes	Use <code>--globals-only</code> alongside <code>pg_dump</code>
<code>pg_basebackup</code>	Physical	Whole cluster	With WAL archive	No	Simple, built in, no incremental (until PG17)
pgBackRest	Physical	Whole cluster	Yes	No	Full/diff/incr, parallel, compression, encryption, S3/Azure/GCS, delta restore. The default recommendation.
Barman	Physical	Whole cluster	Yes	No	Similar; strong rsync/streaming modes, good for many clusters from one host
WAL-G	Physical	Whole cluster	Yes	No	Cloud-native, Go, popular in Kubernetes
Storage snapshot	Physical	Volume	With WAL archive	No	Must be crash-consistent across all volumes including tablespaces

The uncomfortable truth: `pg_dump` is not a backup strategy for anything you care about. It gives you no PITR, its restore time on a multi-terabyte database is measured in days, and it takes a snapshot that holds back vacuum for its whole duration. Use a physical backup tool with WAL archiving, and use `pg_dump` for schema extraction, per-table recovery, and migrations.

16.2 Logical backups

```
# Custom format (compressed, parallel-restorable, selective) — always prefer this over plain SQL
pg_dump -h db1 -U postgres -d appdb -Fc -Z 6 -f appdb.dump

# Directory format supports parallel dump AND parallel restore
pg_dump -d appdb -Fd -j 8 -f /backup/appdb_dir

# Selective
pg_dump -d appdb -Fc -n app -T 'app.audit_*' -f app_schema.dump
pg_dump -d appdb --schema-only -f schema.sql
pg_dump -d appdb --data-only -t app.orders -f orders_data.sql

# Globals: roles, tablespaces, cluster-wide settings. pg_dump does NOT include these.
pg_dumpall --globals-only -f globals.sql

# Restore
createdb -T template0 appdb_restored
pg_restore -d appdb_restored -j 8 --no-owner --no-privileges appdb.dump
pg_restore -d appdb_restored -j 8 -t orders appdb.dump      # single table
pg_restore -l appdb.dump > toc.list                        # edit, then:
pg_restore -d appdb_restored -L toc.list appdb.dump        # selective restore

# Inspect without restoring
pg_restore --list appdb.dump | head -50
```

Useful flags: `--no-owner`, `--no-privileges` (restore into a different ownership model), `--section=pre-data|data|post-data` (load data before creating indexes and constraints — much faster), `--exclude-table-data`, `--if-exists --clean`, and `--disable-triggers` (data-only restores that must not fire triggers; requires superuser).

For the fastest logical restore of a large database: restore `pre-data`, load data with parallel `pg_restore -j`, then restore `post-data` (indexes, FKs) with a large `maintenance_work_mem` and `max_parallel_maintenance_workers`.

Oracle parallel. `pg_dump -Fc/-Fd` is Data Pump; `pg_restore -j` is `impdp PARALLEL`. The TOC list is roughly a `SQLFILE/INCLUDE/EXCLUDE` mechanism, but more explicit. There is no transportable tablespace equivalent — physical portability is all-or-nothing at the cluster level.

16.3 Physical backups with pg_basebackup

```
pg_basebackup -h primary -U replicator -D /var/lib/postgresql/18/data \
  --wal-method=stream --checkpoint=fast --progress --verbose \
  --compress=server-zstd:6 --format=tar \
  --slot=standby1_slot --create-slot

# PG17+: incremental backups
pg_basebackup -D /backup/full -c fast                        # full, with summarize_wal = on
pg_basebackup -D /backup/incr1 --incremental=/backup/full/backup_manifest
pg_combinebackup /backup/full /backup/incr1 -o /restore/target
```

PostgreSQL 17 added incremental base backups, which require `summarize_wal = on` on the source. They are a genuine improvement, but `pgBackRest` still offers more: block-level delta restore, retention policies, parallel compression, encryption, cloud object stores, and verified restores.

16.4 pgBackRest

```
# /etc/pgbackrest/pgbackrest.conf
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=4
repo1-retention-diff=14
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<from a secret manager>
repo1-bundle=y
repo1-block=y                # block-level incremental (2.46+)
process-max=8
compress-type=zst
compress-level=6
```

```
log-level-console=info
log-level-file=detail
start-fast=y
delta=y
archive-async=y
spool-path=/var/spool/pgbackrest

# S3 alternative
#repo1-type=s3
#repo1-s3-bucket=pg-backups
#repo1-s3-region=us-west-2
#repo1-s3-endpoint=s3.us-west-2.amazonaws.com

[main]
pg1-path=/var/lib/pgsql/18/data
pg1-port=5432
```

```
pgbackrest --stanza=main stanza-create
pgbackrest --stanza=main check

pgbackrest --stanza=main --type=full backup
pgbackrest --stanza=main --type=diff backup
pgbackrest --stanza=main --type=incr backup

pgbackrest --stanza=main info
pgbackrest --stanza=main --set=20260901-020000F verify
pgbackrest --stanza=main expire
```

A sane schedule: weekly full, daily differential, hourly incremental, with retention that satisfies your RPO/RTO policy and at least one copy in a different failure domain.

16.5 Backup verification

An unverified backup is a hypothesis. Automate a restore test:

```
#!/usr/bin/env bash
set -euo pipefail
STANZA=main
TARGET=/restore/verify
rm -rf "$TARGET"; mkdir -p "$TARGET"
pgbackrest --stanza=$STANZA --delta --pg1-path="$TARGET" restore
# Start on a spare port, wait for recovery, run checks, then tear down
/usr/pgsql-18/bin/pg_ctl -D "$TARGET" -o "-p 5599" -w -t 900 start
psql -p 5599 -d appdb -c "SELECT count(*) FROM app.orders;"
psql -p 5599 -d appdb -c "SELECT pg_is_in_recovery();"
# Structural verification
/usr/pgsql-18/bin/pg_amcheck -p 5599 -d appdb --heapallindexed --parent-check --progress
/usr/pgsql-18/bin/pg_ctl -D "$TARGET" -m immediate stop
```

Run this weekly and alert on failure. Also verify the backup manifest (`pg_verifybackup`) after every `pg_basebackup`, and keep `--repo1-retention` aligned with the oldest PITR target your policy requires.

17. Point-in-Time Recovery

17.1 The recovery model

Since PostgreSQL 12 there is no `recovery.conf`. Recovery is configured in `postgresql.conf` (or `postgresql.auto.conf`) and triggered by the presence of a signal file in `$PGDATA`:

File	Effect
<code>recovery.signal</code>	Archive recovery: replay to the target, then promote
<code>standby.signal</code>	Standby mode: replay continuously, stay read-only until promoted

Recovery target parameters (set at most one):

```

restore_command = 'pgbackrest --stanza=main archive-get %f "%p"'
recovery_target_time = '2026-09-03 14:25:00+00'
# recovery_target_lsn = '0/1A2B3C4D'
# recovery_target_xid = '183726354'
# recovery_target_name = 'before_release_42'      -- set by pg_create_restore_point()
# recovery_target = 'immediate'                  -- stop as soon as consistency is reached
recovery_target_inclusive = on
recovery_target_action = 'pause'                  -- pause | promote | shutdown
recovery_target_timeline = 'latest'

```

`recovery_target_action = 'pause'` is the correct choice for a real recovery: the server stops at the target and waits, letting you connect read-only and confirm you picked the right moment before committing to it with `pg_wal_replay_resume()` or `pg_promote()`.

17.2 A worked PITR

Scenario: at 14:32 someone ran `DELETE FROM app.orders` without a `WHERE` clause and committed. You want the database as of 14:25.

```

# 1. Stop the damaged cluster. Do not delete anything yet.
systemctl stop postgresql-18
mv /var/lib/pgsql/18/data /var/lib/pgsql/18/data.broken

# 2. Restore the base backup to the recovery target
pgbackrest --stanza=main \
  --type=time --target="2026-09-03 14:25:00+00" \
  --target-action=pause --target-timeline=current \
  restore
# (pgBackRest writes recovery.signal and the recovery_* settings for you)

# 3. Start and let it replay
systemctl start postgresql-18
tail -f /var/lib/pgsql/18/data/log/postgresql-*.log
# LOG: starting point-in-time recovery to 2026-09-03 14:25:00+00
# LOG: restored log file "0000000100000A5F00000031" from archive
# LOG: recovery stopping before commit of transaction 8271623, time 2026-09-03 14:25:00.113+00
# LOG: recovery has paused

```

```

-- 4. Verify, read-only
SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp();
SELECT count(*) FROM app.orders;

-- 5a. Wrong target? Adjust recovery_target_time, restart, replay again.
-- 5b. Right target? Promote.
SELECT pg_promote(wait => true, wait_seconds => 60);
SELECT pg_is_in_recovery(); -- false

```

```

# 6. Immediately take a fresh full backup – you are now on a new timeline.
pgbackrest --stanza=main --type=full backup

```

17.3 Timelines

Every promotion creates a new **timeline**, incrementing the first eight hex digits of the WAL filename and writing a `.history` file to the archive. Timelines are what let you recover *past* a previous recovery point: `recovery_target_timeline = 'latest'` follows the most recent branch, while a specific number follows that branch.

```

ls /var/lib/pgbackrest/archive/main/18-1/ | grep history
cat /var/lib/pgbackrest/archive/main/18-1/00000002.history
# 1 0/A5000148 before 2026-09-03 14:25:00.113+00

```

Getting this wrong — recovering along the wrong timeline — is a common cause of "the data isn't there" after a successful-looking restore. Always check which timeline the standby or restored cluster is on:

```

SELECT timeline_id, redo_lsn FROM pg_control_checkpoint();

```

17.4 Partial recovery: getting one table back

Full PITR to a new cluster on a spare port, then `pg_dump` just the table and load it into production. This is almost always faster and safer than trying to reverse the change in place.

```
pgbackrest --stanza=main --type=time --target="..." --pg1-path=/tmp/recover restore
pg_ctl -D /tmp/recover -o "-p 5599" start
pg_dump -p 5599 -d appdb -t app.orders -Fc -f orders_at_1425.dump
pg_restore -d appdb --data-only -t orders_restore orders_at_1425.dump
```

Oracle parallel. PITR maps closely to RMAN's `RECOVER DATABASE UNTIL TIME`. Timelines are Oracle incarnations, and `.history` files are `V$DATABASE_INCARNATION`. `recovery_target_action = pause` corresponds to opening `READ ONLY` before `RESETLOGS`. What is missing: `FLASHBACK DATABASE` (no fast rewind — you always restore from a backup), tablespace point-in-time recovery, block media recovery, and a recovery catalog in core.

18. Corruption: Detection and Response

18.1 Detection

```
-- Are checksums on?
SHOW data_checksums;
SELECT * FROM pg_stat_database WHERE datname = current_database();
-- checksum_failures, checksum_last_failure (PG12+)

-- Structural verification of heap and indexes (PG14+)
-- Run from the shell against a restored copy, or off-peak
```

```
pg_amcheck --all --heapallindexed --parent-check --progress --jobs=4
pg_amcheck -d appdb -t app.orders --heapallindexed
pg_checksums --check -D /var/lib/pgsql/18/data      # cluster must be offline
pg_verifybackup /backup/full                        # verifies a base backup manifest
```

`amcheck` verifies that every heap tuple has a matching index entry and that B-tree structure invariants hold. It is the closest thing PostgreSQL has to `ANALYZE ... VALIDATE STRUCTURE` and `DBVERIFY`.

Symptoms in the log:

```
WARNING: page verification failed, calculated checksum 12345 but expected 54321
ERROR:   invalid page in block 8123 of relation base/16384/16401
ERROR:   could not access status of transaction 4294967295
PANIC:   could not locate a valid checkpoint record
```

18.2 Response

1. **Stop writing.** Take the cluster out of service. Every additional write reduces your options.
2. **Preserve everything:** a filesystem-level copy of `$PGDATA`, the logs, `pg_control`, the WAL archive.
3. **Determine the blast radius.** Which relation, which block? `SELECT pg_filenode_relation(0, 16401);` maps a `relfilenode` back to a relation.
4. **Prefer restore over repair.** PITR to just before the corruption is almost always the correct answer if you have a good backup.
5. **Check the hardware.** Corruption is usually a symptom: failing storage, bad memory (run `memtest`), a filesystem or LVM misconfiguration, `fsync = off`, an unsafe `full_page_writes = off`, or a storage layer that lies about flushes.

If you must salvage data from a corrupt cluster without a usable backup:

```
-- DANGEROUS: allows reads to skip corrupt pages, returning wrong results silently
SET zero_damaged_pages = on;    -- last resort, on a copy, to extract what is readable
```

```
# Absolute last resort. This does not repair anything; it fabricates a control file
# and discards WAL, producing a cluster that starts but may be logically inconsistent.
pg_resetwal -f /var/lib/pgsql/18/data
# Immediately after: pg_dumpall the survivors into a brand-new cluster. Never keep running on it.
```

There is no DBMS_REPAIR, no block media recovery, and no RMAN BLOCKRECOVER. The mitigation is discipline: checksums on, backups verified, hardware monitored.

Part VII — Replication and High Availability

19. Physical Streaming Replication

19.1 How it works

The primary's `walsender` process streams WAL records to a standby's `walreceiver`, which writes them to the standby's `pg_wal` and hands them to the startup process for replay. The standby is a physical block-for-block copy: same file layout, same OIDs, same LSNs. It can serve read-only queries (`hot_standby = on`) and be promoted to primary.



19.2 Building a standby

On the primary:

```
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'xxx';
SELECT pg_create_physical_replication_slot('standby1');
```

```
# pg_hba.conf on the primary
hostssl replication replicator 10.20.1.0/24 scram-sha-256
```

On the standby:

```
systemctl stop postgresql-18
rm -rf /var/lib/pgsql/18/data/*
```

```
PGPASSWORD=xxx pg_basebackup \
-h primary.example -U replicator -D /var/lib/pgsql/18/data \
--wal-method=stream --slot=standby1 \
--checkpoint=fast --progress --verbose \
--write-recovery-conf # writes primary_conninfo + standby.signal
```

```
# postgresql.conf (or postgresql.auto.conf, written by --write-recovery-conf)
primary_conninfo = 'host=primary.example port=5432 user=replicator password=xxx application_name=standby1
sslmode=verify-full sslrootcert=/etc/pki/tls/certs/ca.crt'
primary_slot_name = 'standby1'
restore_command = 'pgbackrest --stanza=main archive-get %f "%p"' # fallback if streaming falls behind
hot_standby = on
hot_standby_feedback = off
max_standby_streaming_delay = 30s
max_standby_archive_delay = 60s
recovery_min_apply_delay = 0 # e.g. '1h' for a delayed standby
wal_receiver_status_interval = 5s
wal_receiver_timeout = 60s
```

```
touch /var/lib/pgsql/18/data/standby.signal
systemctl start postgresql-18
```


Always configure both `primary_conninfo` and `restore_command`. Streaming is the fast path; the archive is the safety net when the standby falls behind the primary's retained WAL.

19.3 Replication slots

A physical slot makes the primary retain WAL until the standby has consumed it — which removes the "standby fell behind and the WAL it needs is gone" failure. It also introduces a much worse failure: a slot for a standby that never comes back retains WAL forever until `pg_wal` fills the disk and the primary shuts down.

Mitigations:

```
max_slot_wal_keep_size = 64GB      # PG13+: invalidate the slot rather than fill the disk
```

```
SELECT slot_name, slot_type, active, active_pid,
       restart_lsn, confirmed_flush_lsn, wal_status, safe_wal_size,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal,
       inactive_since                                                    -- PG17+
FROM pg_replication_slots ORDER BY retained_wal DESC;

SELECT pg_drop_replication_slot('dead_standby');
```

`wal_status` values: `reserved` (within `max_wal_size`), `extended` (beyond it but still retained), `unreserved` (at risk), `lost` (WAL removed; the slot is now useless and the standby must be rebuilt).

PostgreSQL 18 added automatic invalidation of idle slots via `idle_replication_slot_timeout`, which is a meaningful safety improvement.

19.4 Monitoring replication

```
-- On the primary: one row per connected standby
SELECT application_name, client_addr, state, sync_state, sync_priority,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn)) AS pending_send,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, write_lsn))           AS write_lag_bytes,
       pg_size_pretty(pg_wal_lsn_diff(write_lsn, flush_lsn))         AS flush_lag_bytes,
       pg_size_pretty(pg_wal_lsn_diff(flush_lsn, replay_lsn))        AS replay_lag_bytes,
       write_lag, flush_lag, replay_lag, replay_time
FROM pg_stat_replication;

-- On the standby: how far behind in time
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn(),
       pg_last_xact_replay_timestamp(),
       CASE WHEN pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn() THEN 0
            ELSE extract(epoch FROM now() - pg_last_xact_replay_timestamp()) END AS lag_seconds;

SELECT * FROM pg_stat_wal_receiver;
```

Alert on `replay_lag` in seconds (not bytes — bytes are meaningless without a rate) and on any standby whose `state` is not streaming.

19.5 Synchronous replication

```
synchronous_standby_names = 'ANY 1 (standby1, standby2, standby3)'
# or 'FIRST 2 (standby1, standby2)' -- priority-based
# or '*' -- any connected standby (rarely what you want)
synchronous_commit = on             # or remote_apply for read-your-writes
```

`ANY n (...)` is quorum-based and the right choice for most designs: commit returns when any *n* of the listed standbys have confirmed. `FIRST n (...)` is priority-ordered.

The critical caveat: **if the required number of synchronous standbys is unavailable, commits hang**. Synchronous replication trades availability for durability. With a single synchronous standby, you have made your primary depend on the standby's health — often the opposite of the intent. Use `ANY 1 (s1, s2)` with two standbys so the loss of either is survivable.

`synchronous_commit = remote_apply` additionally guarantees that a subsequent read on that standby sees the commit, at the cost of waiting for replay.

Standby-side query conflicts: replay can need to remove a row that a standby query is reading.

`max_standby_streaming_delay` bounds how long replay waits before cancelling the query (ERROR: `canceling statement due to conflict with recovery`). `hot_standby_feedback = on` avoids this by telling the primary to hold back vacuum — which moves the pain to the primary as bloat. Pick deliberately: for a reporting standby running long queries, `hot_standby_feedback = on` plus watchful autovacuum tuning; for a pure HA standby, leave it off.

```
SELECT * FROM pg_stat_database_conflicts WHERE datname = current_database();
```

19.6 Cascading and delayed standbys

```
# On a standby that should feed another standby
max_wal_senders = 10          # a standby can serve walsenders too
# On the downstream standby
primary_conninfo = 'host=standby1 ...'
```

```
# A delayed standby – the closest thing to Flashback Database
recovery_min_apply_delay = '4h'
```

A four-hour delayed standby means a `DROP TABLE` on the primary can be caught before it replays: pause replay, promote, extract the table. It is a cheap and underused insurance policy.

```
SELECT pg_wal_replay_pause();
SELECT pg_is_wal_replay_paused();
SELECT pg_wal_replay_resume();
```

Oracle parallel. Streaming replication is physical Data Guard. `synchronous_commit = on` with `synchronous_standby_names` is Maximum Availability / Maximum Protection; `remote_write` is closer to SYNC with affirm off. Cascading standbys are cascaded standby databases. `recovery_min_apply_delay` is DELAY in the log apply services. Missing: the Data Guard Broker, fast-start failover, automatic gap resolution (the archive `restore_command` is the manual equivalent), Active Data Guard's real-time query with automatic block repair, and snapshot standby. Patroni fills the broker role — see Chapter 21.

20. Logical Replication and CDC

20.1 Built-in logical replication

Logical replication decodes WAL into row-level changes and applies them to a subscriber, which may be a different major version, a different architecture, or a subset of tables.

```
-- Publisher (requires wal_level = logical)
CREATE PUBLICATION app_pub FOR TABLE app.orders, app.customers;
CREATE PUBLICATION all_pub FOR ALL TABLES;
CREATE PUBLICATION sales_pub FOR TABLES IN SCHEMA sales;          -- PG15+
CREATE PUBLICATION open_pub FOR TABLE app.orders WHERE (status = 'OPEN'); -- row filter, PG15+
CREATE PUBLICATION cols_pub FOR TABLE app.customers (id, name, region); -- column list, PG15+

-- Subscriber
CREATE SUBSCRIPTION app_sub
  CONNECTION 'host=pub.example dbname=appdb user=repl password=xxx sslmode=verify-full'
  PUBLICATION app_pub
  WITH (copy_data = true, streaming = parallel, origin = none, binary = true,
        create_slot = true, slot_name = 'app_sub_slot');

-- Operations
ALTER SUBSCRIPTION app_sub REFRESH PUBLICATION;      -- pick up newly added tables
ALTER SUBSCRIPTION app_sub DISABLE;
ALTER SUBSCRIPTION app_sub SET (disable_on_error = true); -- PG15+
ALTER SUBSCRIPTION app_sub SKIP (lsn = '0/1A2B3C4D');   -- PG15+: skip a failing transaction
DROP SUBSCRIPTION app_sub;                             -- also drops the remote slot if reachable
```

Monitoring:

```
SELECT * FROM pg_stat_subscription;
SELECT * FROM pg_stat_subscription_stats;      -- PG15+; apply/sync error counts, conflicts (PG18+)
SELECT * FROM pg_replication_slots WHERE slot_type = 'logical';
SELECT * FROM pg_publication_tables;
```

Limitations you must design around:

- **DDL is not replicated.** Schema changes must be applied to the subscriber first (for additive changes) and coordinated manually.
- **Sequences are not replicated** (until PG19). After a logical-replication failover you must advance sequences on the new primary or you will get duplicate keys. This is the single most common logical-replication cutover bug.
- Tables need a **replica identity**: a primary key, a unique index declared with `ALTER TABLE ... REPLICA IDENTITY USING INDEX`, or `REPLICA IDENTITY FULL` (which logs the whole old row and makes updates very expensive).
- `TRUNCATE` is replicated (PG11+); large objects are not.
- Conflicts (duplicate key on the subscriber) stop the apply worker until resolved. Use `disable_on_error` so a conflict does not silently stall replication forever.
- Initial data copy of a large table holds a slot open and can generate significant WAL retention on the publisher.

20.2 Using logical replication for near-zero-downtime major upgrades

This is the standard technique for upgrading a large, busy cluster with minutes rather than hours of downtime:

1. Build a new cluster on the target major version.
2. Copy the schema: `pg_dump --schema-only | psql`.
3. Create a publication on the old cluster and a subscription on the new one.
4. Wait for the initial copy and for lag to reach near zero.
5. Quiesce the application, confirm zero lag, advance all sequences on the new cluster.
6. Repoint the application; drop the subscription.

PostgreSQL 17's `pg_createsubscriber` automates most of this by converting a physical standby into a logical subscriber, which avoids the expensive initial copy entirely:

```
pg_createsubscriber -D /var/lib/pgsql/18/data -P "host=primary dbname=appdb" -d appdb --all
```

```
-- Sequence fix-up before cutover
SELECT 'SELECT setval(' || quote_literal(schemaname) || '.' || sequencename) || ', ' ||
       last_value || ', true);'
FROM pg_sequences; -- run on the publisher, execute the output on the subscriber
```

20.3 Change data capture

Logical decoding is the foundation for CDC. Output plugins turn the decoded stream into a format a consumer understands:

Plugin	Output	Used by
pgoutput	Binary, built in	Native logical replication; Debezium (default since 1.x)
wal2json	JSON	Custom consumers, older Debezium configs
decoderbufs	Protobuf	Debezium (legacy)
test_decoding	Text	Debugging only

A typical Debezium/Kafka setup on the PostgreSQL side:

```

ALTER SYSTEM SET wal_level = 'logical';
-- restart

CREATE ROLE debezium WITH REPLICATION LOGIN PASSWORD 'xxx';
GRANT CONNECT ON DATABASE appdb TO debezium;
GRANT USAGE ON SCHEMA app TO debezium;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO debezium;
ALTER DEFAULT PRIVILEGES IN SCHEMA app GRANT SELECT ON TABLES TO debezium;

CREATE PUBLICATION dbz_pub FOR TABLE app.orders, app.order_lines, app.customers;
SELECT pg_create_logical_replication_slot('dbz_slot', 'pgoutput');

-- Ensure updates carry enough information for downstream consumers
ALTER TABLE app.orders REPLICA IDENTITY DEFAULT;      -- PK only (cheapest)
-- ALTER TABLE app.orders REPLICA IDENTITY FULL;      -- full old row; expensive but complete

```

Operational rules for CDC in production:

- **Monitor slot lag relentlessly.** A stopped connector is a slot that retains WAL and eventually fills the primary's disk. Alert on `pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)` per slot, and set `max_slot_wal_keep_size`.
- **Slots do not fail over.** A physical standby promoted to primary historically lost its logical slots. PostgreSQL 16 added logical decoding *on* a standby, and PostgreSQL 17 added **failover slots** (`CREATE SUBSCRIPTION ... WITH (failover = true)` and `pg_create_logical_replication_slot(..., failover := true)`, with `sync_replication_slots = on` on the standby). If you run CDC on a cluster with HA, being on PG17+ is close to mandatory.
- Set `logical_decoding_work_mem` high enough to avoid spilling large transactions to disk, and use `streaming = parallel` subscriptions (or Debezium's streaming support) for large transactions.
- Long-running transactions on the publisher delay decoding of everything after them — decoding is strictly in commit order.
- Heartbeat: on a low-traffic database, add a periodic write to a dummy table so the slot's `confirmed_flush_lsn` advances and WAL is released. Debezium has `heartbeat.interval.ms` and `heartbeat.action.query` for exactly this.

```

-- The CDC health query worth putting on a dashboard
SELECT slot_name, plugin, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS undelivered,
       wal_status, inactive_since
FROM pg_replication_slots WHERE slot_type = 'logical';

```

Oracle parallel. Logical decoding plus an output plugin is the architectural equivalent of GoldenGate's extract reading redo, with `pgoutput/wal2json` playing the part of the trail file format and Debezium or the native subscriber playing the replicat. What PostgreSQL lacks compared with GoldenGate: DDL replication, conflict detection and resolution (CDR), bidirectional/active-active topologies out of the box (see `pgEdge/BDR/spock` for commercial options), heterogeneous targets, and integrated Veridata-style comparison. For data validation between a PostgreSQL source and target, the practical equivalents are checksum-based row comparison scripts, `pgcompare`, or the open-source `data-diff` family.

21. High Availability

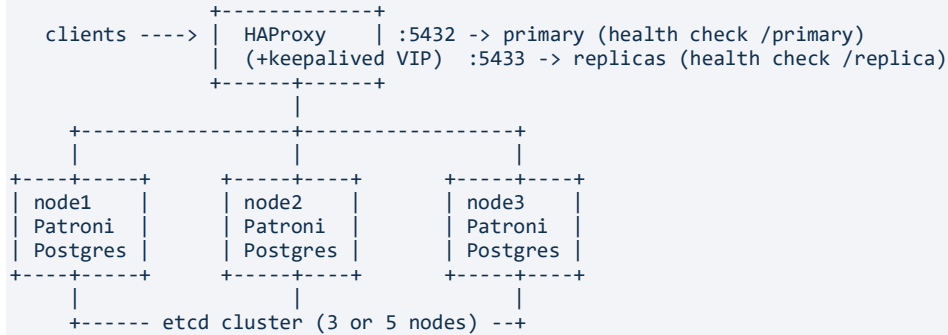
21.1 What core PostgreSQL gives you, and what it does not

Core gives you: replication, `pg_promote()`, and `pg_rewind`. It does **not** give you failure detection, automatic failover, fencing, quorum, or client redirection. Every production HA deployment therefore uses an external cluster manager.

Tool	Model	Notes
Patroni	DCS-based (etcd, Consul, ZooKeeper, Kubernetes)	The de facto standard. REST API, automatic failover, <code>pg_rewind</code> integration, callbacks for VIP/HAProxy.

Tool	Model	Notes
repmgr	Daemon + witness	Simpler, no DCS; more manual. Mature.
pg_auto_failover	Monitor node	Simple 2+1 topology; opinionated.
CloudNativePG / Crunchy PGO / Zalando operator	Kubernetes	Patroni-based or equivalent; the right answer inside K8s.
Pacemaker/Corosync + PAF	Classic Linux HA	Heavy, but familiar in enterprises that already run it.

21.2 A Patroni topology



```

# /etc/patroni/patroni.yml
scope: pg-prod
namespace: /service/
name: node1

restapi:
  listen: 0.0.0.0:8008
  connect_address: 10.20.1.11:8008

etcd3:
  hosts: 10.20.2.11:2379,10.20.2.12:2379,10.20.2.13:2379

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576      # 1 MB; refuse failover to a laggy node
    synchronous_mode: true
    synchronous_mode_strict: false       # true = block writes rather than lose sync
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: logical
        hot_standby: "on"
        max_wal_senders: 10
        max_replication_slots: 10
        wal_keep_size: 1GB
        archive_mode: "on"
        archive_command: "pgbackrest --stanza=main archive-push %p"
  initdb:
    - encoding: UTF8
    - locale: C
    - data-checksums

postgresql:
  listen: 0.0.0.0:5432
  connect_address: 10.20.1.11:5432
  data_dir: /var/lib/pgsql/18/data
  bin_dir: /usr/pgsql-18/bin
  pgpass: /var/lib/pgsql/.pgpass

```

```

authentication:
  replication: {username: replicator, password: "xxx"}
  superuser:   {username: postgres, password: "xxx"}
create_replica_methods:
  - pgbackrest
  - basebackup
pgbackrest:
  command: /usr/bin/pgbackrest --stanza=main --delta restore
  keep_data: true
  no_params: true

tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
  nosync: false

```

```

patronictl -c /etc/patroni/patroni.yml list
patronictl -c /etc/patroni/patroni.yml switchover --candidate node2
patronictl -c /etc/patroni/patroni.yml failover
patronictl -c /etc/patroni/patroni.yml reinit node3
patronictl -c /etc/patroni/patroni.yml pause      # maintenance: stop automatic failover
patronictl -c /etc/patroni/patroni.yml resume
patronictl -c /etc/patroni/patroni.yml edit-config
patronictl -c /etc/patroni/patroni.yml query -c "SELECT pg_is_in_recovery()"

```

Always `patronictl pause` before planned maintenance (a restart, a minor upgrade, a config change requiring restart). Forgetting to pause is how a routine restart becomes an unplanned failover.

21.3 HAProxy for routing

```

listen postgres_primary
  bind *:5432
  mode tcp
  option httpchk GET /primary
  http-check expect status 200
  default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
  server node1 10.20.1.11:5432 check port 8008
  server node2 10.20.1.12:5432 check port 8008
  server node3 10.20.1.13:5432 check port 8008

listen postgres_replicas
  bind *:5433
  mode tcp
  balance leastconn
  option httpchk GET /replica
  http-check expect status 200
  default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
  server node1 10.20.1.11:5432 check port 8008
  server node2 10.20.1.12:5432 check port 8008
  server node3 10.20.1.13:5432 check port 8008

```

`on-marked-down shutdown-sessions` is essential: without it, existing connections to a demoted primary stay open and the application keeps writing to a node that is now a standby (and failing).

An alternative with no proxy at all is libpq's multi-host connection string, which every modern driver supports:

```

postgresql://node1,node2,node3/appdb?target_session_attrs=read-write&connect_timeout=3
postgresql://node1,node2,node3/appdb?target_session_attrs=prefer-standby # PG14+ for reads

```

This removes the proxy as a single point of failure but gives you no connection pooling and slower failure detection.

21.4 pg_rewind

After a failover, the old primary may have WAL the new primary never received (it committed locally before dying). It cannot simply rejoin as a standby. `pg_rewind` rewinds the divergent blocks so it can:

```
# The old primary must have been shut down cleanly (or started and stopped once),
# and needs wal_log_hints = on or data checksums.
pg_rewind --target-pgdata=/var/lib/pgsql/18/data \
  --source-server='host=newprimary user=postgres dbname=postgres' \
  --progress --restore-target-wal
```

Requirements: `wal_log_hints = on` or data checksums enabled (another reason to use `--data-checksums`), and the target must be cleanly shut down. Patroni runs this automatically with `use_pg_rewind: true`. Without it, rejoining a former primary means a full rebuild from backup.

21.5 Failover runbook

Planned switchover:

1. Confirm all standbys are streaming with near-zero lag.
2. `patronictl switchover --candidate nodeN --scheduled now`.
3. Verify the new primary: `SELECT pg_is_in_recovery();` returns false.
4. Verify HAProxy has moved traffic and the old primary rejoined as a standby.
5. Take a fresh backup (new timeline).

Unplanned failover:

1. Patroni promotes automatically once the leader key expires (`ttl`).
2. Confirm the promoted node and check `maximum_lag_on_failover` was respected — quantify potential data loss from `pg_stat_replication` history or the archive.
3. Fence the old primary before it can return (Patroni's DCS leader key does this logically; a VIP or storage fence does it physically).
4. Rebuild the old primary with `pg_rewind` or from backup, then `patronictl reinit`.
5. Post-incident: reconcile any lost transactions, take a full backup, review `maximum_lag_on_failover` and `synchronous_mode`.

Data loss expectations: with `synchronous_mode: false`, you can lose anything not yet streamed. With `synchronous_mode: true` and `synchronous_mode_strict: false`, Patroni will disable synchronous replication rather than block writes when no standby is available — which quietly reintroduces the data-loss window. Set `synchronous_mode_strict: true` only if you would genuinely rather stop taking writes than lose one.

Oracle parallel. Patroni is the Data Guard Broker plus Fast-Start Failover plus the observer, implemented outside the database and backed by a real distributed consensus store. `maximum_lag_on_failover` is `FastStartFailoverLagLimit`. HAProxy plus health checks replaces TAF/FAN/ONS for client redirection — note that PostgreSQL has no equivalent of Transparent Application Failover, so in-flight transactions are always lost and the application must reconnect and retry. `pg_rewind` is `RECOVER ... USING BACKUP CONTROLFILE`-style reinstatement, closest to Flashback-based reinstatement of a failed primary. There is no RAC: PostgreSQL has no shared-disk multi-instance option at all, so scale-out means read replicas, sharding (Citus), or a distributed fork (YugabyteDB, CockroachDB).

Part VIII — Performance

22. The Query Planner

22.1 How a query becomes a plan

```
SQL text
→ parser      (syntax tree)
→ analyzer    (name resolution against the catalog)
→ rewriter    (views, rules, row-level security)
→ planner     (cost-based enumeration of plans)
→ executor    (volcano-style iterator over plan nodes)
```

The planner enumerates join orders and access methods, costs each candidate using table statistics and the cost GUCs, and picks the cheapest. Beyond `geqo_threshold` join relations (default 12) it switches to a genetic algorithm and stops being exhaustive.

Cost is a dimensionless number anchored on `seq_page_cost = 1.0`:

Parameter	Default	Meaning
<code>seq_page_cost</code>	1.0	Sequential page fetch
<code>random_page_cost</code>	4.0	Random page fetch — set to ~1.1 on SSD/NVMe
<code>cpu_tuple_cost</code>	0.01	Processing one row
<code>cpu_index_tuple_cost</code>	0.005	Processing one index entry
<code>cpu_operator_cost</code>	0.0025	One operator or function call
<code>parallel_setup_cost</code>	1000	Starting parallel workers
<code>parallel_tuple_cost</code>	0.1	Passing a row from a worker to the leader
<code>jit_above_cost</code>	100000	Compile the query with LLVM above this cost
<code>effective_cache_size</code>	4GB	Assumed cache size; affects index scan costing

22.2 Statistics

`ANALYZE` samples `300 × default_statistics_target` rows per table and stores per-column statistics in `pg_statistic`, readable through `pg_stats`:

```
SELECT attname, null_frac, avg_width, n_distinct, correlation,
       most_common_vals, most_common_freqs, histogram_bounds
FROM pg_stats WHERE schemaname='app' AND tablename='orders' AND attname='status';
```

Statistic	What it drives
<code>null_frac</code>	IS NULL selectivity
<code>n_distinct</code>	Group and join cardinality (negative = a ratio of table rows)
<code>most_common_vals / _freqs</code>	Equality selectivity for skewed values
<code>histogram_bounds</code>	Range selectivity
<code>correlation</code>	Physical/logical ordering; drives index scan versus bitmap scan

Fixing bad estimates:


```
-- More detail on a skewed column
ALTER TABLE app.orders ALTER COLUMN status SET STATISTICS 1000;
ANALYZE app.orders;

-- Tell the planner the truth when sampling gets n_distinct wrong (common on huge tables)
ALTER TABLE app.events ALTER COLUMN tenant_id SET (n_distinct = 5000);
ANALYZE app.events;

-- Extended statistics for correlated columns (PG10+)
CREATE STATISTICS orders_corr (dependencies, ndistinct, mcv)
  ON country, state, city FROM app.orders;
ANALYZE app.orders;
SELECT * FROM pg_stats_ext WHERE statistics_name = 'orders_corr';

-- Statistics on expressions (PG14+)
CREATE STATISTICS orders_expr ON (lower(email)) FROM app.orders;
```

Extended statistics are the answer to the classic "the planner thinks `country='US' AND state='CA'` is $1/50 \times 1/50$ selective, when in reality every CA row is US" problem. They fix estimates that no amount of `default_statistics_target` will.

22.3 Reading EXPLAIN

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, WAL, FORMAT TEXT)
SELECT c.name, sum(o.total)
FROM app.customers c JOIN app.orders o ON o.customer_id = c.id
WHERE o.order_date >= date '2026-08-01'
GROUP BY c.name ORDER BY 2 DESC LIMIT 20;
```

In PostgreSQL 18, `EXPLAIN ANALYZE` shows buffer counts by default, and `VERBOSE` adds CPU and average read statistics.

```
Limit (cost=51234.11..51234.16 rows=20 width=40) (actual time=812.334..812.341 rows=20 loops=1)
  Buffers: shared hit=12043 read=88231
    -> Sort (cost=51234.11..51309.22 rows=30044 width=40) (actual time=812.331..812.334 rows=20 loops=1)
      Sort Key: (sum(o.total)) DESC
      Sort Method: top-N heapsort Memory: 27kB
      -> HashAggregate (cost=50134.00..50434.44 rows=30044 width=40) (actual time=798.112..806.221
rows=29881 loops=1)
        Group Key: c.name
        Batches: 1 Memory Usage: 4113kB
        -> Hash Join (cost=1200.00..44100.00 rows=1206800 width=36) (actual time=12.441..611.882
rows=1204118 loops=1)
          Hash Cond: (o.customer_id = c.id)
          -> Index Scan using orders_date_idx on orders o (cost=0.43..38000.00 rows=1206800
width=16) (actual time=0.041..291.334 rows=1204118 loops=1)
            Index Cond: (order_date >= '2026-08-01'::date)
            -> Hash (cost=800.00..800.00 rows=32000 width=28) (actual time=12.301..12.302 rows=30012
loops=1)
              Buckets: 32768 Batches: 1 Memory Usage: 2011kB
              -> Seq Scan on customers c (cost=0.00..800.00 rows=32000 width=28)
Planning Time: 0.412 ms
Execution Time: 812.402 ms
```

How to read it:

- **Read inside-out and bottom-up.** The deepest node runs first.
- **rows estimated versus rows actual** is the first thing to check. An order-of-magnitude gap is the root cause of most bad plans, and the fix is statistics, not the query.
- **loops** multiplies: **actual time** and **rows** are **per loop**. A node showing `rows=1 loops=500000` processed 500 000 rows.
- **Buffers: shared hit=X read=Y** — `hit` came from `shared_buffers`, `read` came from the OS or disk. `dirty` and `written` show write activity. This is the most reliable measure of work done, better than time on a noisy machine.
- **Sort Method: external merge Disk: 84720kB** means the sort spilled — raise `work_mem`.
- **Batches: 8** on a Hash node means the hash table spilled to disk — raise `work_mem` or `hash_mem_multiplier`.

- **Rows Removed by Filter** — a large number means an index or a better predicate is missing.
- **Heap Fetches** on an index-only scan — a large number means the table needs vacuuming.
- **never executed** on a subplan is normal, not a warning.

`EXPLAIN (ANALYZE)` actually runs the query, including its side effects. Wrap DML in a transaction you roll back.

Use <https://explain.dalibo.com> or <https://explain.depesz.com> for visual analysis of large plans — pasting a 400-line plan into a text editor is not a good use of your time.

22.4 Scan and join nodes

Node	When chosen
Seq Scan	Small table, or a large fraction of rows needed. Not automatically bad.
Index Scan	Selective predicate, good correlation; returns rows in index order.
Index Only Scan	All needed columns in the index and pages all-visible.
Bitmap Heap Scan + Bitmap Index Scan	Medium selectivity, poor correlation; can combine several indexes with <code>BitmapAnd/BitmapOr</code> .
Tid Scan	Direct <code>ctid</code> access.
Nested Loop	Small outer side with an indexed inner side. Catastrophic if the outer estimate is wrong.
Hash Join	Large unsorted inputs, equality join. Builds a hash on the smaller side.
Merge Join	Both inputs already sorted (or cheaply sortable), equality or range.
Gather / Gather Merge	Parallel plan collection point.
Memoize (PG14+)	Caches inner-side results in a nested loop — often a large win on skewed joins.
Incremental Sort (PG13+)	Input partially sorted; sorts within groups only.

For diagnosis you can disable a node type temporarily. These are **debugging tools, not production settings**:

```
SET enable_seqscan = off;
SET enable_nestloop = off;
SET enable_hashjoin = off;
SET enable_indexscan = off;
EXPLAIN (ANALYZE) ...;
RESET ALL;
```

If forcing a different plan makes the query dramatically faster, the planner's cardinality estimate is wrong — go and fix the statistics rather than shipping the `SET`.

22.5 Parallel query

```
max_worker_processes = 16      # cluster-wide pool
max_parallel_workers = 16      # of that pool, how many for queries
max_parallel_workers_per_gather = 4 # per Gather node
max_parallel_maintenance_workers = 4 # CREATE INDEX, VACUUM
min_parallel_table_scan_size = 8MB
min_parallel_index_scan_size = 512kB
parallel_setup_cost = 1000
parallel_leader_participation = on
```

A table's parallel degree can be forced:

```
ALTER TABLE app.events SET (parallel_workers = 8);
```

Parallelism is not used for: queries writing data (except `CREATE TABLE AS`, `CREATE MATERIALIZED VIEW`, and `CREATE INDEX`), cursors, queries with `FOR UPDATE`, functions not marked `PARALLEL SAFE`, and plans inside a `SERIALIZABLE`

transaction (until PG12 — permitted since). If a plan you expect to parallelise does not, check that every function it calls is marked parallel-safe:

```
SELECT proname, proparallel FROM pg_proc WHERE proname = 'my_function';
ALTER FUNCTION my_function(int) PARALLEL SAFE;
```

22.6 JIT

LLVM JIT compilation is on by default above `jit_above_cost = 100000`. It helps long analytical queries and actively hurts short ones whose cost estimate is inflated — compilation can take longer than the query. If you see `JIT: Functions: 42 ... Timing: Generation 12ms, Inlining 88ms, Optimization 320ms, Emission 210ms` next to a 50 ms execution, turn it off for that workload:

```
SET jit = off; -- session
ALTER DATABASE oltp SET jit = off; -- whole database; a common choice for OLTP
```

23. Finding the Queries That Matter

23.1 pg_stat_statements

```
shared_preload_libraries = 'pg_stat_statements'
pg_stat_statements.max = 10000
pg_stat_statements.track = top # top | all | none
pg_stat_statements.track_utility = on
pg_stat_statements.track_planning = off # meaningful overhead; leave off unless investigating
compute_query_id = on
```

```
CREATE EXTENSION pg_stat_statements;

-- Top consumers by total time
SELECT queryid,
       round(total_exec_time::numeric, 1) AS total_ms,
       calls,
       round(mean_exec_time::numeric, 3) AS mean_ms,
       round(stddev_exec_time::numeric, 3) AS stddev_ms,
       rows,
       round(100.0*shared_blks_hit/nullif(shared_blks_hit+shared_blks_read,0),1) AS hit_pct,
       round(total_exec_time::numeric*100/sum(total_exec_time::numeric) OVER (),1) AS pct_of_total,
       left(query, 120) AS query
FROM pg_stat_statements
WHERE dbid = (SELECT oid FROM pg_database WHERE datname = current_database())
ORDER BY total_exec_time DESC LIMIT 25;

-- Queries that spill to disk
SELECT queryid, calls, temp_blks_written, left(query,100)
FROM pg_stat_statements WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC LIMIT 20;

-- Queries with high variance – the ones users complain about intermittently
SELECT queryid, calls, mean_exec_time, stddev_exec_time, max_exec_time, left(query,100)
FROM pg_stat_statements
WHERE calls > 100 AND stddev_exec_time > mean_exec_time
ORDER BY stddev_exec_time DESC LIMIT 20;

-- Write-heavy statements (WAL, PG13+)
SELECT queryid, calls, wal_records, pg_size_pretty(wal_bytes) AS wal, left(query,100)
FROM pg_stat_statements ORDER BY wal_bytes DESC LIMIT 20;

SELECT pg_stat_statements_reset();
```

Snapshot `pg_stat_statements` into a history table on a schedule (via `pg_cron`) and you have built a rudimentary AWR. Several projects do this properly: **pgwatch2**, **PoWA** (which adds plan capture and index advice), **pganalyze** (commercial), and **pgCenter** for a live top-like view.

23.2 auto_explain

```
shared_preload_libraries = 'auto_explain'
auto_explain.log_min_duration = 3000
auto_explain.log_analyze = on
auto_explain.log_buffers = on
auto_explain.log_timing = off      # per-node timing is expensive; off keeps overhead ~2 %
auto_explain.log_verbose = on
auto_explain.log_nested_statements = on
auto_explain.log_format = json
auto_explain.sample_rate = 0.1    # sample 10 % of qualifying statements
```

`auto_explain` captures the plan of the slow execution — not the plan you get when you run `EXPLAIN` afterwards on an idle system with different parameters. For intermittent problems it is the only thing that works.

23.3 Wait events

`pg_stat_activity` exposes what each backend is waiting on right now:

```
SELECT wait_event_type, wait_event, state, count(*)
FROM pg_stat_activity
WHERE backend_type = 'client backend'
GROUP BY 1,2,3 ORDER BY 4 DESC;
```

wait_event_type	Common events	Usually means
Lock	transactionid, relation, tuple	Blocking — see Chapter 13
LWLock	WALWrite, BufferMapping, ProcArray, LockManager	Internal contention; often too many connections
IO	DataFileRead, WALSync, DataFileExtend	Storage-bound
Client	ClientRead	Waiting on the application — often "idle in transaction"
IPC	MessageQueueSend, ParallelFinish	Parallel worker coordination
Timeout	VacuumDelay, PgSleep	Throttling
BufferPin		A query waiting for a pinned buffer

PostgreSQL has no built-in ASH-style history. Sample `pg_stat_activity` every second into a table (`pg_cron` plus an insert, or `pg_wait_sampling`, or `pgsentinel` which implements a genuine `pg_active_session_history`) and you can do real wait-event analysis.

```
CREATE EXTENSION pg_wait_sampling;
SELECT event_type, event, count FROM pg_wait_sampling_profile ORDER BY count DESC LIMIT 20;
```

23.4 I/O statistics

PostgreSQL 16 introduced `pg_stat_io`, which is the closest thing to a system-level I/O breakdown:

```
SELECT backend_type, object, context,
       reads, read_time, writes, write_time, extends, evictions, hits,
       round(100.0*hits/nullif(hits+reads,0),1) AS hit_pct
FROM pg_stat_io
WHERE reads > 0 OR writes > 0
ORDER BY reads DESC;
```

`context = 'bulkread' / 'bulkwrite'` shows ring-buffer activity (sequential scans and COPY), `'vacuum'` shows vacuum's own I/O, and `'normal'` is everything else. High `evictions` in the `normal` context is a signal that `shared_buffers` is too small.

```
-- Classic cache hit ratio (aim for > 99 % on OLTP, but treat it as a smell, not a target)
SELECT datname,
       round(100.0*blks_hit/nullif(blks_hit+blks_read,0),2) AS cache_hit_pct,
       xact_commit, xact_rollback, tup_returned, tup_fetched,
       temp_files, pg_size_pretty(temp_bytes) AS temp,
       deadlocks, conflicts, blk_read_time, blk_write_time
FROM pg_stat_database WHERE datname NOT LIKE 'template%';
```

Enable `track_io_timing = on` to populate the timing columns — the overhead is negligible on modern hardware (test with `pg_test_timing`).

23.5 Buffer cache contents

```
CREATE EXTENSION pg_buffercache;
SELECT c.relname,
       count(*) AS buffers,
       pg_size_pretty(count(*) * 8192::bigint) AS size,
       round(100.0*count(*) FILTER (WHERE b.isdirty) / count(*), 1) AS dirty_pct
FROM pg_buffercache b JOIN pg_class c ON c.relfilenode = b.relfilenode
GROUP BY 1 ORDER BY 2 DESC LIMIT 20;
```

`pg_prewarm` can load a relation back into the cache after a restart, and `pg_prewarm.autoprewarm = on` dumps and restores the buffer contents automatically across restarts — worth enabling on large OLTP systems where a cold cache means a slow morning.

24. A Query Tuning Workflow

1. **Find the query.** `pg_stat_statements` ordered by `total_exec_time`. Optimise what consumes the most time in aggregate, not the query someone complained about.
2. **Reproduce it** with realistic parameters. Beware of plans that differ between a literal and a bind parameter — use `PREPARE/EXECUTE` to reproduce the prepared-statement plan.
3. **Get a real plan.** `EXPLAIN (ANALYZE, BUFFERS)`, or `auto_explain` output from production.
4. **Find the first node where estimate and actual diverge.** Work outward from the deepest such node.
5. **Fix the cause, in this order:**
 - Missing or stale statistics → `ANALYZE`, raise `SET STATISTICS`, add extended statistics.
 - Missing index → add one; validate with `hypopg` first.
 - Query shape → rewrite: replace `NOT IN` with `NOT EXISTS`, avoid functions on indexed columns (`WHERE date_trunc('day', ts) = x` cannot use an index on `ts`; `WHERE ts >= x AND ts < x+1` can), avoid `OR` across columns (use `UNION ALL`), avoid implicit casts that defeat indexes.
 - Data type mismatch → `bigint` column compared to an `int` parameter can prevent index use in some drivers.
 - Memory → raise `work_mem` for that session if the plan spills.
 - Cost parameters → `random_page_cost`, `effective_cache_size` if the planner systematically prefers sequential scans.
1. **Re-measure with `BUFFERS`**, not just wall-clock time.
2. **Check the regression risk:** does the new index slow down writes? Does the rewrite change semantics for `NULL`s?

Common rewrites that pay off:

```
-- NOT IN with a nullable column is both slow and semantically dangerous
SELECT * FROM a WHERE id NOT IN (SELECT b_id FROM b);           -- avoid
SELECT * FROM a WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.b_id = a.id); -- prefer

-- OFFSET-based pagination degrades linearly; use keyset pagination
SELECT * FROM orders ORDER BY id LIMIT 50 OFFSET 100000;      -- avoid
SELECT * FROM orders WHERE id > $last_id ORDER BY id LIMIT 50; -- prefer

-- Function on the indexed column defeats the index
```

```

WHERE lower(email) = 'a@b.c'           -- needs an expression index on lower(email)
WHERE ts::date = '2026-09-01'         -- avoid
WHERE ts >= '2026-09-01' AND ts < '2026-09-02' -- prefer

-- DISTINCT that should be a semi-join
SELECT DISTINCT a.* FROM a JOIN b ON b.a_id = a.id;           -- avoid
SELECT * FROM a WHERE EXISTS (SELECT 1 FROM b WHERE b.a_id = a.id); -- prefer

-- Counting: exact counts on huge tables are expensive; an estimate is often enough
SELECT reltuples::bigint FROM pg_class WHERE oid = 'app.orders'::regclass;

```

24.1 Plan stability

PostgreSQL has no plan baselines. Options when a plan flips:

- **Fix the estimate** (extended statistics, `SET STATISTICS`, `n_distinct` override). This is the durable fix.
- **pg_hint_plan** — an extension providing Oracle-style hints in a leading comment. Effective, but unavailable on most managed services and it decouples the plan from the data.

```

/*+ IndexScan(orders orders_date_idx) Leading((orders customers)) */
SELECT ...;

```

- **plan_cache_mode** — controls the generic-versus-custom plan decision for prepared statements:

```

SET plan_cache_mode = 'force_custom_plan'; -- replan every execution
SET plan_cache_mode = 'force_generic_plan'; -- one plan for all parameter values

```

The default (`auto`) builds five custom plans and then switches to a generic plan if it is not more expensive on average. On skewed data this produces the classic "fast for a week, then suddenly slow" symptom; `force_custom_plan` is the usual cure for a specific statement.

Oracle parallel. There is no SQL Plan Management, no baselines, no profiles, no SQL Tuning Advisor, no `DBMS_STATS` lock-and-copy of statistics, and no adaptive cursor sharing beyond `plan_cache_mode`. The compensating strength is that PostgreSQL's planner is simpler and more predictable, and that statistics fixes tend to be durable. Bind peeking's Oracle-style plan instability shows up here as the generic-versus-custom plan decision.

Part IX — Operations

25. Monitoring

25.1 The statistics views

PostgreSQL's monitoring surface is a set of **cumulative counters since the last reset**, not a point-in-time wait interface. To get rates you must sample and differentiate.

View	Content
<code>pg_stat_activity</code>	One row per backend: state, query, wait event, transaction age
<code>pg_stat_database</code>	Per-database: commits, rollbacks, blocks read/hit, temp files, deadlocks, conflicts, session time (PG14+)
<code>pg_stat_database_conflicts</code>	Standby recovery conflicts by cause
<code>pg_stat_user_tables</code>	Per-table: seq/index scans, tuples in/out, dead tuples, vacuum/analyze timestamps and counts
<code>pg_stat_user_indexes</code>	Per-index: scans, tuples read/fetched, <code>last_idx_scan</code> (PG16+)
<code>pg_statio_user_tables / _indexes</code>	Block-level hit/read for heap, index, TOAST

View	Content
pg_stat_io	PG16+: reads/writes/extends/evictions by backend type, object, context
pg_stat_wal	WAL records, bytes, buffers full, write/sync counts and time
pg_stat_bgwriter	Background writer activity (checkpoint columns moved to pg_stat_checkpoint in PG17)
pg_stat_checkpoint	PG17+: timed/requested checkpoints, write and sync time
pg_stat_replication	Per-standby lag in bytes and time
pg_stat_replication_slots	Per-slot spill and stream counts for logical decoding
pg_stat_wal_receiver	Standby side of streaming
pg_stat_subscription, pg_stat_subscription_stats	Logical replication apply workers and errors
pg_stat_archiver	Archived and failed WAL counts
pg_stat_ssl, pg_stat_gssapi	Per-connection encryption details
pg_stat_progress_*	Live progress for vacuum, analyze, cluster, create index, basebackup, copy
pg_locks	Every lock, granted or waiting
pg_replication_slots	Slot status and retained WAL
pg_stat_statements	Extension: per-normalised-statement aggregates

```

SELECT pg_stat_reset();                -- current database counters
SELECT pg_stat_reset_shared('bgwriter'); -- also 'archiver', 'io', 'wal', 'recovery_prefetch'
SELECT pg_stat_reset_single_table_counters('app.orders'::regclass);

```

25.2 What to alert on

Signal	Query / source	Threshold
Cluster down	TCP + SELECT 1	any failure
Connection saturation	count(*) FROM pg_stat_activity vs max_connections	> 80 %
Replication lag (time)	pg_stat_replication.replay_lag	> 60 s warn, > 300 s page
Standby not streaming	pg_stat_replication.state <> 'streaming'	any
Replication slot retention	pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)	> 10 GB warn
Inactive slot	pg_replication_slots.active = false	> 15 min
XID wraparound	age(datfrozenxid)	> 500 M warn, > 1 B page
Multixact age	mxid_age(datminmxid)	same
Archiver failures	pg_stat_archiver.failed_count increasing	any increase
pg_wal size	filesystem	> 2 × max_wal_size
Disk free	filesystem	< 20 % warn, < 10 % page
Long transaction	now() - xact_start	> 15 min warn, > 1 h page
Idle in transaction	state = 'idle in transaction' age	> 5 min
Blocked sessions	pg_blocking_pids() non-empty	> 60 s
Deadlocks	pg_stat_database.deadlocks rate	any sustained increase
Dead tuple ratio	pg_stat_user_tables	> 20 % on tables > 1 GB

Signal	Query / source	Threshold
Autovacuum saturation	workers at <code>autovacuum_max_workers</code>	sustained > 30 min
Checkpoints requested	<code>num_requested / (num_timed+num_requested)</code>	> 10 %
Temp file bytes	<code>pg_stat_database.temp_bytes</code> rate	sustained growth
Checksum failures	<code>pg_stat_database.checksum_failures</code>	any
Backup age	backup tool	> RPO
Cache hit ratio	<code>pg_stat_database</code>	< 95 % sustained (context-dependent)

25.3 Tooling

Layer	Options
Metrics exporter	<code>postgres_exporter</code> (Prometheus), <code>pgwatch2</code> , Datadog/New Relic agents
Dashboards	Grafana (the community PostgreSQL dashboards are a reasonable start)
Log analysis	<code>pgBadger</code> — run it nightly against the previous day's logs; it is excellent and free
Query history	PoWA, <code>pganalyze</code> , <code>pg_stat_statements</code> snapshots
Wait sampling	<code>pg_wait_sampling</code> , <code>pgsentinel</code>
Live top	<code>pg_activity</code> , <code>pgCenter</code>
Health check	<code>pgDash</code> , <code>check_postgres</code> (Nagios/Icinga plugin)

```
pgbadger -j 4 -f stderr -o /var/www/pgbadger/$(date +%F).html \
/var/lib/postgresql/18/data/log/postgresql-$(date -d yesterday +%F)*.log
```

`log_line_prefix = '%m [%p] %q%u@%d app=%a %h '` is the minimum `pgBadger` needs; add `%e` for SQLSTATE and consider `jsonlog` if you ship to a structured log platform.

25.4 A daily health-check script

```
\pset format aligned
\echo '=== Version and uptime ==='
SELECT version(), pg_postmaster_start_time(), now() - pg_postmaster_start_time() AS uptime;

\echo '=== Database sizes ==='
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size,
       numbackends, xact_commit, xact_rollback,
       round(100.0*blks_hit/nullif(blks_hit+blks_read,0),2) AS hit_pct,
       temp_files, pg_size_pretty(temp_bytes) AS temp, deadlocks
FROM pg_stat_database WHERE datname NOT LIKE 'template%';

\echo '=== Wraparound risk ==='
SELECT datname, age(datfrozenxid) AS xid_age, mxid_age(datminmxid) AS mxid_age
FROM pg_database ORDER BY 2 DESC LIMIT 5;

\echo '=== Replication ==='
SELECT application_name, state, sync_state, write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;

\echo '=== Archiver ==='
SELECT archived_count, last_archived_time, failed_count, last_failed_time FROM pg_stat_archiver;

\echo '=== Checkpoints ==='
SELECT * FROM pg_stat_checkpoint;
```



```
\echo '=== Long transactions ==='
SELECT pid, username, application_name, state, now()-xact_start AS xact_age, left(query,80)
FROM pg_stat_activity WHERE xact_start < now() - interval '5 minutes' ORDER BY xact_start;

\echo '=== Bloat candidates ==='
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/nullif(n_live_tup+n_dead_tup,0),1) AS dead_pct, last_autovacuum
FROM pg_stat_user_tables WHERE n_dead_tup > 100000 ORDER BY 5 DESC LIMIT 15;

\echo '=== Unused large indexes ==='
SELECT relname, indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes WHERE idx_scan < 50
AND pg_relation_size(indexrelid) > 50*1024*1024
ORDER BY pg_relation_size(indexrelid) DESC LIMIT 15;

\echo '=== Invalid indexes and constraints ==='
SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;
SELECT conrelid::regclass, conname FROM pg_constraint WHERE NOT convalidated;
```

26. Routine Maintenance

Cadence	Task
Continuous	Autovacuum and autoanalyze (monitored, not scheduled)
Continuous	WAL archiving; alert on failed_count
Hourly	Incremental backup; slot lag check
Daily	Differential backup; pgBadger report; health-check script; disk trend review
Weekly	Full backup; restore test ; <code>pg_amcheck</code> on a restored copy; unused-index review
Monthly	Minor version review; extension version review; <code>REINDEX CONCURRENTLY</code> on identified bloated indexes; capacity review
Quarterly	Failover drill; DR test; parameter review against workload change; certificate expiry check
Annually	Major version upgrade planning; OS/glibc upgrade impact review (collations!)

26.1 Scheduling with pg_cron

```
CREATE EXTENSION pg_cron; -- needs cron.database_name in postgresql.conf and a restart

SELECT cron.schedule('nightly-analyze', '0 2 * * *', $$ANALYZE$$);
SELECT cron.schedule('partman', '@hourly', $$CALL partman.run_maintenance_proc()$$);
SELECT cron.schedule('pgss-snapshot', '*/10 * * * *',
  $$INSERT INTO ops.pgss_history SELECT now(), * FROM pg_stat_statements$$);
SELECT cron.schedule_in_database('vacuum-hot', '30 3 * * 0', $$VACUUM (ANALYZE) app.orders$$, 'appdb');

SELECT * FROM cron.job;
SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 20;
SELECT cron.unschedule('nightly-analyze');
```

Note `pg_cron` runs on the primary only; after a failover the jobs run wherever the extension is installed and the database is writable — check that your HA tooling accounts for it (Patroni callbacks can enable/disable `cron.job` rows on promotion).

26.2 Statistics after bulk operations

`ANALYZE` is not automatic after `COPY`, `pg_restore`, or a large `INSERT ... SELECT` until autovacuum's threshold is crossed — which may be long after the queries start running badly. Always analyze explicitly:

```
COPY app.staging FROM '/data/load.csv' WITH (FORMAT csv, HEADER);
ANALYZE app.staging;
```

After a full restore, `vacuumdb --analyze-in-stages` gets usable statistics quickly and then refines them.

27. Upgrades

27.1 Minor version upgrades

Minor releases (18.5 → 18.6) never change the on-disk format. The procedure is: install the new binaries, restart.

```
dnf update postgresql18-server postgresql18-contrib
systemctl restart postgresql-18
```

In a Patroni cluster, upgrade standbys first, then switch over and upgrade the old primary — with `patronictl pause` around the whole operation. Read the release notes: occasionally a minor release requires a `REINDEX` (for example after a `btree` or collation-related bug fix) or an extension update.

Do not skip minor releases. They are cumulative bug and security fixes with an extremely good track record.

27.2 Major version upgrades

Three options:

Method	Downtime	Risk	When
<code>pg_dumpall</code> / <code>pg_dump</code> + restore	Hours to days	Low	Small databases; changing architecture, encoding, or collation
<code>pg_upgrade --link</code> (or <code>--clone</code> , <code>--swap</code>)	Minutes	Medium	The standard choice
Logical replication	Seconds to minutes	Medium–high	Large, busy clusters that cannot take minutes of downtime

`pg_upgrade` procedure:

```
# 1. Install the new major version alongside the old
dnf -y install postgresql19-server postgresql19-contrib
sudo -u postgres /usr/pgsql-19/bin/initdb -D /var/lib/pgsql/19/data \
    --data-checksums --locale=C # MUST match the old cluster's checksum and locale settings

# 2. Check compatibility WITHOUT touching anything (old cluster can stay up)
sudo -u postgres /usr/pgsql-19/bin/pg_upgrade \
    --old-bindir=/usr/pgsql-18/bin --new-bindir=/usr/pgsql-19/bin \
    --old-datadir=/var/lib/pgsql/18/data --new-datadir=/var/lib/pgsql/19/data \
    --jobs=8 --check

# 3. Take a backup. Then stop both clusters.
systemctl stop postgresql-18

# 4. Upgrade. --link uses hard links (fast, but the old cluster becomes unusable).
#    --clone uses reflinks on XFS/Btrfs/APFS (fast AND keeps the old cluster usable).
#    --swap (PG18+) moves the directories instead of copying – fastest for many objects.
sudo -u postgres /usr/pgsql-19/bin/pg_upgrade \
    --old-bindir=/usr/pgsql-18/bin --new-bindir=/usr/pgsql-19/bin \
    --old-datadir=/var/lib/pgsql/18/data --new-datadir=/var/lib/pgsql/19/data \
    --link --jobs=8

# 5. Copy config, start, and re-analyze
#    PG18+ carries column statistics across (extended statistics are NOT preserved),
#    so this step is far less urgent than it used to be – but still run it.
systemctl start postgresql-19
sudo -u postgres /usr/pgsql-19/bin/vacuumdb --all --analyze-in-stages --jobs=8

# 6. Verify, then reclaim
./delete_old_cluster.sh
```

Things that catch people out:

- **Standbys are not upgraded by `pg_upgrade`.** With `--link` you must rebuild every standby from the upgraded primary (`pg_basebackup`), or use the `rsync` procedure documented in the `pg_upgrade` manual page. Plan for it.
- **Extensions must exist in the new version** before the upgrade, and be `ALTER EXTENSION ... UPDATED` afterwards.
- **The new cluster must match** on `--data-checksums`, locale, and encoding.
- `--check` is safe to run against a live old cluster and should be run days in advance.
- Logical replication slots are not carried over before PG17 (PG17+ can preserve them, which enables in-place upgrades of publishers).

Pre-upgrade checklist:

```
-- Extensions and their versions
SELECT extname, extversion FROM pg_extension;
SELECT * FROM pg_available_extension_versions WHERE name = 'postgis';
-- Deprecated / removed types and functions in the target version (read the release notes)
-- Unlogged tables (their data is not carried over by pg_upgrade)
SELECT relname FROM pg_class WHERE relpersistence = 'u';
-- Prepared transactions must be zero
SELECT * FROM pg_prepared_xacts;
-- Invalid indexes and unvalidated constraints should be fixed first
SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;
```

27.3 The logical replication upgrade path

Covered in Section 20.2. Use it when `pg_upgrade`'s few minutes of downtime is still too much, or when you also want to change the operating system, the hardware, the collation, or the encoding at the same time. Budget for the sequence fix-up and for the fact that DDL changes must be frozen during the migration window.

28. Extensions

```
SELECT * FROM pg_available_extensions ORDER BY name;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
ALTER EXTENSION postgis UPDATE TO '3.5.0';
SELECT extname, extversion, extnamespace::regnamespace FROM pg_extension;
DROP EXTENSION pg_trgm CASCADE;
```

The extensions worth knowing:

Extension	Purpose
<code>pg_stat_statements</code>	Query performance aggregates. Install everywhere.
<code>auto_explain</code>	Plans for slow queries. Install everywhere.
<code>pgaudit</code>	Compliance auditing
<code>pg_cron</code>	In-database scheduler
<code>pg_partman</code>	Partition lifecycle automation
<code>pg_repack</code>	Online table/index rebuild
<code>pgstattuple</code> , <code>pgstatindex</code>	Accurate bloat measurement
<code>pageinspect</code> , <code>pg_visibility</code> , <code>amcheck</code>	Low-level inspection and verification
<code>pg_buffercache</code> , <code>pg_prewarm</code>	Buffer cache inspection and warming
<code>pg_trgm</code>	Trigram similarity and fast LIKE '%x%'
<code>btree_gin</code> , <code>btree_gist</code>	Mix scalar columns into GIN/GiST indexes and exclusion constraints
<code>postgres_fdw</code> , <code>file_fdw</code> , <code>oracle_fdw</code> , <code>tds_fdw</code>	Foreign data wrappers
<code>dblink</code>	Ad-hoc cross-database queries; also the standard autonomous-transaction workaround

Extension	Purpose
hstore, ltree, citext, uuid-oss, intarray	Useful data types
pgcrypto	Hashing and encryption functions
PostGIS	Spatial. Best-in-class.
TimescaleDB	Time-series: automatic partitioning, continuous aggregates, compression
Citus	Distributed/sharded PostgreSQL
pgvector	Vector similarity search for embeddings
orafce	Oracle compatibility functions
pg_hint_plan	Planner hints
hypopg	Hypothetical indexes
pg_wait_sampling, pgsentinel	Wait-event history
wal2json	Logical decoding to JSON
plpgsql_check	Static analysis and linting of PL/pgSQL
pg_squeeze	Alternative online table rewrite
pgbouncer/pgpool-II	(Not extensions, but part of the standard toolkit)

`shared_preload_libraries` requires a restart and is the gate for `pg_stat_statements`, `auto_explain`, `pgaudit`, `pg_cron`, `pg_wait_sampling`, `pg_prewarm` (for `autoprewarm`), and `citus`.

Extension discipline: pin versions, test upgrades in staging, and check availability on your target platform before adopting one — managed services allow only a curated list, and an extension you depend on can block a cloud migration entirely.

29. Managed PostgreSQL in the Cloud

29.1 What changes

Capability	Self-managed	AWS RDS	Aurora PostgreSQL	Azure Flexible Server	Google Cloud SQL / AlloyDB	OCI Database with PostgreSQL
Superuser	Yes	No (<code>rds_superuser</code>)	No	No (<code>azure_pg_admin</code>)	No (<code>cloudsqlsuperuser</code>)	No
<code>shared_preload_libraries</code>	Free	Parameter group, curated list	Curated	Curated	Curated	Curated
Filesystem / COPY FROM PROGRAM	Yes	No	No	No	No	No
Custom extensions	Any	Curated list	Curated	Curated	Curated	Curated
<code>pg_hba.conf</code>	Direct	Parameter + security groups	Same	Firewall + <code>azure.*</code> params	Authorized networks	NSG
Backups	Your tool	Automated snapshots + PITR	Continuous to S3, PITR to the second	Automated + PITR	Automated + PITR	Automated + PITR

Capability	Self-managed	AWS RDS	Aurora PostgreSQL	Azure Flexible Server	Google Cloud SQL / AlloyDB	OCI Database with PostgreSQL
Replication	Yours	Read replicas, cross-region	Shared storage, up to 15 replicas, sub-10 ms lag	Read replicas	Read replicas	Read replicas
Major upgrade	You	In-place or Blue/Green	In-place	In-place	In-place	In-place
Storage engine	Standard	Standard	Log-structured, shared, 6-way replicated	Standard	Standard / AlloyDB columnar engine	Standard

Aurora is the biggest departure: it replaces the storage layer entirely, so `full_page_writes` is unnecessary, checkpoints behave differently, `pg_basebackup` and physical replication as you know them do not apply, and some `pg_stat_*` counters mean different things. Aurora's failover is fast because replicas share storage; its write throughput characteristics and its behaviour under vacuum pressure differ from community PostgreSQL and need their own testing.

29.2 Operating a managed instance

What you still own:

- **Schema and index design** — no provider fixes this for you.
- **Vacuum and bloat.** Managed services set autovacuum defaults that are frequently too lax for large tables. Per-table `reloptions` are yours to set and are the main lever.
- **Wraparound monitoring.** RDS exposes `maximum_used_transaction_ids` in CloudWatch; use it.
- **Connection management.** RDS Proxy, Azure's built-in PgBouncer, or your own pooler.
- **Query performance.** `pg_stat_statements` is available everywhere; Performance Insights and Query Store are the provider-specific equivalents.
- **Parameter tuning.** Parameter groups are the interface; many parameters are locked or formula-derived from instance size.
- **Extension and version lifecycle**, including forced minor upgrades during maintenance windows.

What you give up: filesystem access, superuser, `pg_repack` in some cases (it needs superuser on some platforms), custom `archive_command`, physical backups you control, and the ability to install an arbitrary extension.

For an Oracle-to-PostgreSQL migration targeting a managed service, verify the extension list and the superuser restrictions **before** designing the target schema. `oracle_fdw` in particular is unavailable on most managed platforms, which changes the migration architecture.

30. Troubleshooting Playbooks

30.1 "The database is slow"

```
-- 1. What is running right now, and what is it waiting on?
SELECT pid, username, application_name, state, wait_event_type, wait_event,
       now()-query_start AS runtime, left(query,100)
FROM pg_stat_activity
WHERE state <> 'idle' AND backend_type='client backend'
```

```

ORDER BY query_start;

-- 2. Is anything blocked?
SELECT blocked.pid, blocking.pid AS by, now()-blocked.query_start AS waiting,
       left(blocked.query,60) AS blocked_q, left(blocking.query,60) AS blocking_q
FROM pg_stat_activity blocked
JOIN LATERAL unnest(pg_blocking_pids(blocked.pid)) bp(pid) ON true
JOIN pg_stat_activity blocking ON blocking.pid = bp.pid;

-- 3. Aggregate wait profile
SELECT wait_event_type, wait_event, count(*) FROM pg_stat_activity
WHERE state='active' GROUP BY 1,2 ORDER BY 3 DESC;

-- 4. Which statements consume the most time overall?
SELECT calls, round(total_exec_time::numeric,0) AS total_ms,
       round(mean_exec_time::numeric,2) AS mean_ms, left(query,100)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

```

Then check, in order: connection count against `max_connections`; checkpoint storms (`log_checkpoints` output); autovacuum activity; disk latency at the OS level (`iostat -xz 1`); and whether a plan changed (compare `mean_exec_time` over time from your `pg_stat_statements` snapshots).

30.2 "Disk is filling up"

```

-- Where is the space?
SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database ORDER BY 2 DESC;
SELECT nspname||'.'||relname AS relation,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total,
       pg_size_pretty(pg_relation_size(c.oid)) AS heap,
       pg_size_pretty(pg_indexes_size(c.oid)) AS idx
FROM pg_class c JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE relkind IN ('r','m','p') ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 20;

-- pg_wal not being recycled? Check these four in order:
SELECT * FROM pg_stat_archiver;           -- failing archive_command
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;               -- abandoned slot
SHOW max_wal_size; SHOW wal_keep_size;   -- configuration
SELECT * FROM pg_stat_replication;       -- a standby that stopped consuming

```

Emergency space recovery, safest first: drop an abandoned replication slot; fix and let the archiver drain; `VACUUM (INDEX_CLEANUP OFF, TRUNCATE ON)` on the biggest bloated table; drop unused indexes; drop old partitions. **Never delete files from `pg_wal` manually** — use `pg_archivecleanup` if you must, and only after understanding what is still needed.

30.3 "Cannot connect"

Work down the list: network/firewall reachability to the port; `max_connections` exhausted (the log says `sorry, too many clients already`); `pg_hba.conf` (check `pg_hba_file_rules` for the first matching line); password or SCRAM support in the driver; TLS (`sslmode`, certificate expiry, CA path); role `VALID UNTIL` expiry; `CONNECT` privilege on the database; per-role or per-database `CONNECTION LIMIT`; and the database being in recovery when the client requires `target_session_attrs=read-write`.

30.4 "Server won't start"

Read the log — PostgreSQL is unusually good at saying why.

Message	Cause
could not bind IPv4 address ... Address already in use	Another postmaster or a stale <code>postmaster.pid</code>
lock file "postmaster.pid" already exists	Stale PID file after a crash; verify no process, then remove it
database files are incompatible with server	Wrong binary major version for this <code>\$PGDATA</code>

Message	Cause
could not access file "pg_stat_statements": No such file	An entry in <code>shared_preload_libraries</code> whose package is missing
FATAL: could not map anonymous shared memory	<code>shared_buffers</code> /huge pages exceed what the kernel will give; check <code>vm.nr_hugepages</code> and <code>huge_pages=on</code>
PANIC: could not locate a valid checkpoint record	<code>pg_control</code> or WAL damage — restore from backup
invalid value for parameter	Typo in <code>postgresql.conf</code> or <code>postgresql.auto.conf</code>

30.5 "A query that was fast is now slow"

1. Compare `mean_exec_time` history in `pg_stat_statements` snapshots to establish when it changed.
2. `EXPLAIN (ANALYZE, BUFFERS)` now; compare against an `auto_explain` capture from before.
3. Check whether statistics are stale (`last_analyze`, `last_autoanalyze`) or whether the data distribution changed.
4. Check for a generic-plan flip on a prepared statement (`plan_cache_mode = force_custom_plan` to test).
5. Check for bloat or a lost index-only scan (`Heap Fetches`).
6. Check for an index that was dropped, or invalidated by a failed `CREATE INDEX CONCURRENTLY`.
7. Check for a partition added without matching indexes.

31. Migrating from Oracle

31.1 Assessment

Inventory before you plan: object counts by type, PL/SQL line counts, use of features with no PostgreSQL equivalent (autonomous transactions, `DBMS_*` packages, hierarchical queries, analytic functions, materialized view fast refresh, Advanced Queuing, Spatial, Text, partitioning specifics, RAC dependencies), the number of application SQL statements, and the data volume.

Tools:

- **ora2pg** — the mainstay. Assessment report (`ora2pg -t SHOW_REPORT --estimate_cost`), schema conversion, data migration, PL/SQL conversion with a difficulty score.
- **AWS Schema Conversion Tool (SCT)** and **DMS** — assessment plus schema conversion plus CDC-based data movement.
- **Ora2Pg + Debezium/GoldenGate** for the ongoing-replication phase.
- **EDB Migration Portal / Migration Toolkit** — particularly if targeting EDB Advanced Server.

```
ora2pg -c ora2pg.conf -t SHOW_REPORT --estimate_cost --dump_as_html > assessment.html
ora2pg -c ora2pg.conf -t TABLE -o tables.sql -b ./schema
ora2pg -c ora2pg.conf -t PACKAGE -o packages.sql -b ./plpgsql
ora2pg -c ora2pg.conf -t COPY -j 8 -J 4 # parallel data load
```

31.2 The conversion issues that actually cost time

Issue	Resolution
<code>' '</code> vs <code>NULL</code>	Audit every string predicate and <code>NVL</code> . There is no shortcut.
Oracle <code>DATE</code> has a time part	Map to <code>timestamp(0)</code> , not <code>date</code> .
Identifier case folding	Fold everything to lower case; never quote identifiers.
30-character identifier limit vs 63	Fine in one direction; watch generated names.
<code>NUMBER</code> → <code>numeric</code> everywhere	Map integral columns to <code>int/bigint</code> ; <code>numeric</code> arithmetic is 10–100× slower.

Issue	Resolution
Implicit type conversion	Oracle is permissive; PostgreSQL is strict. Expect errors in application SQL.
ROWNUM	<code>LIMIT</code> , or <code>row_number()</code> when it is used mid-query.
<code>CONNECT BY</code>	<code>WITH RECURSIVE</code> ; a mechanical but sizeable rewrite.
<code>MERGE</code>	Native since PG15; <code>INSERT ... ON CONFLICT</code> before that.
Packages	Become schemas of functions. Package-level variables have no equivalent — use a session-scoped temp table or <code>set_config()</code> .
Autonomous transactions	<code>dblink</code> back to the same database, or restructure.
<code>DBMS_OUTPUT</code> , <code>UTL_FILE</code> , <code>DBMS_LOB</code>	<code>orafce</code> covers much of this.
Sequences in triggers	Convert to <code>GENERATED ALWAYS AS IDENTITY</code> .
Global temporary tables	Session temp tables — but beware catalog bloat from high-frequency creation; consider <code>ON COMMIT DELETE ROWS</code> on a persistent table plus RLS, or unlogged tables.
Hints	Delete them. Fix estimates instead.
Database links	<code>postgres_fdw</code> / <code>oracle_fdw</code> .
Jobs (<code>DBMS_SCHEDULER</code>)	<code>pg_cron</code> or external scheduler.
Advanced Queuing	<code>LISTEN/NOTIFY</code> plus a table, <code>pgmq</code> , or an external broker.
Materialized view fast refresh	Full refresh (<code>CONCURRENTLY</code> to stay online), or incremental logic you write, or <code>pg_ivm</code> .

31.3 Cutover with CDC

For a large, always-on system, the pattern is: initial bulk load, then continuous change capture from Oracle to PostgreSQL, then a short cutover window.

```

Oracle  --(GoldenGate / DMS / Debezium-Oracle)-->  Kafka  -->  PostgreSQL
|
+----- validation / row comparison -----+

```

Cutover checklist:

1. Freeze DDL on the source.
2. Confirm replication lag is zero and stable.
3. Quiesce the application; drain in-flight transactions.
4. Final lag check; stop the CDC pipeline.
5. **Advance all PostgreSQL sequences** past the maximum key values.
6. Run `ANALYZE` across the target (or `vacuumdb --analyze-in-stages`).
7. Row-count and checksum validation on critical tables.
8. Repoint the application; smoke test.
9. Keep Oracle read-only and available for a defined rollback window.

Validation is the step teams under-invest in. At minimum, compare row counts and aggregate checksums per table; better, compare sampled rows column by column. This is the role Oracle Veridata plays in an Oracle-to-Oracle project; on the PostgreSQL side the practical equivalents are custom checksum queries, `pgcompare`, or `data-diff`-style tooling.

```

-- A simple, parallelisable per-table checksum for validation
SELECT count(*) AS rows,
       sum(hashtext(t::text)::bigint) AS checksum
FROM app.orders t;

```


Run the equivalent on Oracle with `ORA_HASH` or `DBMS_CRYPTO`, bearing in mind that number and date formatting differences will change the hash — normalise the projection (explicit `TO_CHAR` formats, trimmed strings, scaled numerics) on both sides before comparing.

Appendices

Appendix A — Parameter Quick Reference

Defaults shown are PostgreSQL 18 shipped defaults. "Restart" means the parameter's context is `postmaster`.

A.1 Memory

Parameter	Default	Restart	Typical production
<code>shared_buffers</code>	128MB	Yes	25 % of RAM
<code>effective_cache_size</code>	4GB	No	75 % of RAM
<code>work_mem</code>	4MB	No	16–64MB; per-role overrides for ETL
<code>hash_mem_multiplier</code>	2.0	No	2.0
<code>maintenance_work_mem</code>	64MB	No	1–4GB
<code>autovacuum_work_mem</code>	–1	No	512MB–2GB
<code>temp_buffers</code>	8MB	No	32MB
<code>wal_buffers</code>	–1 (1/32 of <code>shared_buffers</code> , ≤16MB)	Yes	64MB
<code>huge_pages</code>	try	Yes	on (with <code>vm.nr_hugepages</code> set)
<code>logical_decoding_work_mem</code>	64MB	No	256MB–1GB with CDC
<code>max_connections</code>	100	Yes	200–500 behind a pooler

A.2 WAL and checkpoints

Parameter	Default	Restart	Typical
<code>wal_level</code>	replica	Yes	replica, or logical for CDC
<code>wal_compression</code>	off	No	zstd
<code>fsync</code>	on	No	on
<code>synchronous_commit</code>	on	No	on (per-transaction off where safe)
<code>full_page_writes</code>	on	No	on
<code>wal_log_hints</code>	off	Yes	on (needed by <code>pg_rewind</code> without checksums)
<code>min_wal_size</code>	80MB	No	2GB
<code>max_wal_size</code>	1GB	No	8–32GB
<code>checkpoint_timeout</code>	5min	No	15min
<code>checkpoint_completion_target</code>	0.9	No	0.9
<code>archive_mode</code>	off	Yes	on
<code>archive_timeout</code>	0	No	60–300
<code>summarize_wal</code>	off	Yes	on (PG17+ incremental backup)

A.3 Autovacuum

Parameter	Default	Typical
<code>autovacuum</code>	on	on
<code>autovacuum_max_workers</code>	3	4–8 (restart)
<code>autovacuum_naptime</code>	1min	15–30s
<code>autovacuum_vacuum_threshold</code>	50	1000–10000 per table
<code>autovacuum_vacuum_scale_factor</code>	0.2	0.01–0.05 on large tables
<code>autovacuum_vacuum_insert_threshold</code>	1000	as needed
<code>autovacuum_vacuum_insert_scale_factor</code>	0.2	0.05–0.1
<code>autovacuum_vacuum_max_threshold</code>	100000000	PG18+; caps the scale factor
<code>autovacuum_analyze_threshold</code>	50	1000
<code>autovacuum_analyze_scale_factor</code>	0.1	0.01–0.02
<code>autovacuum_vacuum_cost_delay</code>	2ms	0–2ms
<code>autovacuum_vacuum_cost_limit</code>	–1 (→200)	2000–5000
<code>autovacuum_freeze_max_age</code>	200000000	200M–800M
<code>autovacuum_multixact_freeze_max_age</code>	400000000	400M
<code>vacuum_freeze_min_age</code>	50000000	lower for append-only
<code>vacuum_failsafe_age</code>	1600000000	default
<code>log_autovacuum_min_duration</code>	10min	0

A.4 Planner

Parameter	Default	Typical
<code>random_page_cost</code>	4.0	1.1 (NVMe), 1.5 (SSD), 4.0 (spinning)
<code>seq_page_cost</code>	1.0	1.0
<code>effective_io_concurrency</code>	16	200+ on NVMe
<code>maintenance_io_concurrency</code>	16	200+
<code>default_statistics_target</code>	100	100, raised per column
<code>jit</code>	on	off for pure OLTP
<code>jit_above_cost</code>	100000	500000 or <code>jit=off</code>
<code>max_parallel_workers_per_gather</code>	2	4
<code>enable_partitionwise_join</code>	off	on for partitioned analytics
<code>enable_partitionwise_aggregate</code>	off	on for partitioned analytics
<code>plan_cache_mode</code>	auto	<code>force_custom_plan</code> for skewed prepared statements
<code>from_collapse_limit</code> / <code>join_collapse_limit</code>	8	8 (raise only deliberately)
<code>geqo_threshold</code>	12	12

A.5 Logging

Parameter	Default	Typical
<code>logging_collector</code>	off	on
<code>log_destination</code>	stderr	stderr,jsonlog
<code>log_min_duration_statement</code>	−1	500–2000 ms
<code>log_checkpoints</code>	on	on
<code>log_connections</code> / <code>log_disconnections</code>	off	on
<code>log_lock_waits</code>	off	on
<code>log_temp_files</code>	−1	0
<code>log_statement</code>	none	ddl
<code>log_line_prefix</code>	%m [%p]	%m [%p] %q%u@d app=%a %h %e
<code>log_autovacuum_min_duration</code>	10min	0
<code>track_io_timing</code>	off	on
<code>track_wal_io_timing</code>	off	on
<code>track_functions</code>	none	pl
<code>compute_query_id</code>	auto	on

A.6 Timeouts

Parameter	Default	Typical
<code>statement_timeout</code>	0	per-role: 30s app, 0 for ETL
<code>lock_timeout</code>	0	3–10s, and always SET LOCAL before DDL
<code>idle_in_transaction_session_timeout</code>	0	5–10min
<code>idle_session_timeout</code>	0	0 behind a pooler
<code>deadlock_timeout</code>	1s	1s
<code>transaction_timeout</code>	0	PG17+; bounds the whole transaction
<code>authentication_timeout</code>	1min	1min
<code>tcp_keepalives_idle</code> / <code>_interval</code> / <code>_count</code>	OS	60 / 10 / 6

Appendix B — psql

B.1 Meta-commands

Command	Purpose
<code>\l</code> / <code>\l+</code>	List databases
<code>\c dbname [user]</code>	Connect (new connection)
<code>\dn</code> / <code>\dn+</code>	Schemas
<code>\dt</code> / <code>\dt+ schema.*</code>	Tables
<code>\di</code> , <code>\ds</code> , <code>\dv</code> , <code>\dm</code> , <code>\df</code> , <code>\dT</code> , <code>\dx</code>	Indexes, sequences, views, matviews, functions, types, extensions
<code>\d table</code>	Describe (columns, indexes, constraints, triggers, partitions)

Command	Purpose
<code>\d+ table</code>	Adds storage, stats target, description, partition bounds
<code>\dp / \z</code>	Table privileges
<code>\ddp</code>	Default privileges
<code>\du / \dg</code>	Roles
<code>\dconfig</code>	Non-default parameters (PG15+)
<code>\dP</code>	Partitioned tables and indexes
<code>\sf function</code>	Show function source
<code>\ef function</code>	Edit function in \$EDITOR
<code>\timing on</code>	Report execution time
<code>\x auto</code>	Expanded output when rows are wide
<code>\watch 5</code>	Re-run the last query every 5 s
<code>\copy tbl FROM 'f.csv' CSV HEADER</code>	Client-side COPY (no server file access needed)
<code>\i file.sql / \ir file.sql</code>	Execute a script (relative to the current file)
<code>\o out.txt</code>	Redirect output
<code>\e</code>	Edit the query buffer
<code>\gexec</code>	Execute each row of the result as SQL
<code>\gdesc</code>	Describe the result columns without running it
<code>\conninfo</code>	Current connection details
<code>\errverbose</code>	Full detail of the last error
<code>\set ON_ERROR_STOP on</code>	Abort a script on the first error — always use in automation
<code>\pset null '(null)'</code>	Make NULLs visible
<code>\! command</code>	Shell escape
<code>\q</code>	Quit

B.2 A useful ~/.psqlrc

```

\set QUIET 1
\timing on
\x auto
\set ON_ERROR_ROLLBACK interactive
\set COMP_KEYWORD_CASE upper
\set HISTFILE ~/.psql_history- :DBNAME
\set HISTSIZE 5000
\set VERBOSITY verbose
\pset null '␣'
\pset linestyle unicode
\pset border 2
\setenv PAGER 'less -SFX'
\set PROMPT1 '%[%033[1;33m%]%n@%/%R[%033[0m%]%# '
\set QUIET 0

```

B.3 SQLPlus → psql

SQL*Plus	psql
DESC table	<code>\d table</code>
SET LINESIZE/PAGESIZE	<code>\pset</code>

SQL*Plus	psql
SET AUTOTRACE ON	EXPLAIN (ANALYZE, BUFFERS)
SPOOL file	\o file
@script.sql	\i script.sql
SHOW PARAMETER x	SHOW x / SELECT * FROM pg_settings WHERE name LIKE '%x%'
SHOW ERRORS	\errverbose
CONNECT user/pass@db	\c db user
EXIT/QUIT	\q
WHENEVER SQLERROR EXIT	\set ON_ERROR_STOP on
HOST cmd	\! cmd
SET SERVEROUTPUT ON	(nothing needed; RAISE NOTICE prints)

B.4 Command-line utilities

Utility	Purpose
psql	Interactive terminal
pg_dump, pg_dumpall, pg_restore	Logical backup/restore
pg_basebackup	Physical base backup
pg_combinebackup	PG17+: combine full + incremental
pg_verifybackup	Verify a base backup manifest
pg_ctl	Start/stop/reload/promote
initdb	Create a cluster
pg_upgrade	Major version upgrade
pg_rewind	Resynchronise a diverged former primary
pg_controldata	Dump the control file
pg_resetwal	Destructive last-resort control file reset
pg_waldump	Decode WAL segments (excellent for forensics)
pg_checksums	Enable/disable/verify checksums offline
pg_amcheck	Verify heap and B-tree consistency
vacuumdb, reindexdb, clusterdb	Maintenance drivers
createdb, dropdb, createuser, dropuser	Wrappers
pgbench	Built-in benchmark and load generator
pg_isready	Connection readiness check for scripts and health probes
pg_receivewal	Stream WAL to a file archive
pg_recvlogical	Consume a logical replication slot from the shell
pg_test_fsync, pg_test_timing	Storage and clock micro-benchmarks

Appendix C — Oracle View → PostgreSQL Catalog Cross-Reference

Oracle	PostgreSQL
V\$SESSION	pg_stat_activity
V\$SESSION_WAIT, V\$SESSION_EVENT	pg_stat_activity.wait_event_type/wait_event (+ pg_wait_sampling)
V\$ACTIVE_SESSION_HISTORY	none in core (pgsentinel, sampling into a table)
V\$SQL, V\$SQLAREA, V\$SQLSTATS	pg_stat_statements
V\$SQL_PLAN	EXPLAIN, auto_explain
V\$LOCK, DBA_BLOCKERS, DBA_WAITERS	pg_locks, pg_blocking_pids()
V\$TRANSACTION	pg_stat_activity.backend_xid/backend_xmin, pg_prepared_xacts
V\$DATABASE	pg_control_system(), pg_database
V\$INSTANCE	pg_postmaster_start_time(), version()
V\$PARAMETER	pg_settings, pg_file_settings
V\$SGASTAT, V\$PGASTAT	pg_bufferscache, pg_shmem_allocations
V\$BUFFER_POOL_STATISTICS	pg_stat_database, pg_statio_*, pg_stat_io
V\$FILESTAT, V\$IOSTAT_FILE	pg_stat_io (PG16+), pg_statio_user_tables
V\$LOG, V\$LOGFILE, V\$LOG_HISTORY	pg_ls_waldir(), pg_stat_wal, pg_stat_archiver
V\$ARCHIVED_LOG	pg_stat_archiver, backup tool catalog
V\$DATAGUARD_STATS	pg_stat_replication, pg_stat_wal_receiver
V\$RECOVERY_PROGRESS	pg_stat_progress_basebackup, pg_last_wal_replay_lsn()
DBA_TABLES	pg_class / pg_tables / pg_stat_user_tables
DBA_TAB_COLUMNS	information_schema.columns, pg_attribute
DBA_INDEXES, DBA_IND_COLUMNS	pg_indexes, pg_index, pg_stat_user_indexes
DBA_CONSTRAINTS	pg_constraint, information_schema.table_constraints
DBA_TAB_PARTITIONS	pg_partition_tree(), pg_inherits, pg_class.relpartbound
DBA_SEGMENTS, DBA_EXTENTS	pg_relation_size(), pg_total_relation_size() (no extent view)
DBA_FREE_SPACE	pg_freespace() (pg_freespacemap extension)
DBA_TAB_STATISTICS, DBA_TAB_HISTOGRAMS	pg_stats, pg_statistic, pg_stats_ext
DBA_USERS, DBA_ROLES	pg_roles, pg_authid
DBA_TAB_PRIVS, DBA_SYS_PRIVS	information_schema.role_table_grants, has_*_privilege()
DBA_OBJECTS	pg_class + pg_proc + pg_type + ... (no single view)
DBA_SOURCE	pg_get_functiondef(), pg_proc.prosrc
DBA_DEPENDENCIES	pg_depend, pg_shdepend
DBA_SCHEDULER_JOBS	cron.job (pg_cron)
DBA_HIST_* (AWR)	none in core; snapshot pg_stat_* yourself
DBA_AUDIT_TRAIL, UNIFIED_AUDIT_TRAIL	pgaudit output in the server log
DBMS_XPLAN.DISPLAY_CURSOR	EXPLAIN (ANALYZE, BUFFERS), auto_explain
DBMS_STATS.GATHER_TABLE_STATS	ANALYZE
DBMS_SPACE.SPACE_USAGE	pgstattuple()
UTL_FILE	COPY ... TO/FROM (server-side, superuser), \copy (client-side), orafce

Oracle	PostgreSQL
DBMS_LOCK	pg_advisory_lock() family
DBMS_ALERT / AQ	LISTEN/NOTIFY

Appendix D — Snippet Library

```
-- ----- Sizes -----
SELECT pg_size_pretty(pg_database_size(current_database()));
SELECT n.nspname||'.'||c.relname AS relation,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total,
       pg_size_pretty(pg_relation_size(c.oid))      AS heap,
       pg_size_pretty(pg_indexes_size(c.oid))       AS indexes,
       pg_size_pretty(pg_total_relation_size(reltoastrelid)) AS toast
FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r','p','m')
ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 25;

-- ----- Every table's row-count estimate without counting -----
SELECT relname, reltuples::bigint AS est_rows, relpages
FROM pg_class WHERE relkind = 'r' ORDER BY reltuples DESC LIMIT 25;

-- ----- Missing foreign key indexes -----
SELECT c.conrelid::regclass AS table, c.conname, a.attname AS unindexed_fk_column
FROM pg_constraint c
CROSS JOIN LATERAL unnest(c.conkey) k(attnum)
JOIN pg_attribute a ON a.attrelid = c.conrelid AND a.attnum = k.attnum
WHERE c.contype = 'f'
      AND NOT EXISTS (
        SELECT 1 FROM pg_index i
        WHERE i.indrelid = c.conrelid
              AND (i.indkey::smallint[])[0:cardinality(c.conkey)-1] @> c.conkey)
ORDER BY 1;

-- ----- Sequences approaching their limit -----
SELECT schemaname||'.'||sequencename AS seq, last_value, max_value,
       round(100.0*last_value/max_value, 2) AS pct_used
FROM pg_sequences WHERE last_value IS NOT NULL ORDER BY 4 DESC NULLS LAST LIMIT 20;

-- ----- Integer primary keys at risk of overflow -----
SELECT c.relname, a.attname, pg_get_serial_sequence(c.oid::regclass::text, a.attname) AS seq
FROM pg_class c JOIN pg_attribute a ON a.attrelid = c.oid
JOIN pg_type t ON t.oid = a.atttypid
WHERE c.relkind = 'r' AND t.typname = 'int4' AND a.attnum > 0
      AND pg_get_serial_sequence(c.oid::regclass::text, a.attname) IS NOT NULL;

-- ----- Table definition as DDL -----
SELECT pg_get_tabledef FROM (SELECT 1) x; -- not built in; use: pg_dump -s -t app.orders

-- ----- Generate GRANTS for every table in a schema -----
SELECT format('GRANT SELECT ON %I.%I TO app_readonly;', schemaname, tablename)
FROM pg_tables WHERE schemaname = 'app' \gexec

-- ----- Kill all idle-in-transaction sessions older than 10 minutes -----
SELECT pg_terminate_backend(pid), username, application_name, now()-xact_start AS age
FROM pg_stat_activity
WHERE state = 'idle in transaction' AND xact_start < now() - interval '10 minutes';

-- ----- Reset all statistics -----
SELECT pg_stat_reset();
SELECT pg_stat_statements_reset();

-- ----- Cluster-wide settings audit -----
SELECT name, setting, unit, boot_val, reset_val, source, sourcefile, sourceline
FROM pg_settings WHERE source NOT IN ('default','override') ORDER BY name;

-- ----- Objects a role owns, in this database -----
SELECT c.relkind, n.nspname, c.relname
```

```

FROM pg_class c JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE c.relowner = 'app_owner'::regrole ORDER BY 1,2,3;

-- ----- Find the query behind a queryid seen in the logs -----
SELECT query FROM pg_stat_statements WHERE queryid = -1234567890123456789;

-- ----- Safe pattern for any production DDL -----
DO $$
BEGIN
    PERFORM set_config('lock_timeout', '3s', true);
    EXECUTE 'ALTER TABLE app.orders ADD COLUMN note text';
EXCEPTION WHEN lock_not_available THEN
    RAISE NOTICE 'could not acquire lock; retry later';
END $$;

-- ----- Batched delete that does not bloat or block -----
DO $$
DECLARE n int;
BEGIN
    LOOP
        DELETE FROM app.events
        WHERE ctid IN (SELECT ctid FROM app.events
                       WHERE created_at < now() - interval '13 months' LIMIT 10000);
        GET DIAGNOSTICS n = ROW_COUNT;
        EXIT WHEN n = 0;
        COMMIT;
        PERFORM pg_sleep(0.05);
    END LOOP;
END $$;

```

Appendix E — Version Feature Matrix

Features an administrator is likely to depend on, by the version that introduced them.

Version	Features that change how you operate
14	idle_session_timeout; pg_stat_progress_copy; REINDEX on partitioned tables; LZ4 TOAST compression; predefined pg_read_all_data/pg_write_all_data; Memoize node; reduced B-tree bloat from index deduplication improvements; ALTER TABLE ... DETACH PARTITION CONCURRENTLY
15	MERGE; public schema no longer world-writable; server-side compression in pg_basebackup (lz4/zstd); wal_compression = lz4/zstd; jsonlog; row filters and column lists in logical replication; GRANT ... ON PARAMETER; pg_stat_subscription_stats; stats collector moved to shared memory; archive_library
16	pg_stat_io; logical decoding on a standby; CREATEROLE restricted to roles it created; bidirectional-friendly origin = none; reserved_connections; parallel FULL/RIGHT hash joins; pg_ident_file_mappings; last_idx_scan in index stats; VACUUM (BUFFER_USAGE_LIMIT)
17	Incremental base backups + pg_combinebackup; pg_createsubscriber; failover slots (failover = true, sync_replication_slots); vastly more memory-efficient vacuum TID store; MAINTAIN privilege and pg_maintain role; pg_stat_checkpoint; transaction_timeout; EXPLAIN (SERIALIZE, MEMORY); COPY ... ON_ERROR ignore; sslnegotiation=direct; JSON_TABLE

Version	Features that change how you operate
18	Asynchronous I/O (<code>io_method = worker</code> by default, <code>io_uring</code> available); data checksums on by default ; B-tree skip scan; <code>uuidv7()</code> (and <code>uuidv4()</code> as an alias for <code>gen_random_uuid()</code>); virtual generated columns; OAuth 2.0 authentication; <code>pg_upgrade --swap</code> , parallel checks, and column statistics carried across major upgrades (extended statistics are not); temporal <code>PRIMARY KEY ... WITHOUT OVERLAPS</code> and <code>FOREIGN KEY ... PERIOD</code> ; OLD/NEW in <code>RETURNING</code> ; <code>autovacuum_vacuum_max_threshold</code> ; per-connection I/O and WAL statistics; buffer counts in <code>EXPLAIN ANALYZE</code> by default; <code>idle_replication_slot_timeout</code> ; <code>PG_UNICODE_FAST</code> collation; protocol 3.2; MD5 authentication deprecated
19 (beta at time of writing; expected Sept/Oct 2026)	Auto-scaling AIO workers (<code>io_min_workers/io_max_workers</code>); parallel autovacuum (<code>autovacuum_max_parallel_workers</code>); <code>REPACK</code> command with <code>CONCURRENTLY</code> ; <code>ALTER TABLE ... MERGE/SPLIT PARTITIONS</code> ; sequence replication in logical replication; <code>CREATE PUBLICATION ... EXCEPT</code> ; logical replication enabled without restart; <code>pg_stat_lock</code> and <code>pg_stat_recovery</code> ; <code>WAIT FOR LSN</code> ; SQL/PGQ property graphs; <code>GROUP BY ALL</code> ; <code>INSERT ... ON CONFLICT DO SELECT</code> ; server-side SNI via <code>pg_hosts.conf</code> ; password expiration warnings

Verify anything version-specific against the release notes for your exact minor release before relying on it in production.

Appendix F — Further Reading

- PostgreSQL documentation — <https://www.postgresql.org/docs/> (the reference manual is genuinely good; read the "Server Administration" part end to end once)
- Release notes — <https://www.postgresql.org/docs/release/>
- Versioning and EOL policy — <https://www.postgresql.org/support/versioning/>
- The wiki, especially the "Don't Do This" page — https://wiki.postgresql.org/wiki/Don%27t_Do_This
- pgBackRest user guide — <https://pgbackrest.org/user-guide.html>
- Patroni documentation — <https://patroni.readthedocs.io/>
- `pg_stat_statements`, `auto_explain`, `pgaudit`, `pg_partman` documentation
- *PostgreSQL 14 Internals* (Egor Rogov) — free PDF; the best explanation of MVCC, vacuum, buffer management, and the planner available anywhere
- `pgsql-hackers` and `pgsql-general` mailing lists; the annual PGConf and FOSDEM PGDay talks